

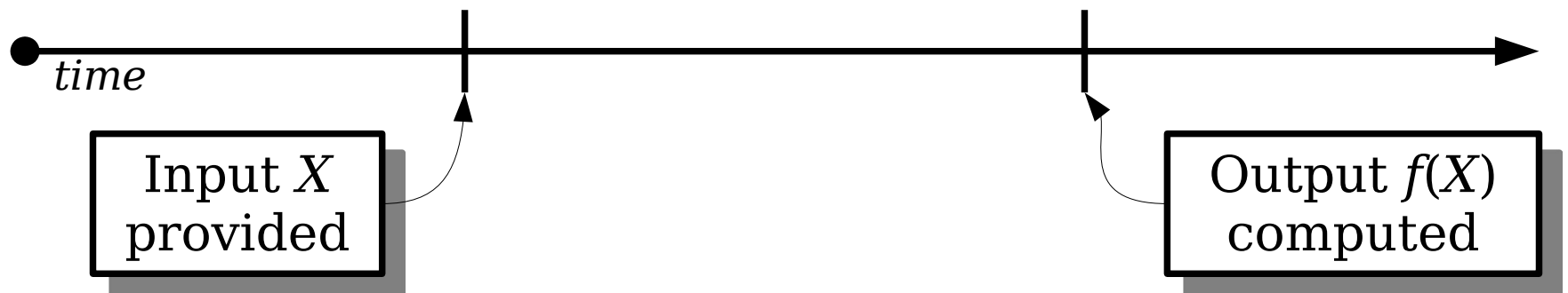
Dynamic Problems

Classical Algorithms

- The “classical” algorithms model goes something like this:

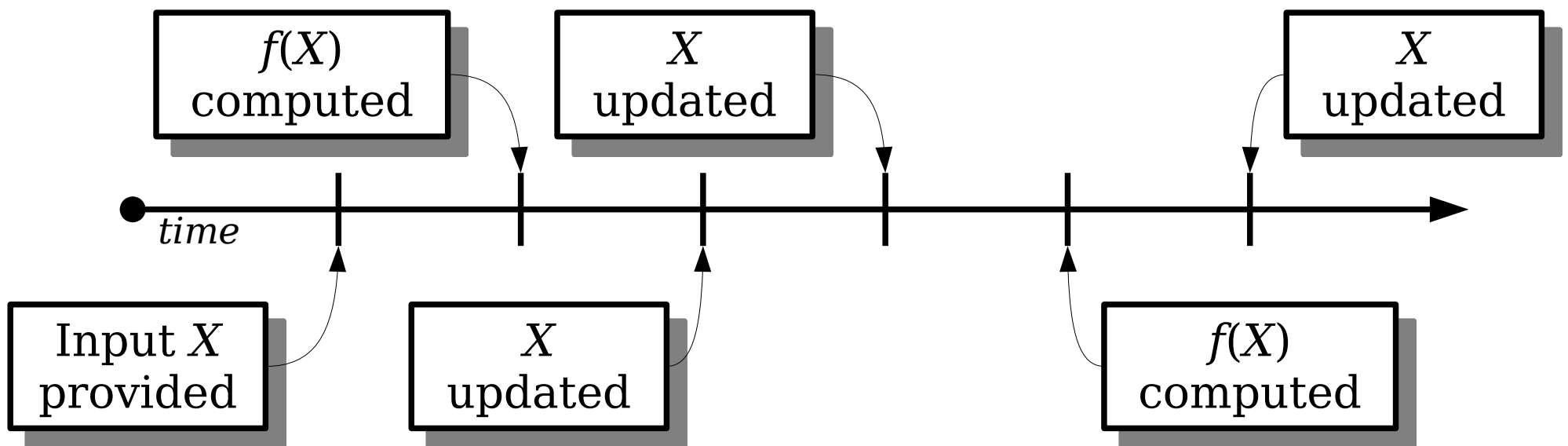
Given some input X , compute some interesting function $f(X)$.

- The input X is provided up front, and only a single answer is produced.



Dynamic Problems

- **Dynamic versions** of problems are framed like this:
Given an input X that can change in fixed ways, maintain X while being able to compute $f(X)$ efficiently at any point in time.
- These problems are typically harder to solve efficiently than the “classical” static versions.



Dynamic Selection

- The **selection** problem is the following:
Given a list of distinct values and a number k , return the k th-smallest value.
- In the static case, where the data set is fixed in advance and k is known, we can solve this in time $O(n)$ using quickselect or the median-of-medians algorithm.
- **Goal:** Solve this problem efficiently when the data set is changing - that is, the underlying set of elements can have insertions and deletions intermixed with queries.

31

41

59

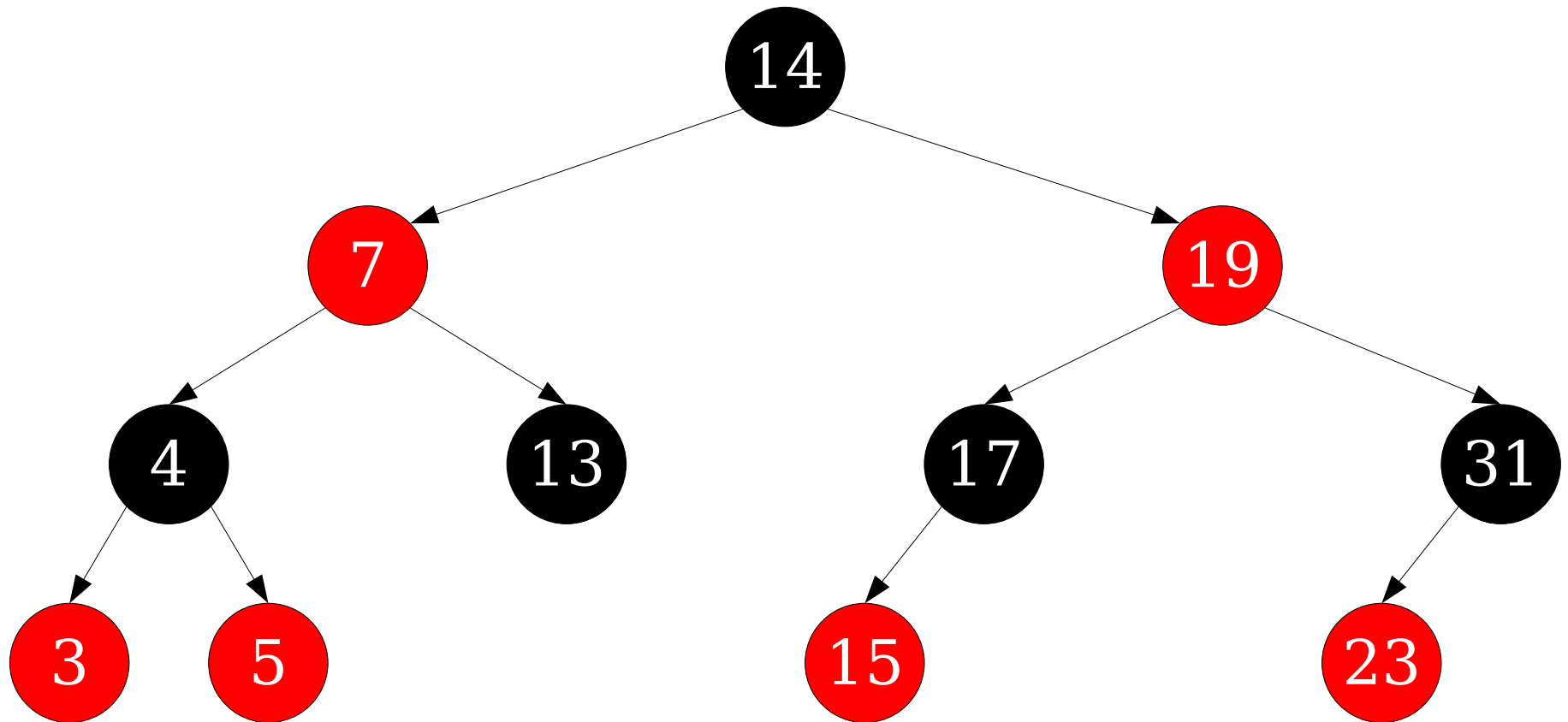
26

53

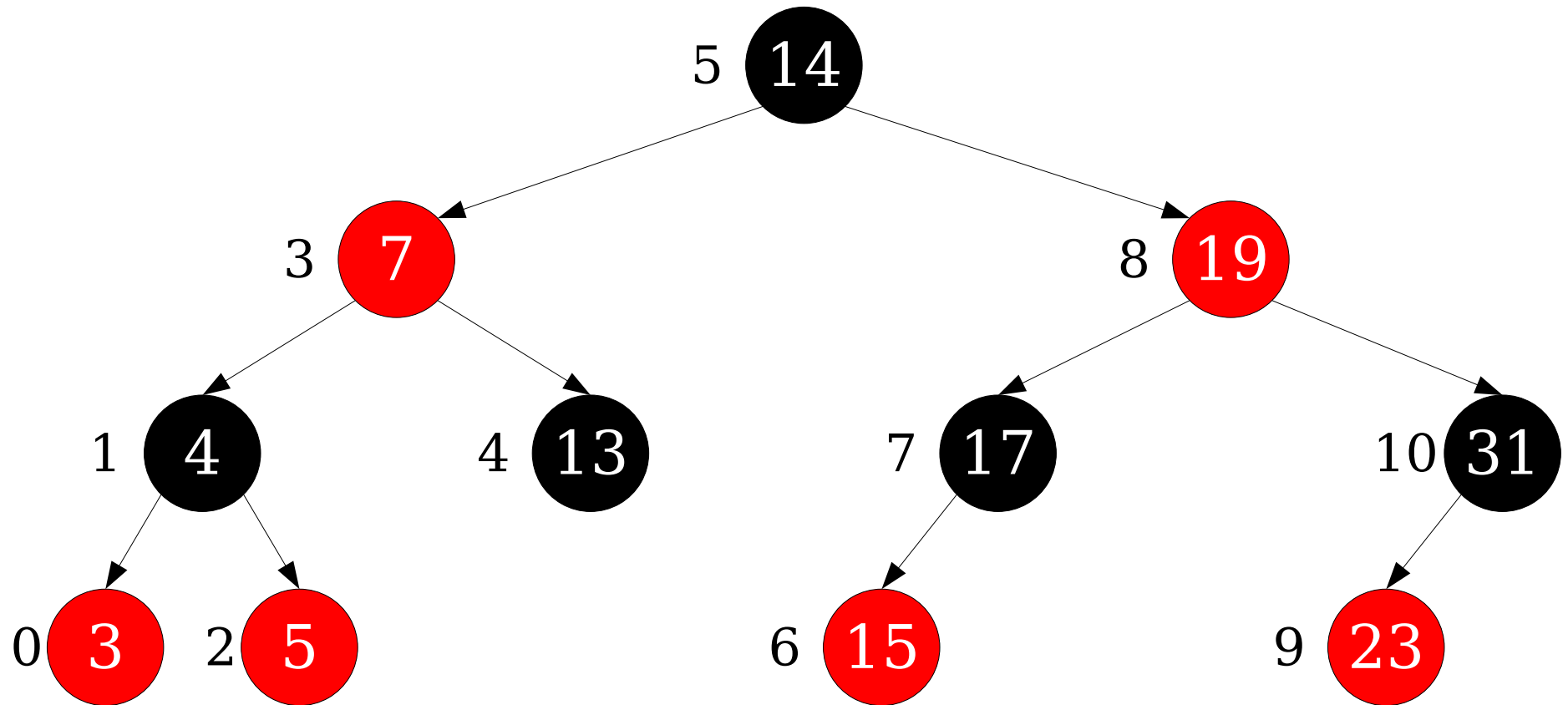
58

79

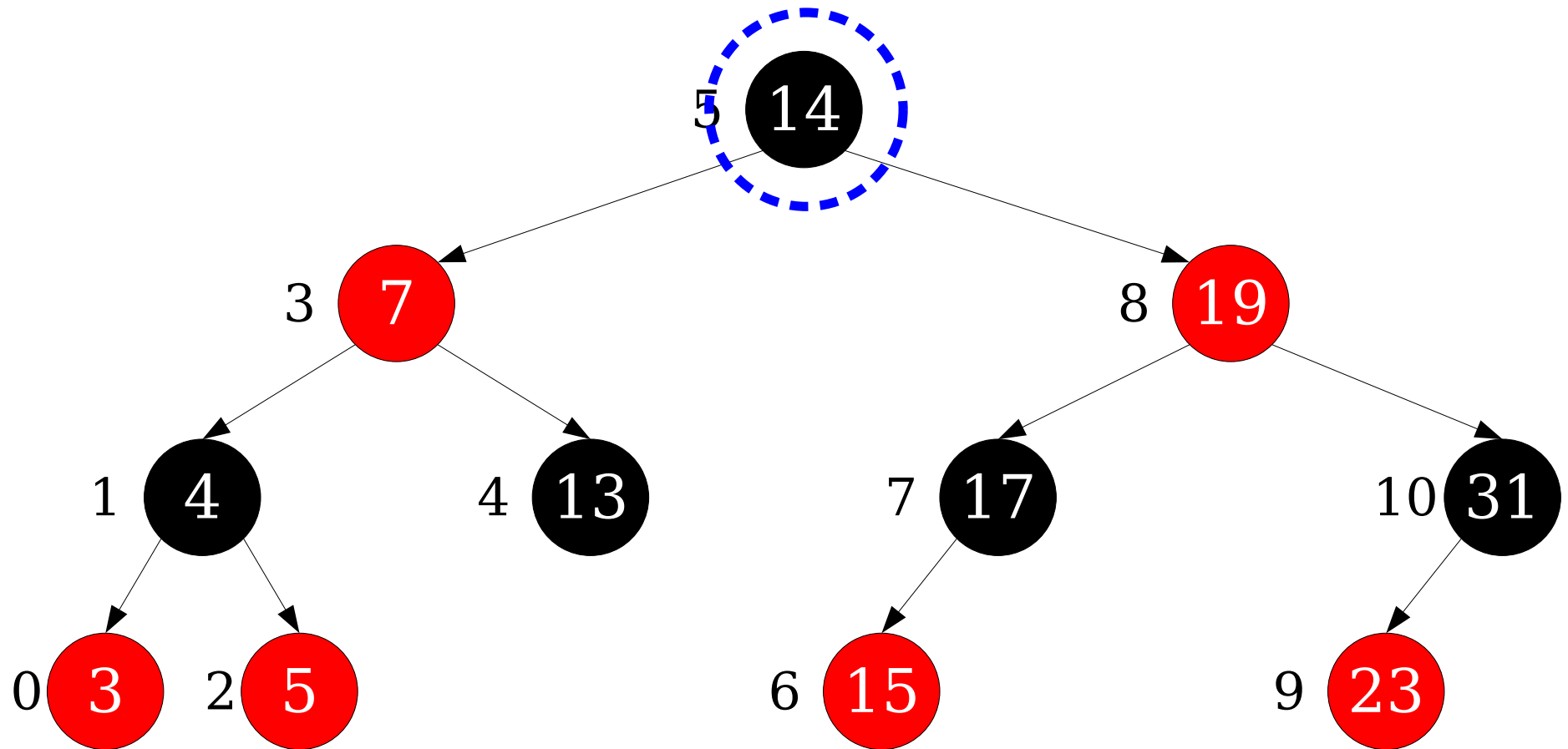
Dynamic Selection



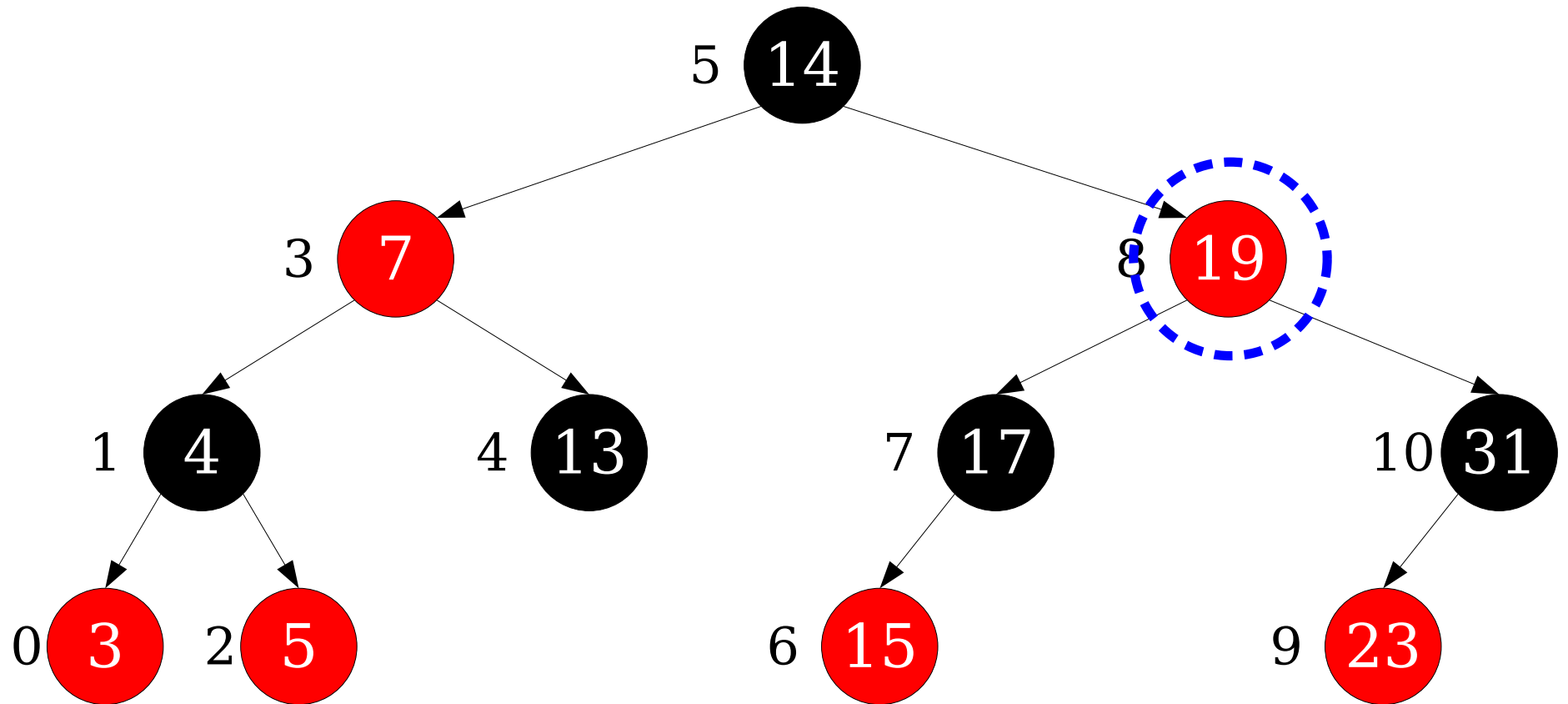
Dynamic Selection



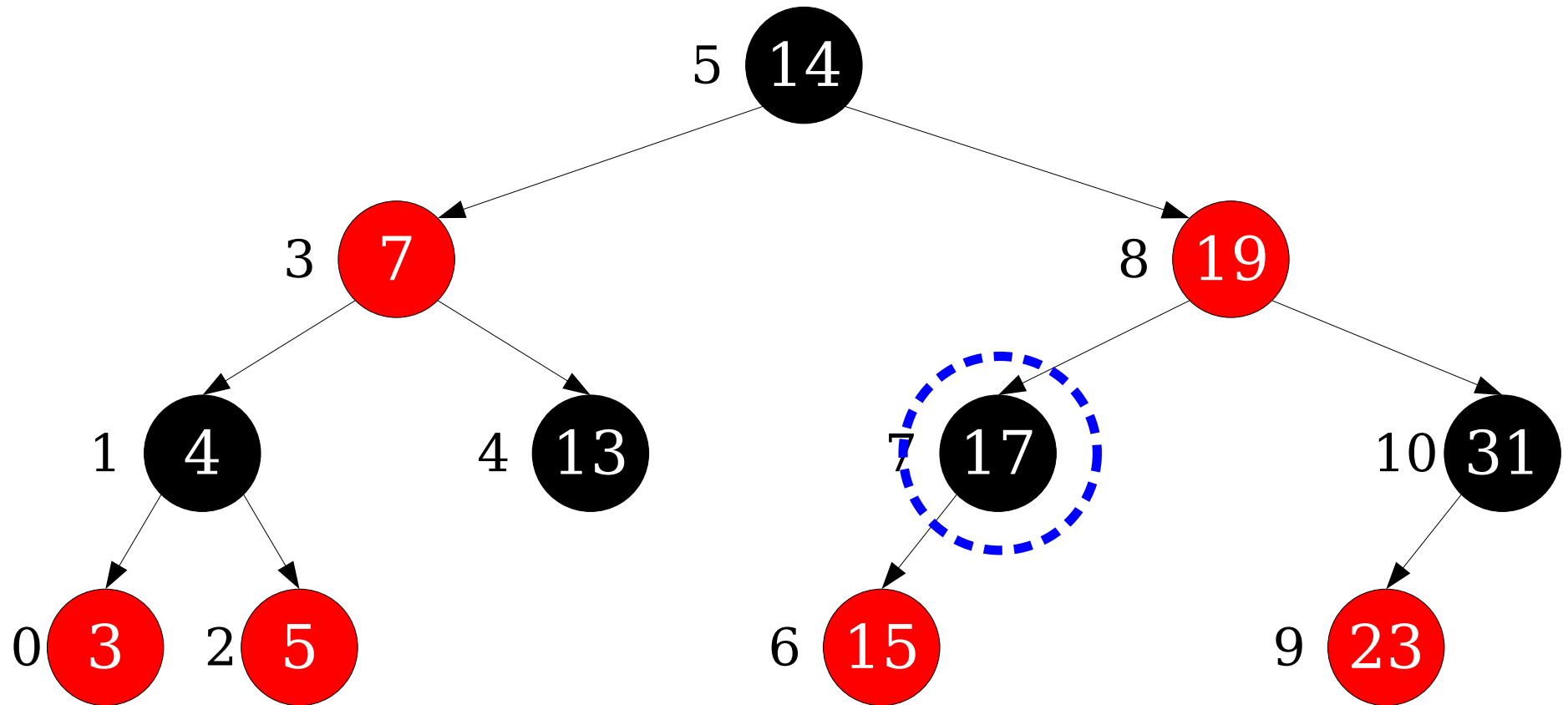
Dynamic Selection



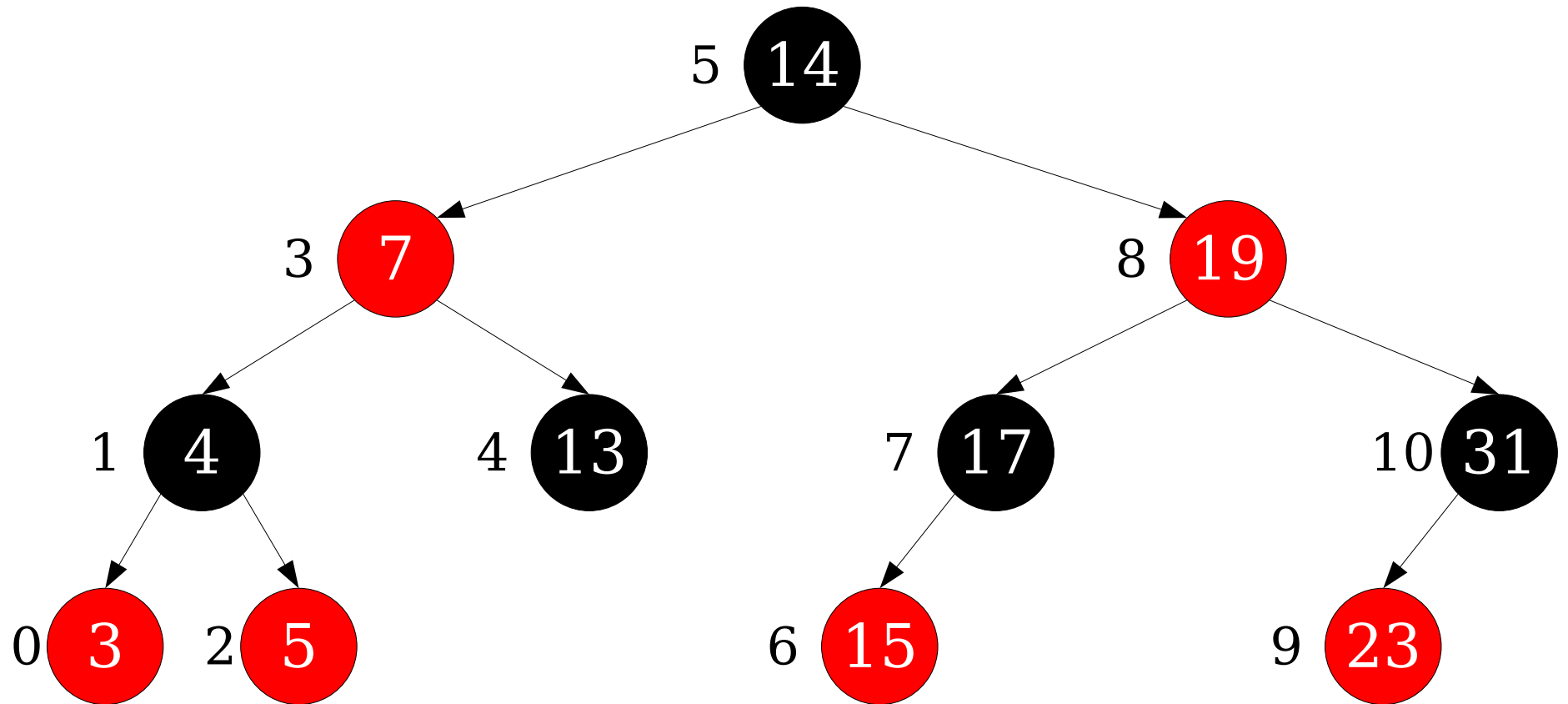
Dynamic Selection



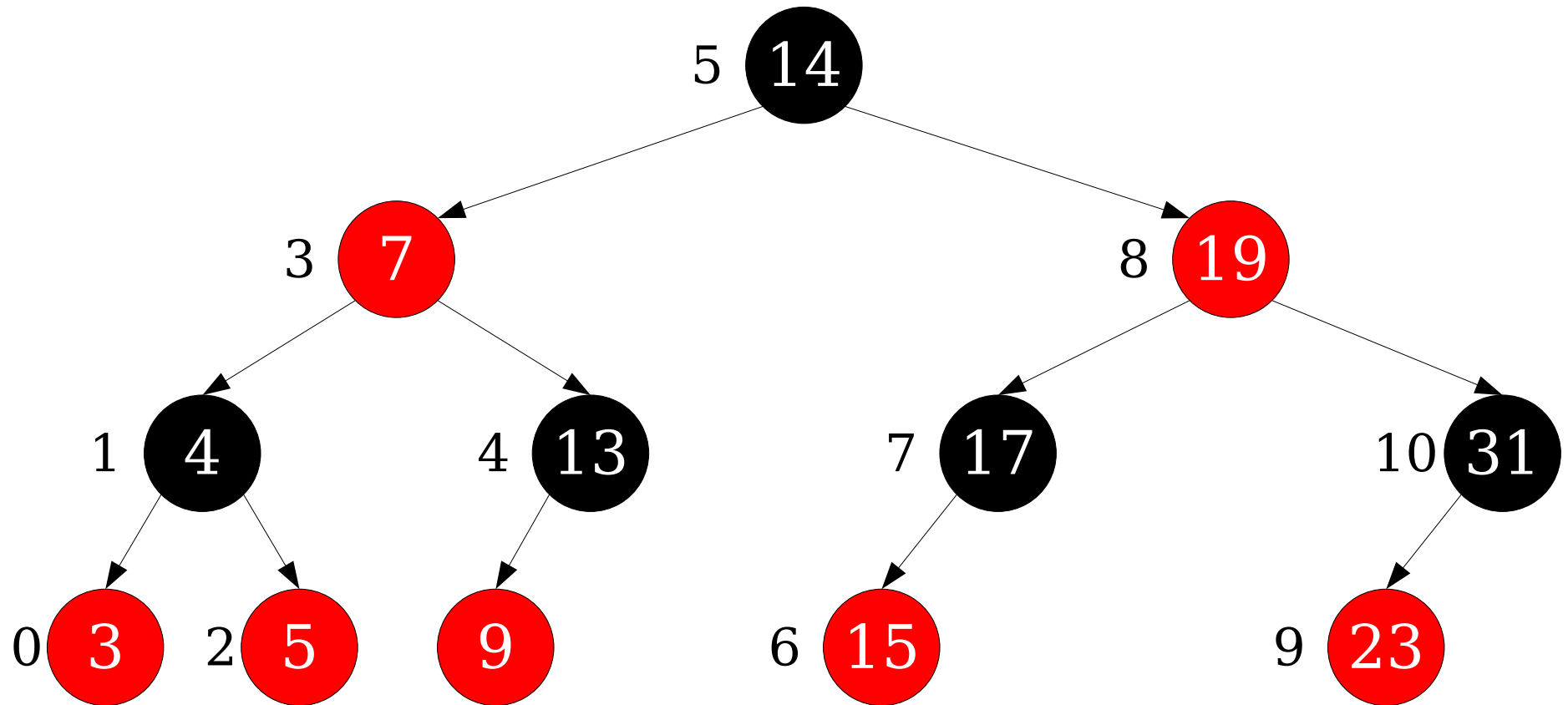
Dynamic Selection



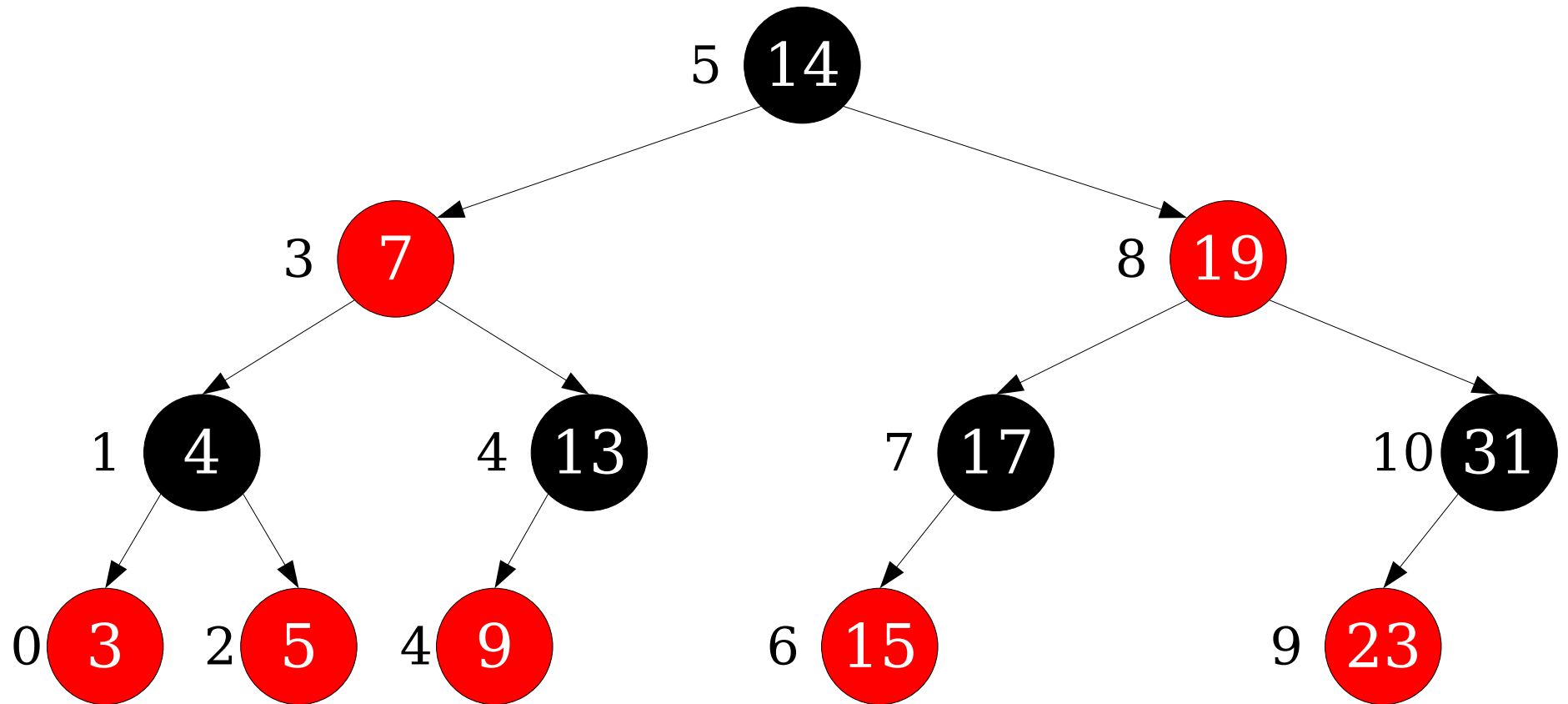
Dynamic Selection



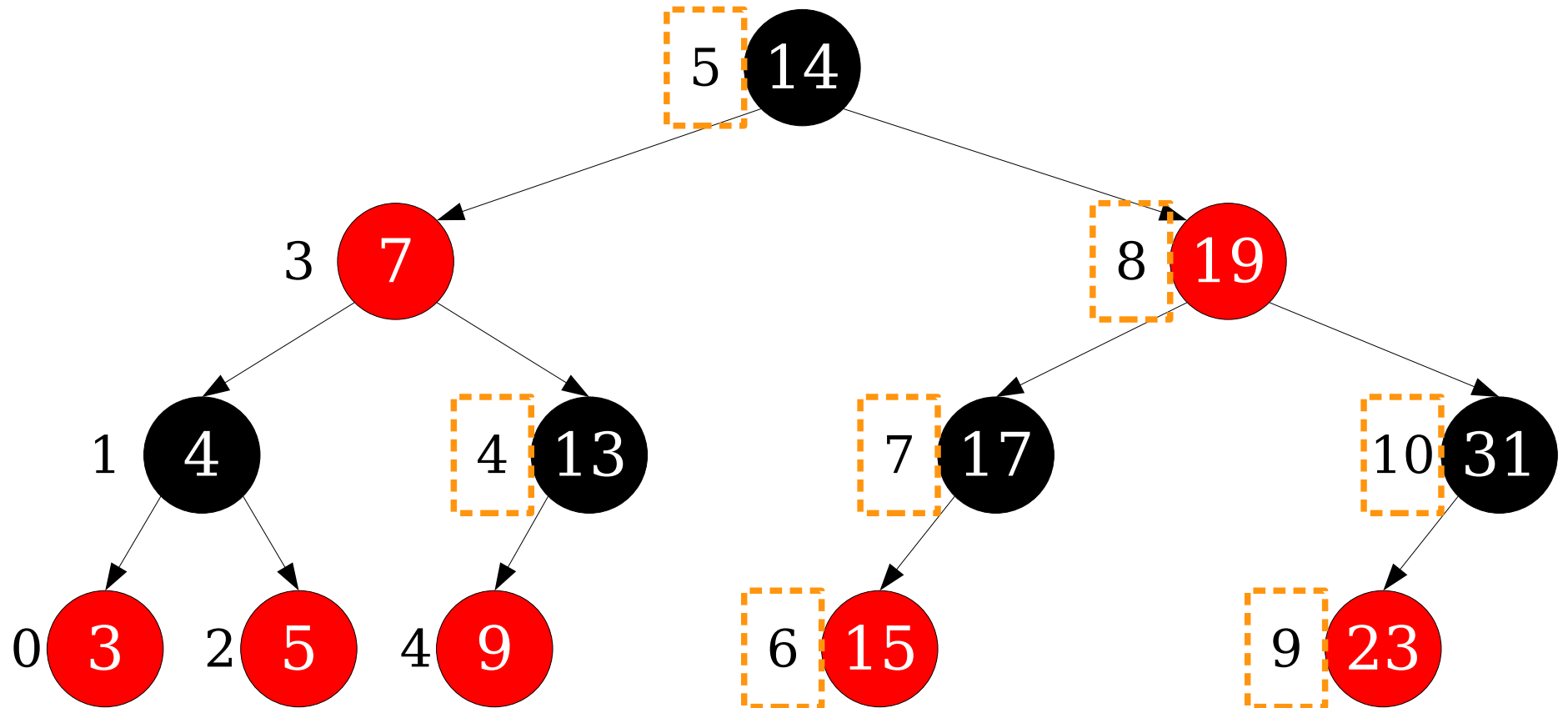
Dynamic Selection



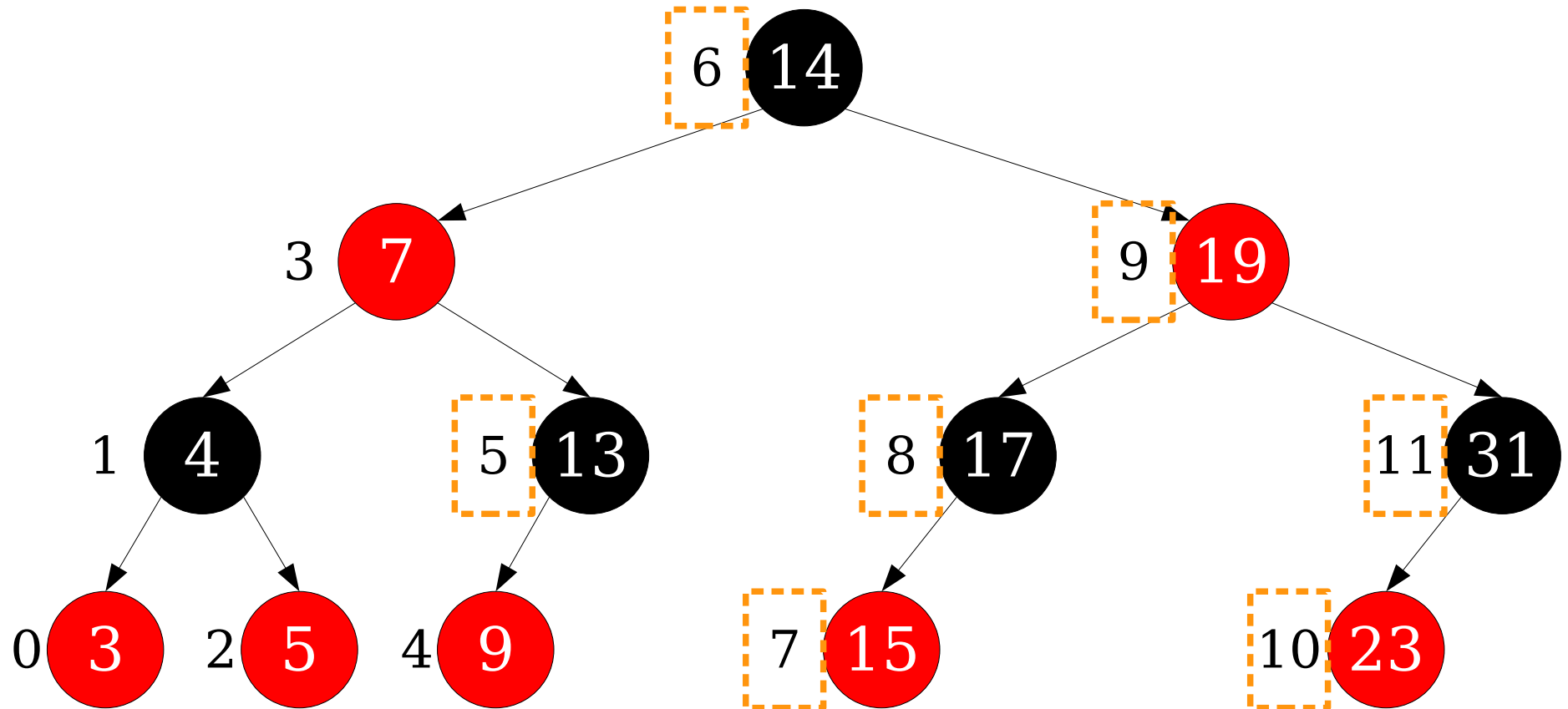
Dynamic Selection



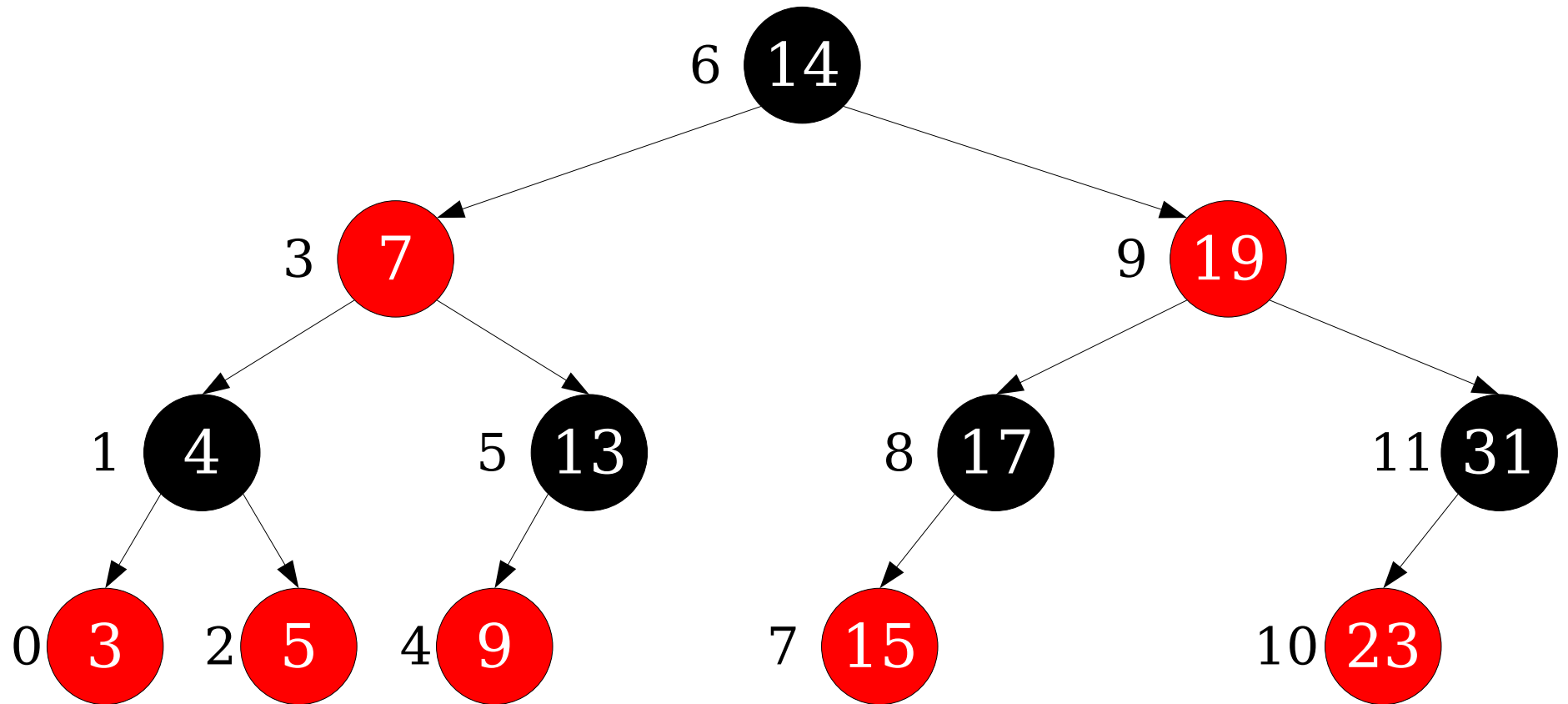
Dynamic Selection



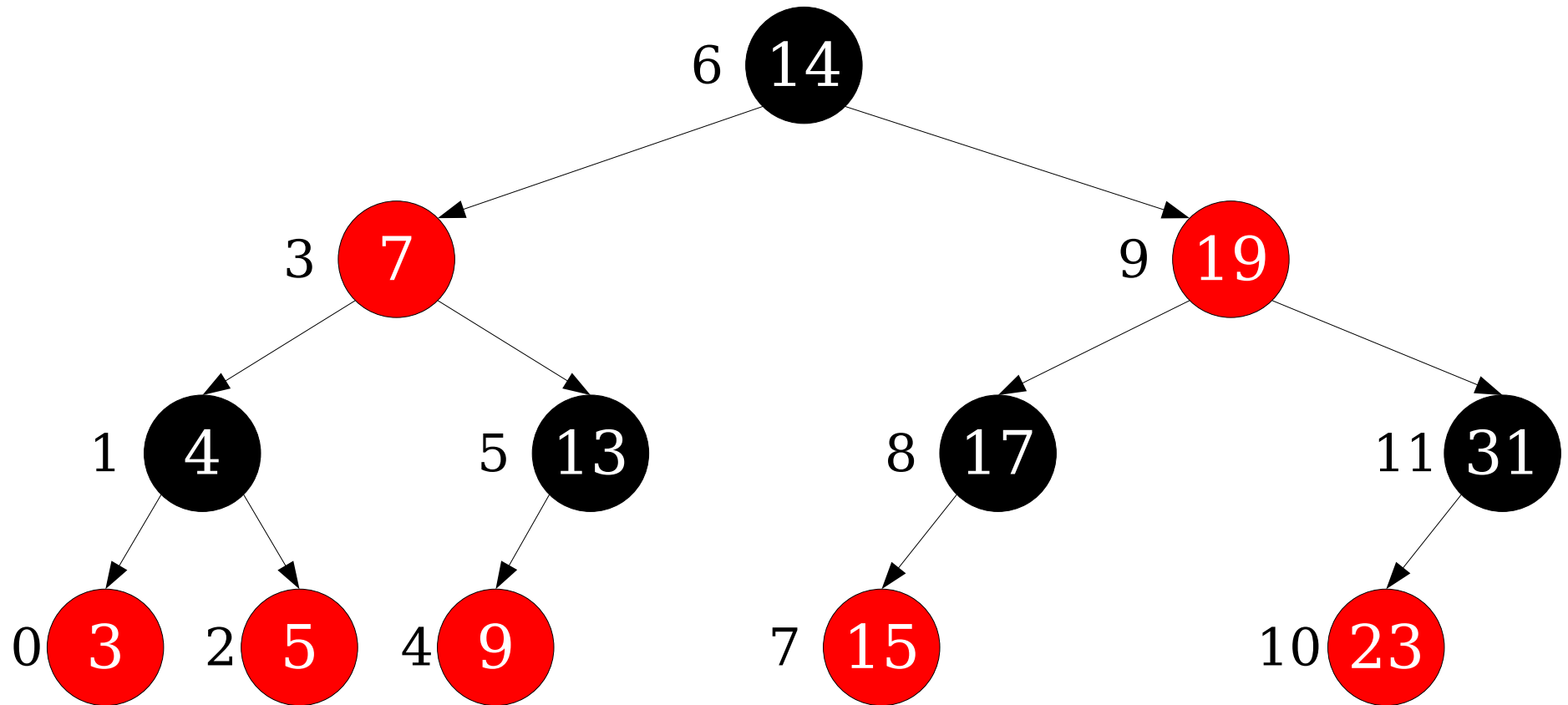
Dynamic Selection



Dynamic Selection



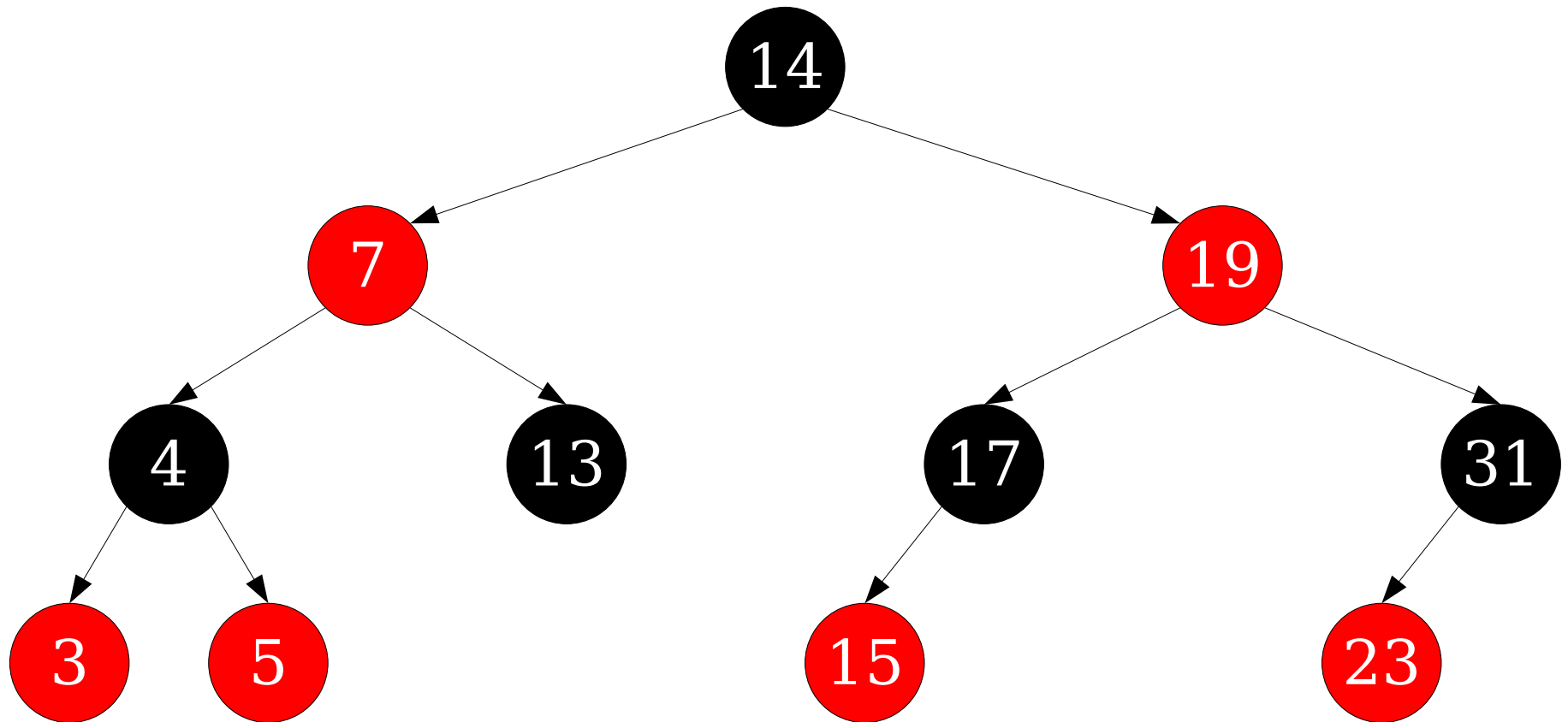
Dynamic Selection



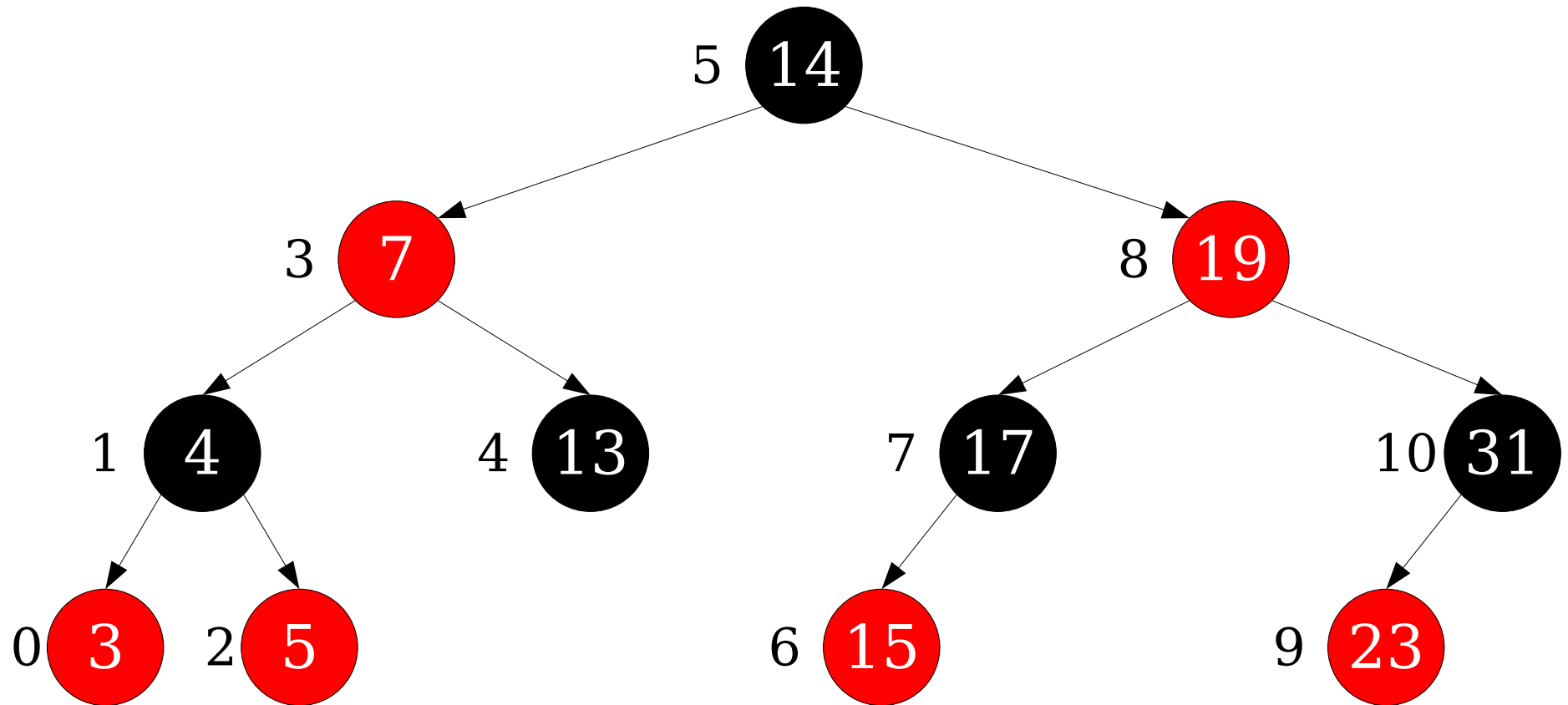
Problem: After inserting a new value, we may have to update $\Theta(n)$ values.

This is inherent in this solution route. These numbers track *global* properties of the tree.

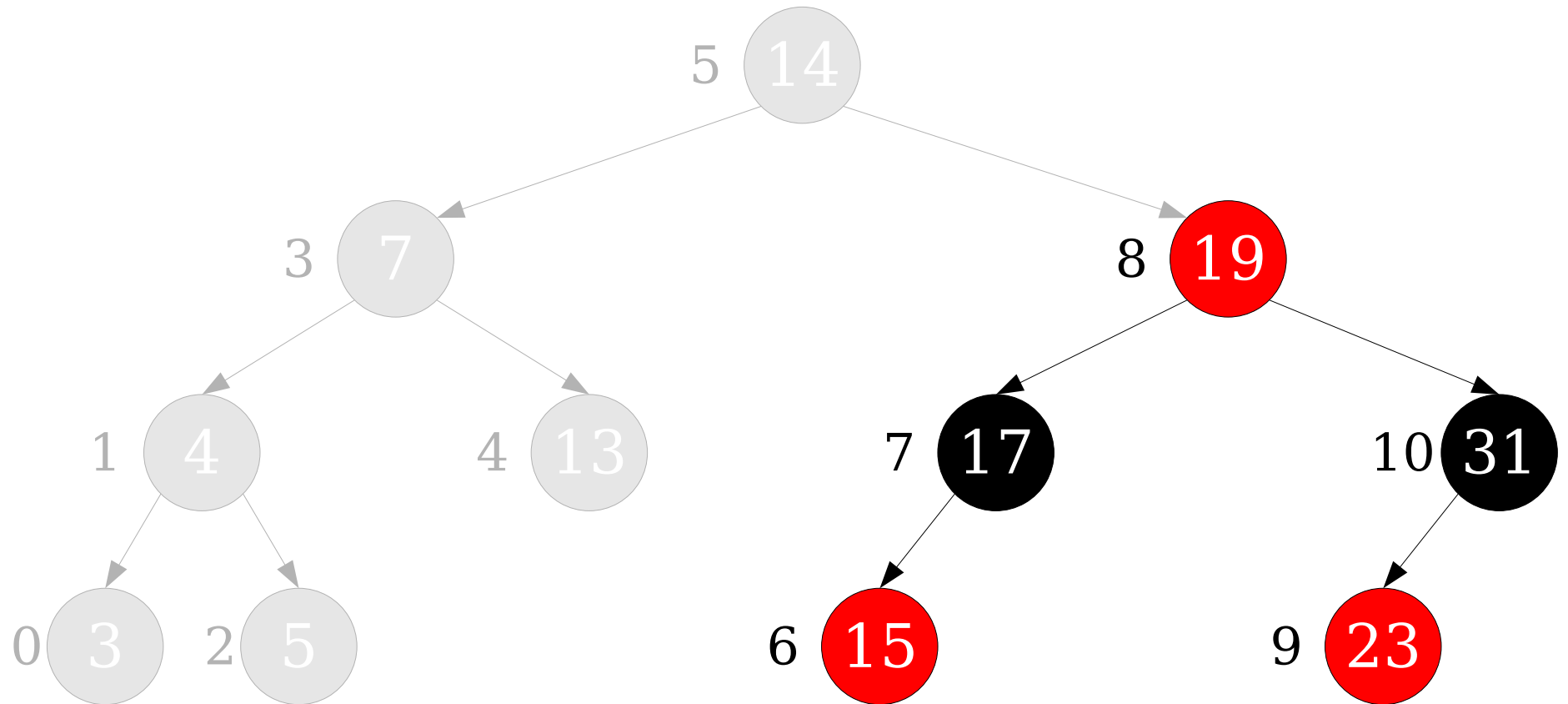
Dynamic Selection



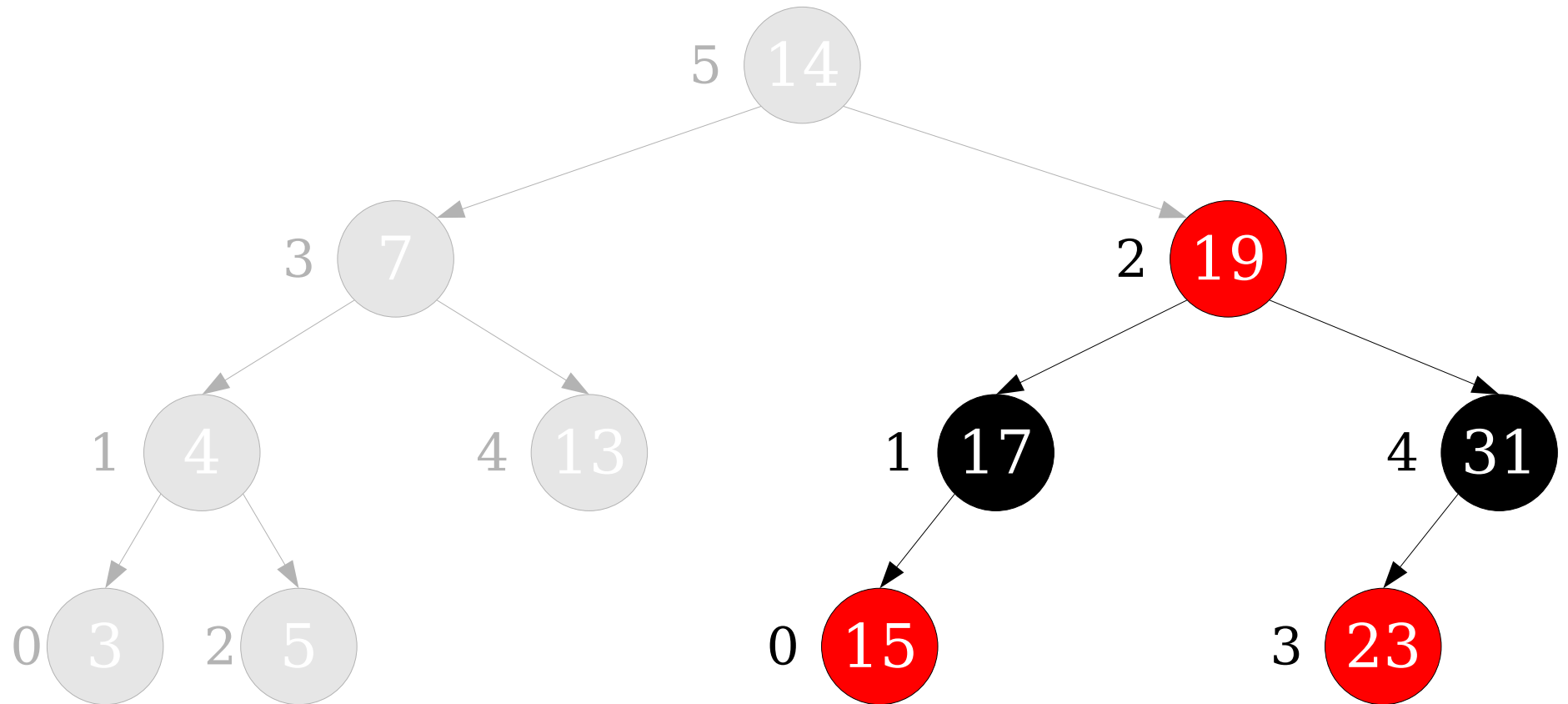
Dynamic Selection



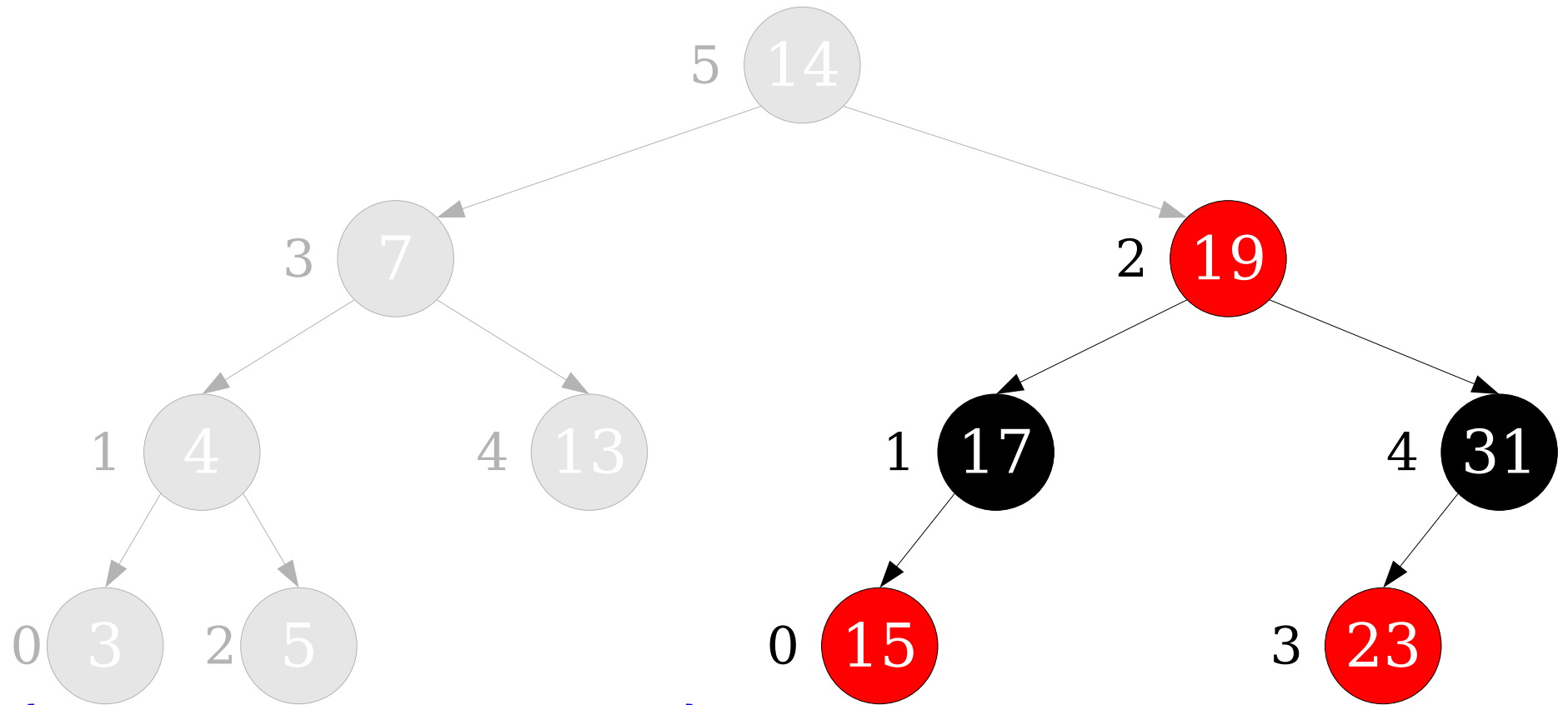
Dynamic Selection



Dynamic Selection

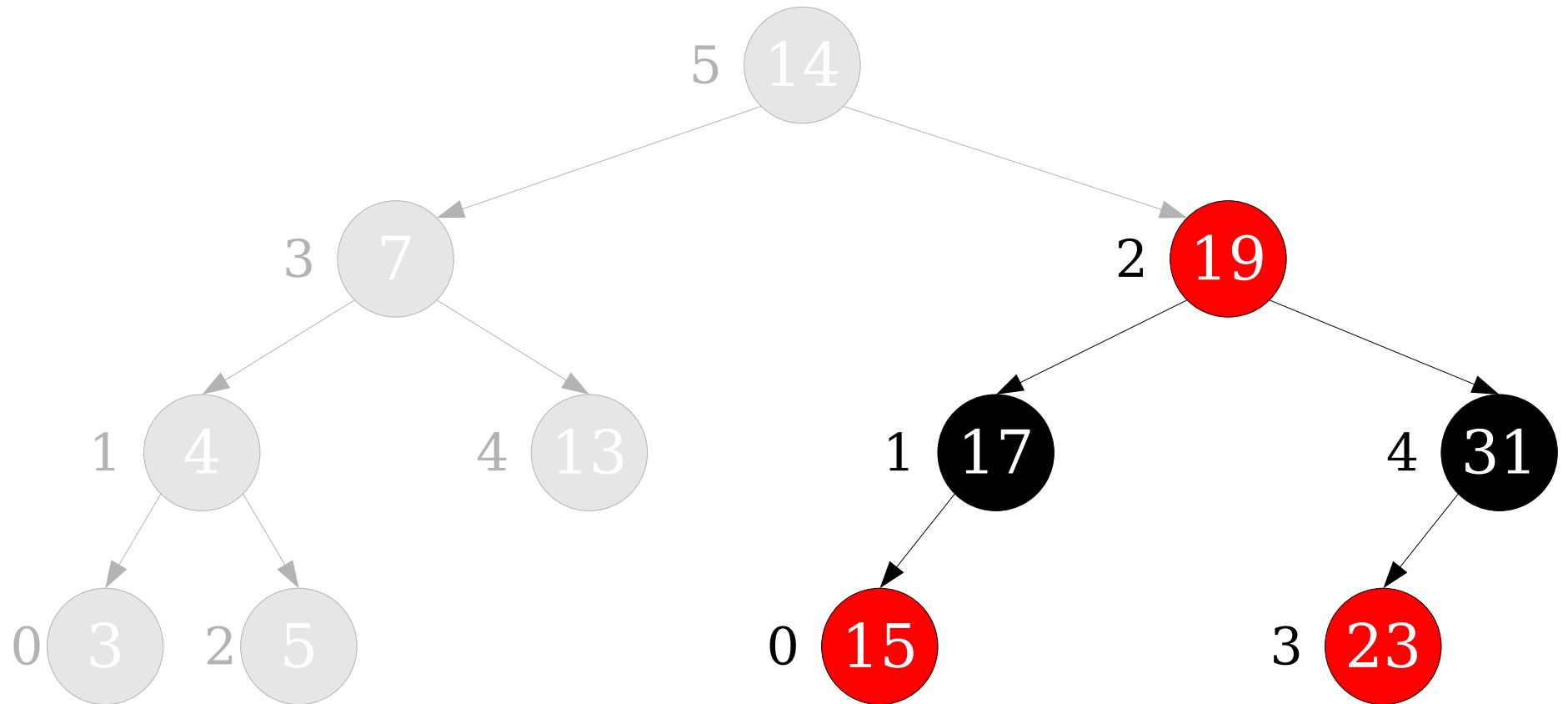


Dynamic Selection

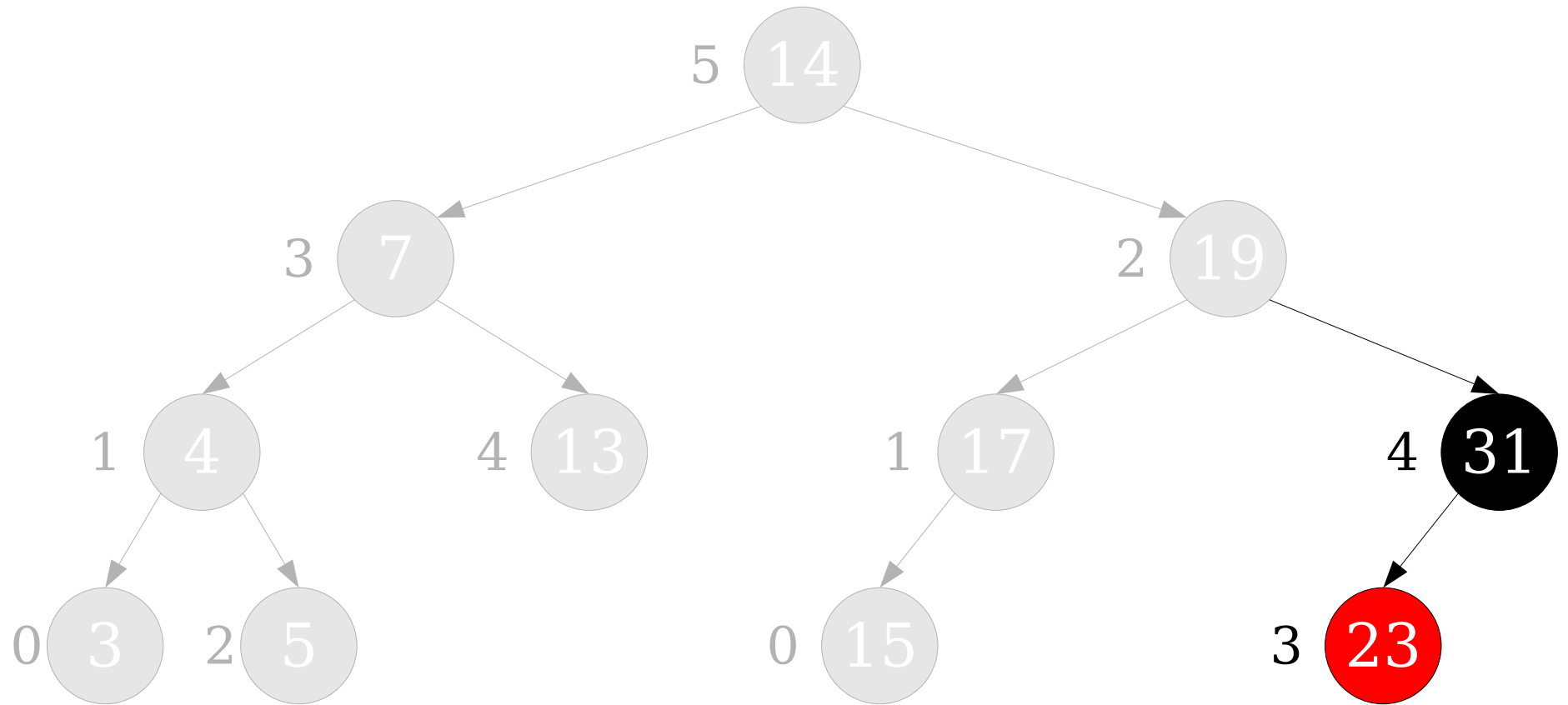


If new nodes are added to the left subtree, the numbers on the right don't need to update.

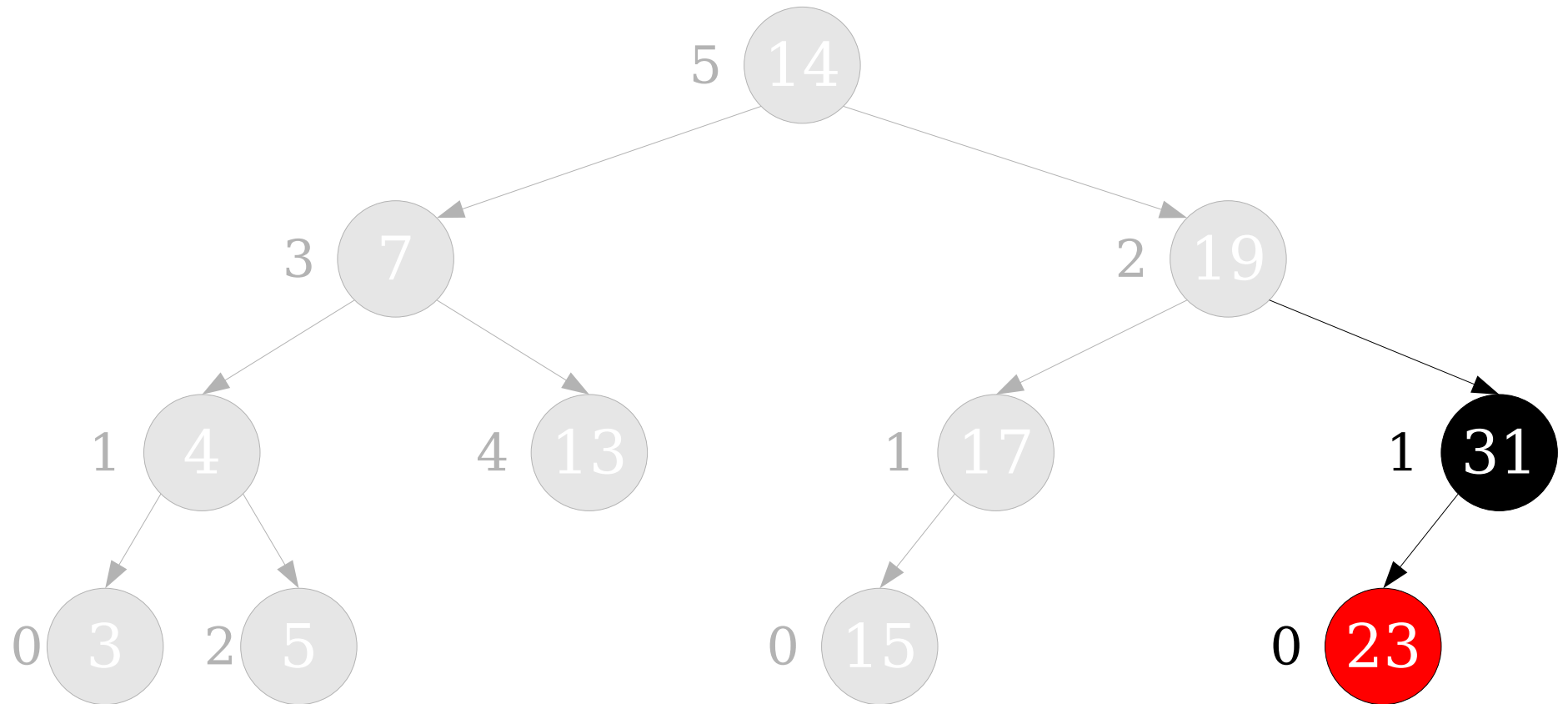
Dynamic Selection



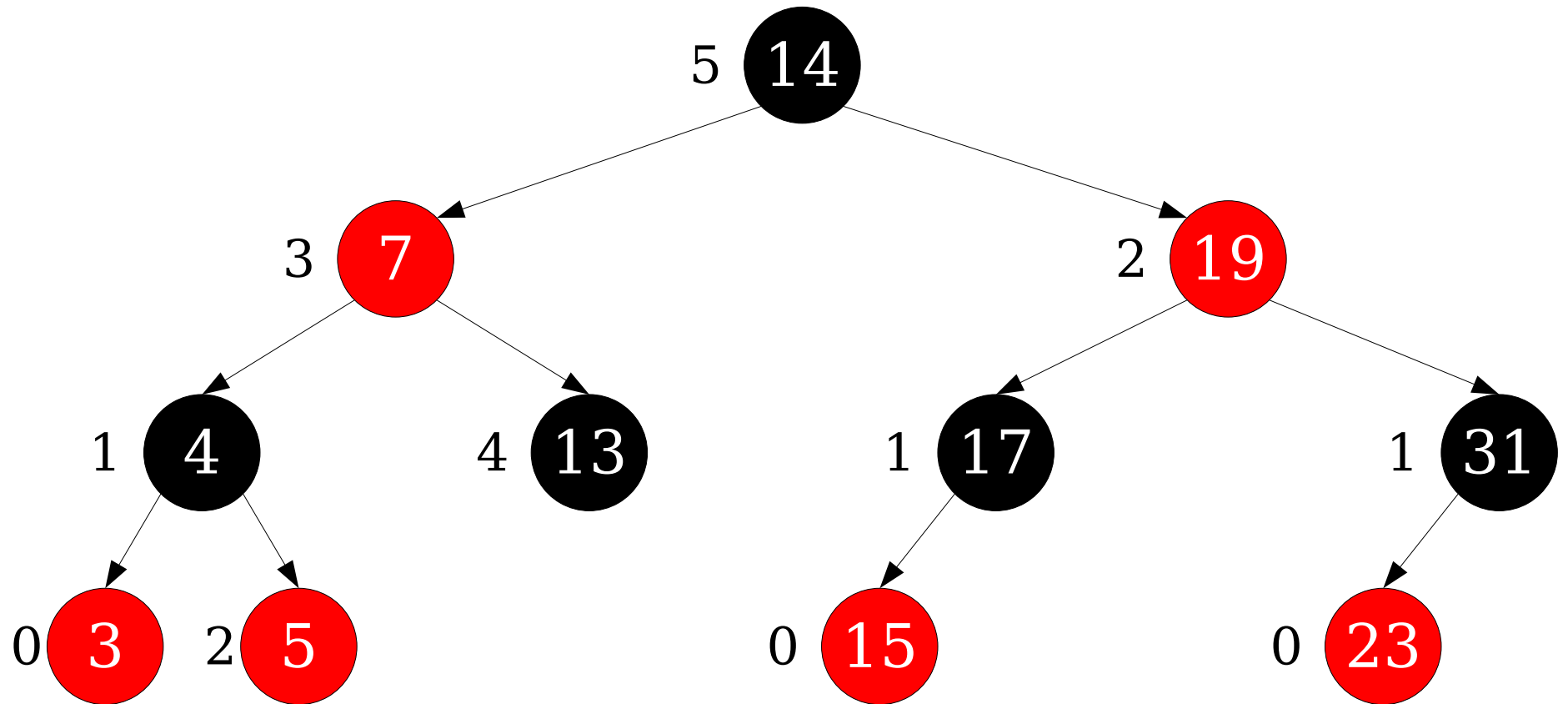
Dynamic Selection



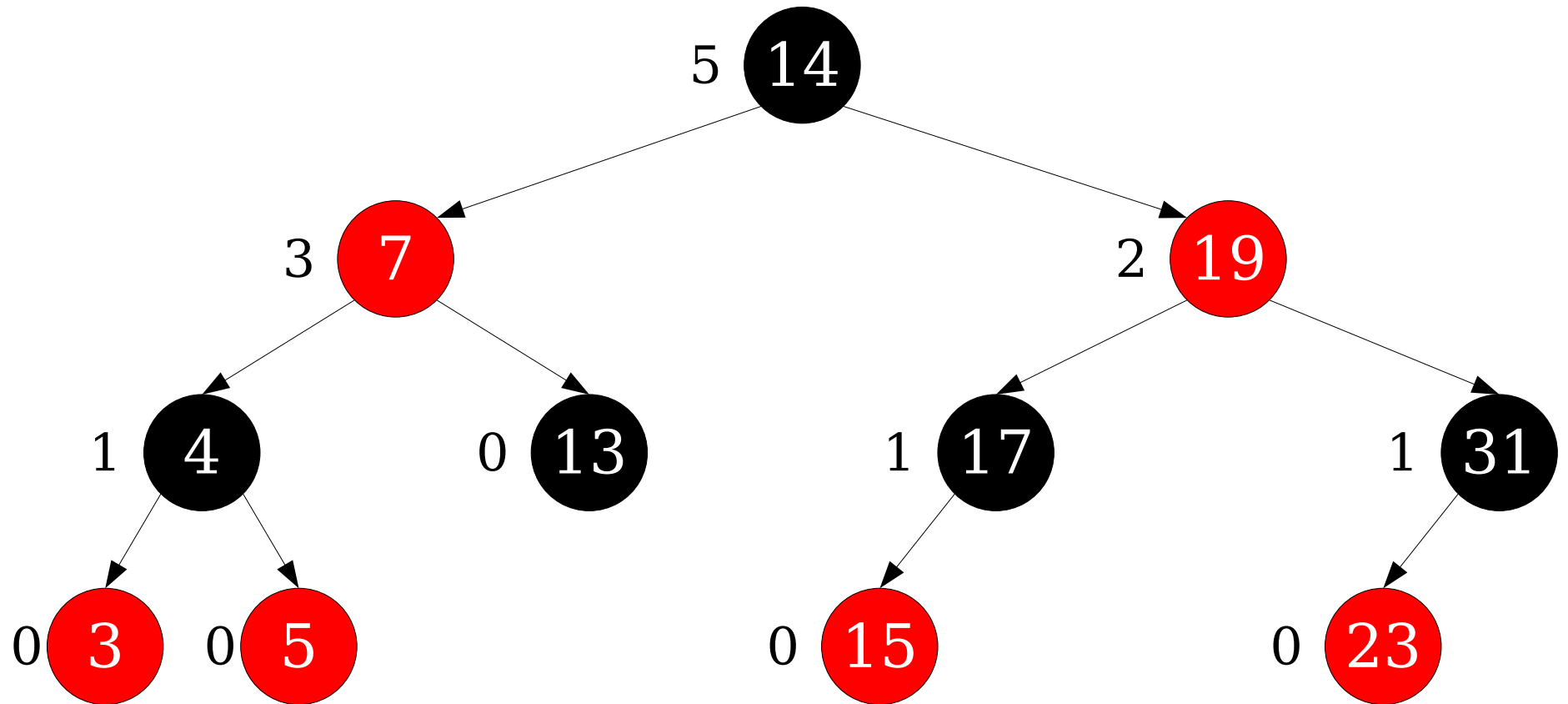
Dynamic Selection



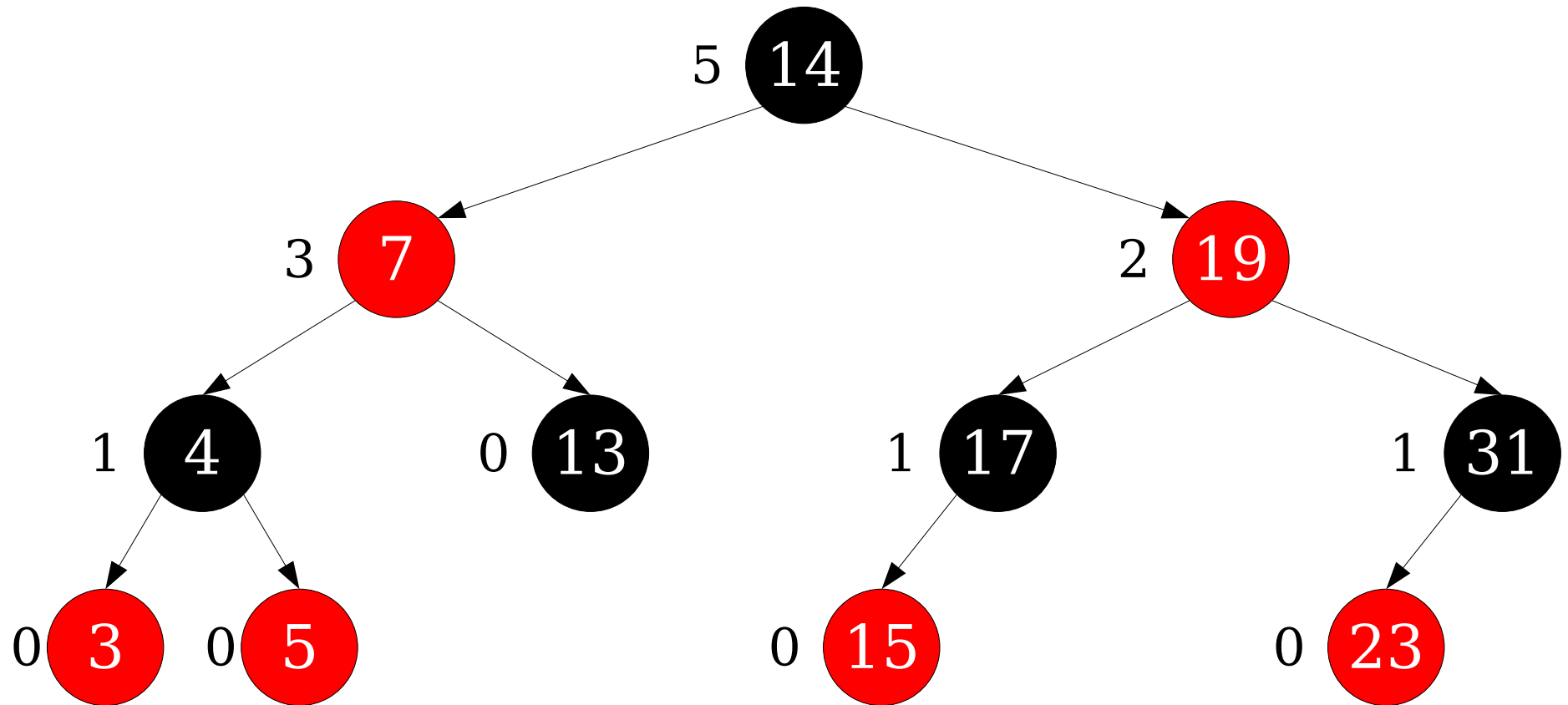
Dynamic Selection



Dynamic Selection



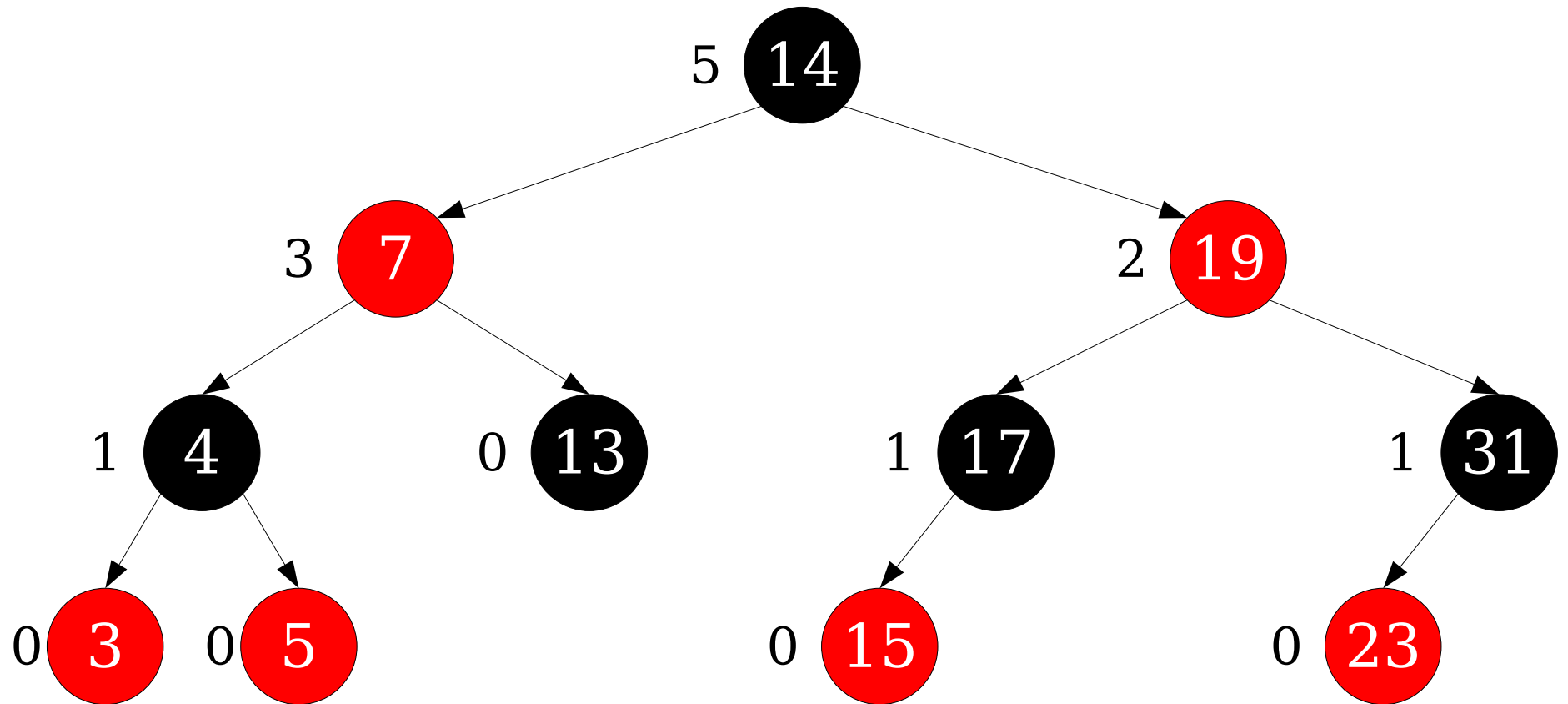
Dynamic Selection



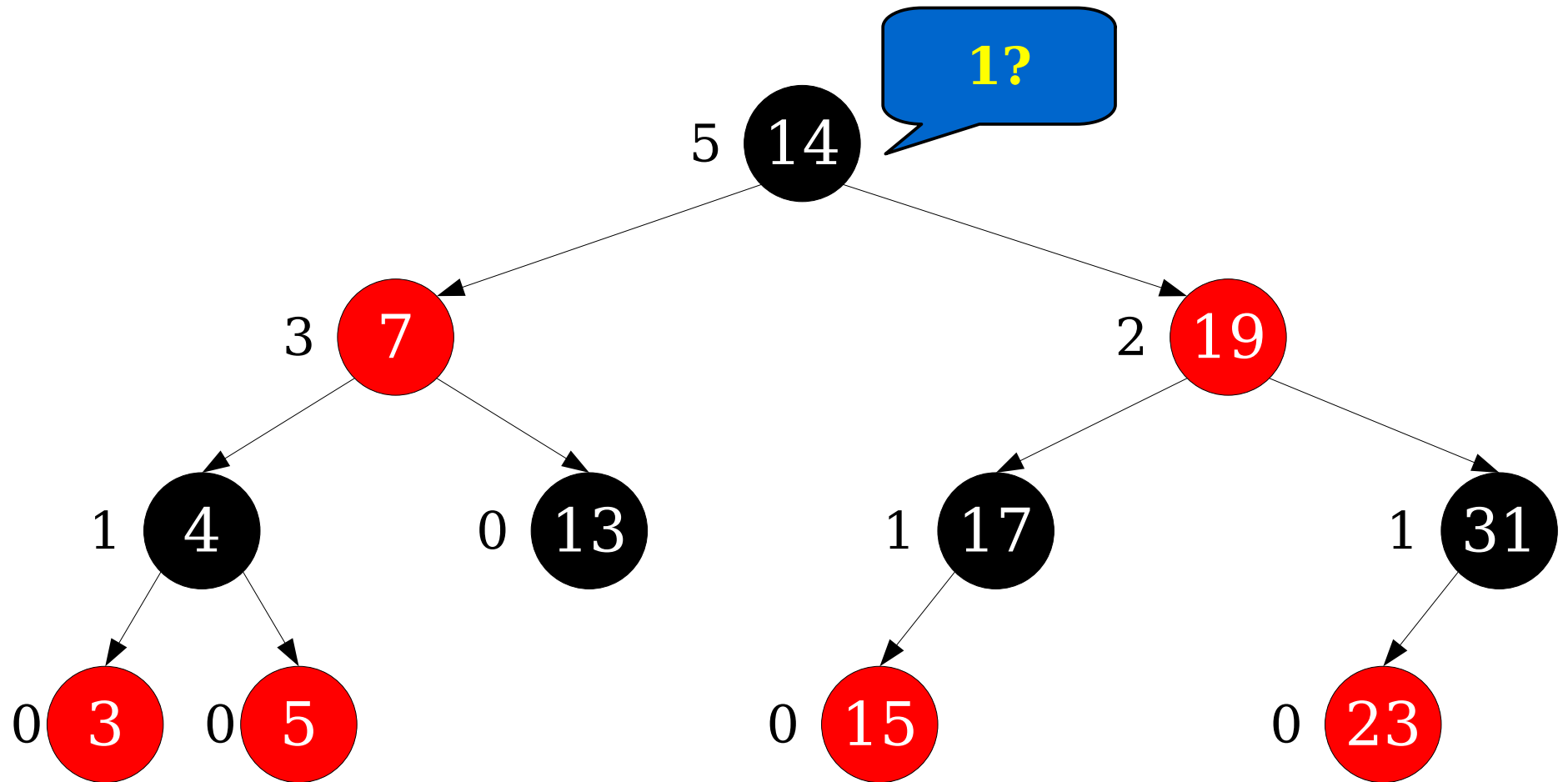
Mechanically: Number each key so that it only stores its order statistic in the subtree rooted at itself.

Operationally: Annotate each key with the number of keys in its left subtree.

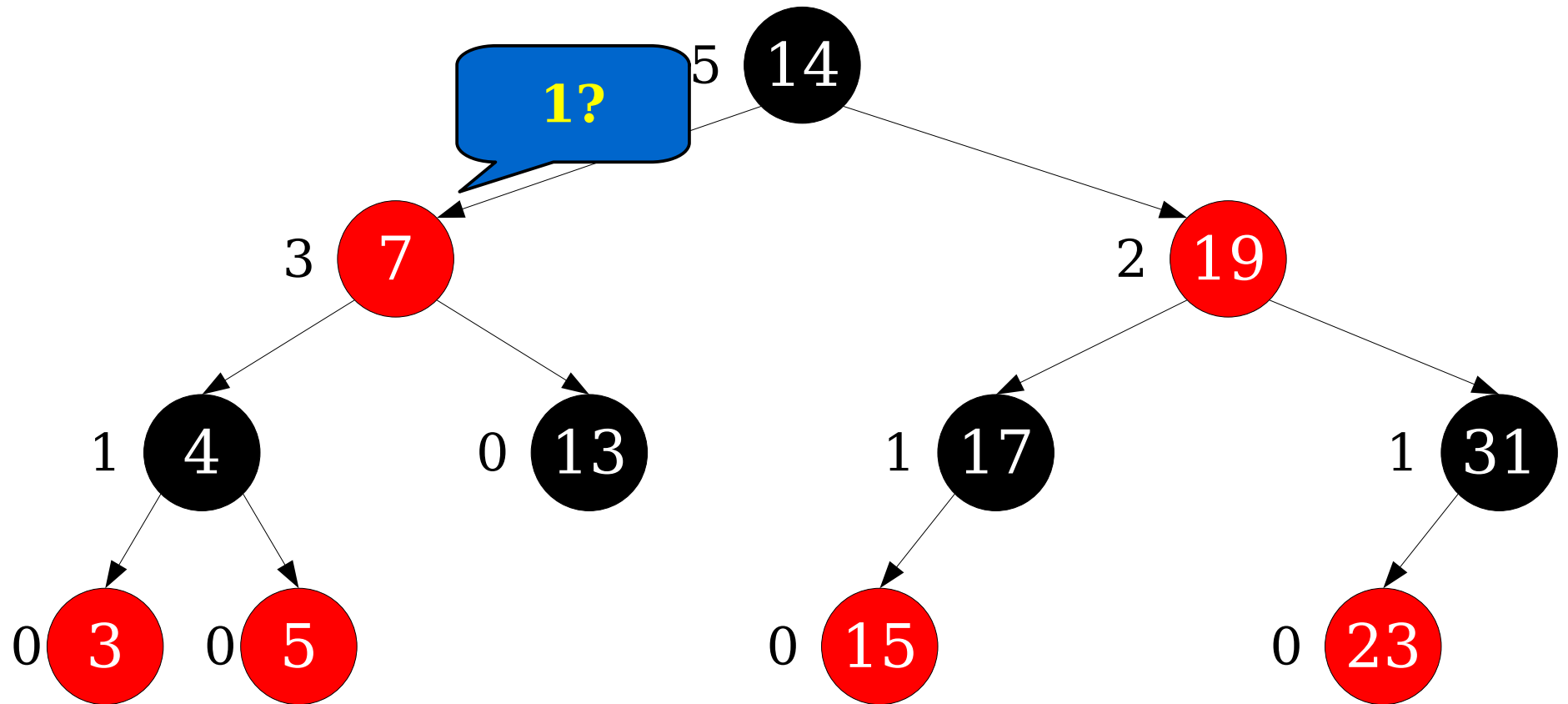
Dynamic Selection



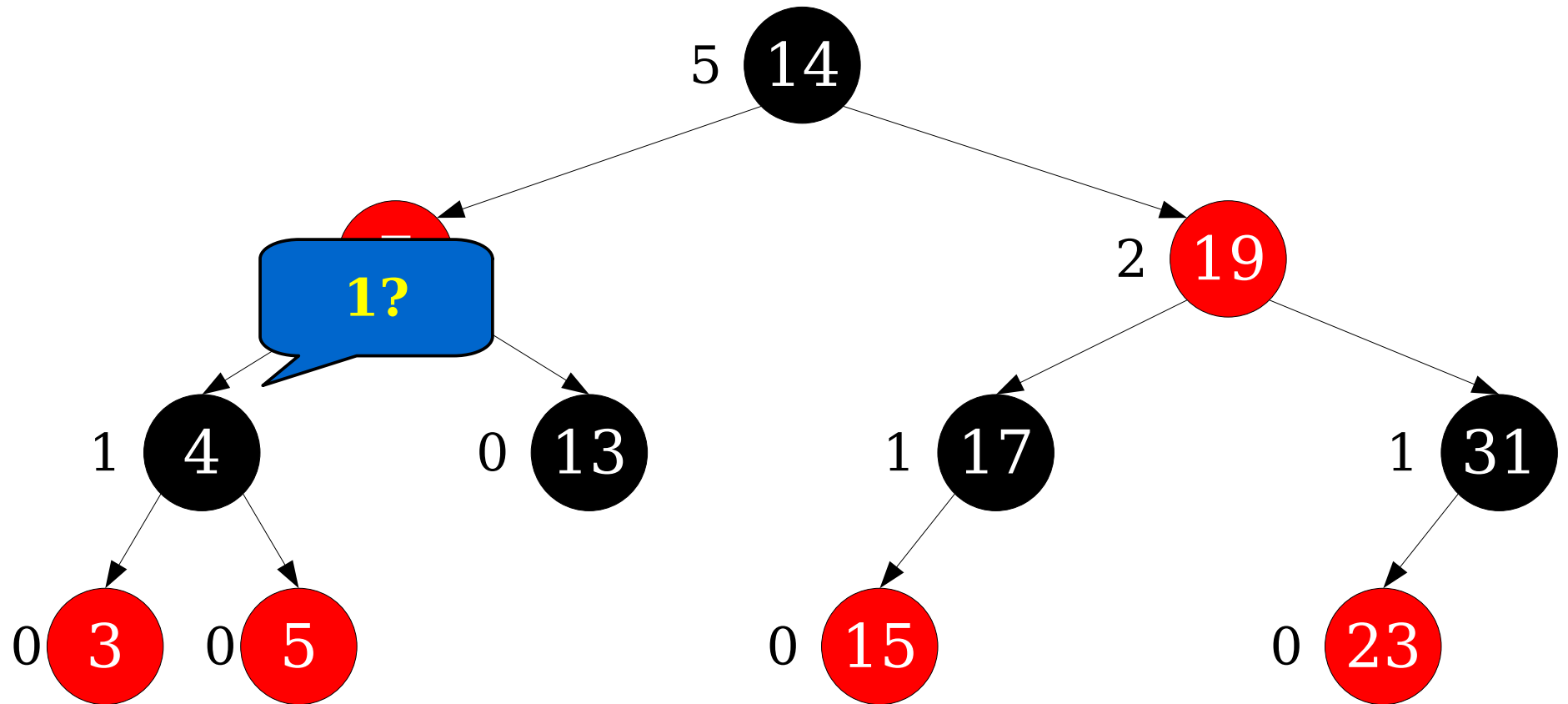
Dynamic Selection



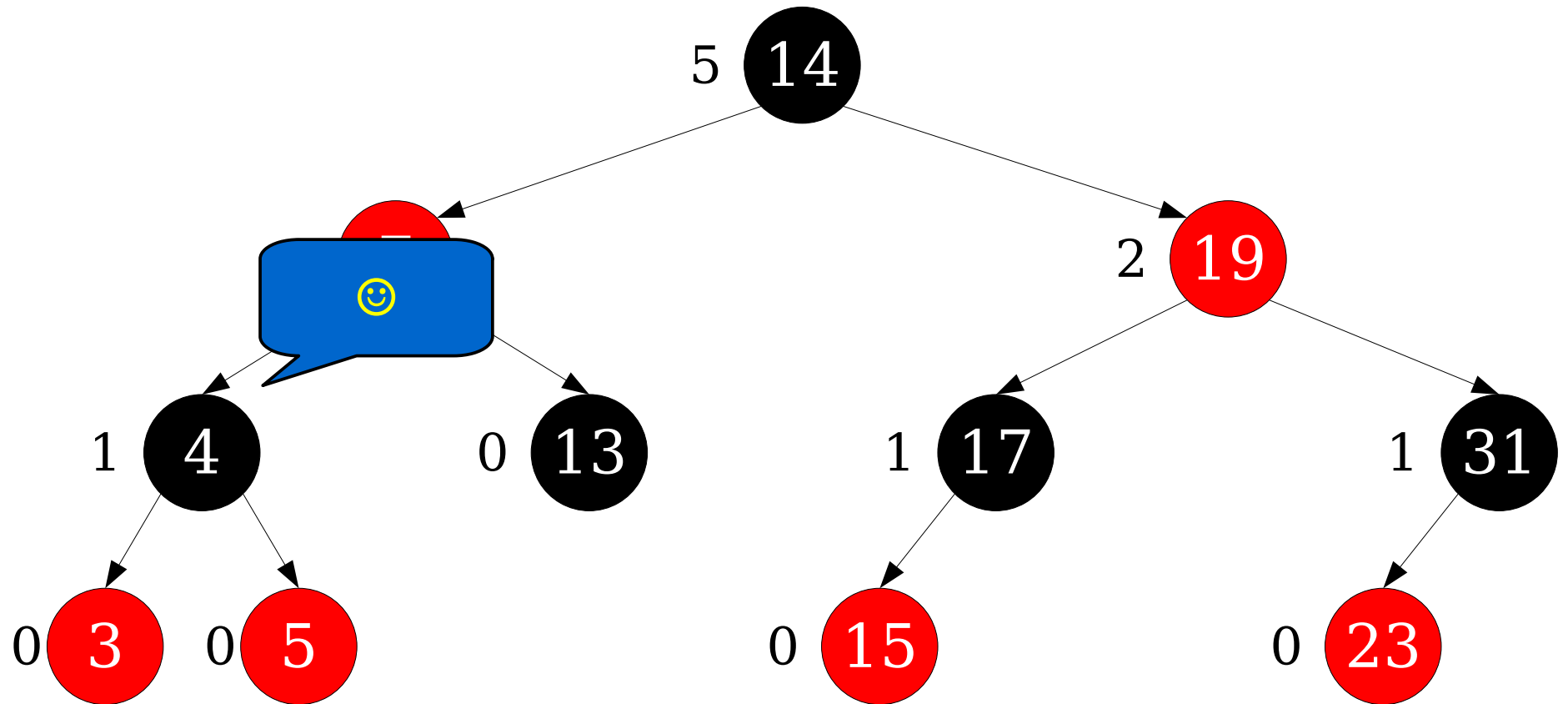
Dynamic Selection



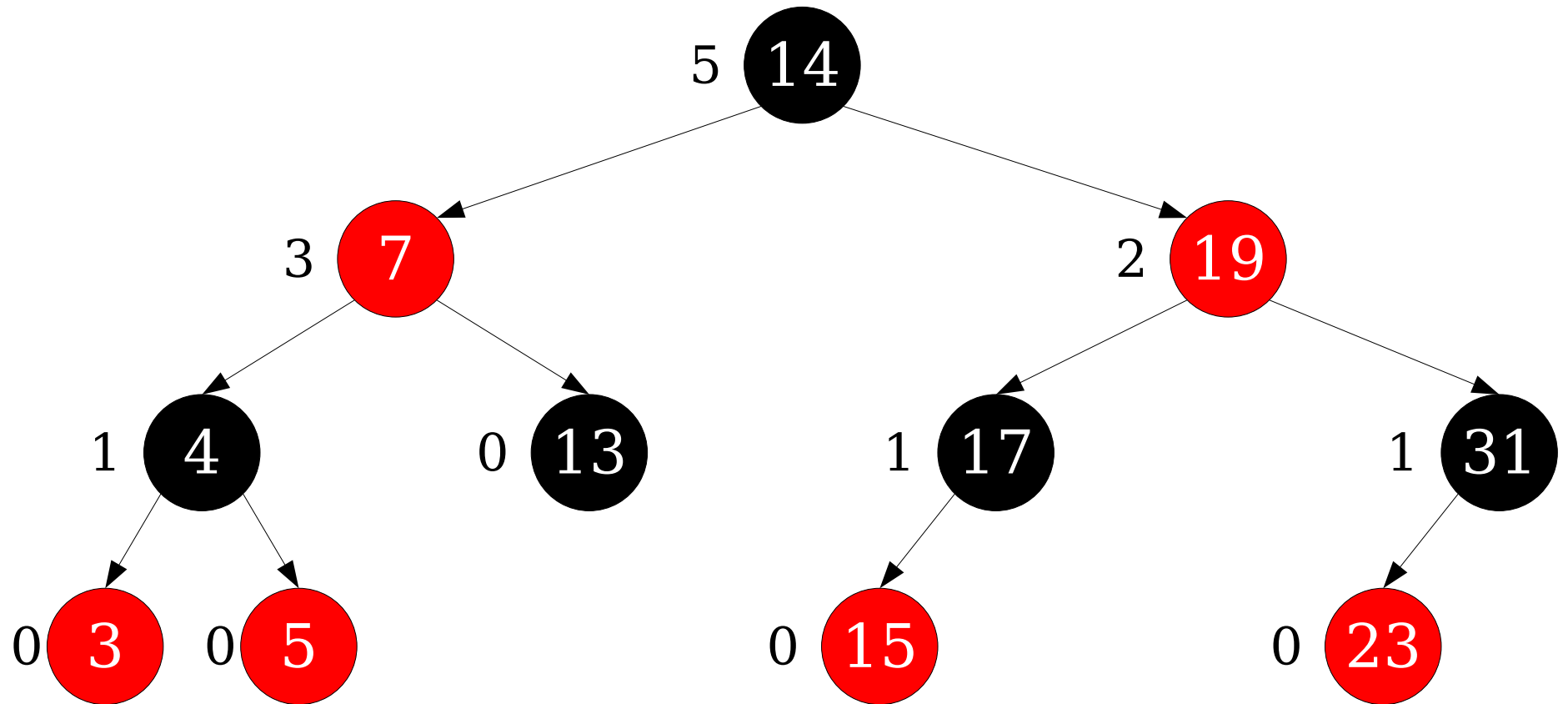
Dynamic Selection



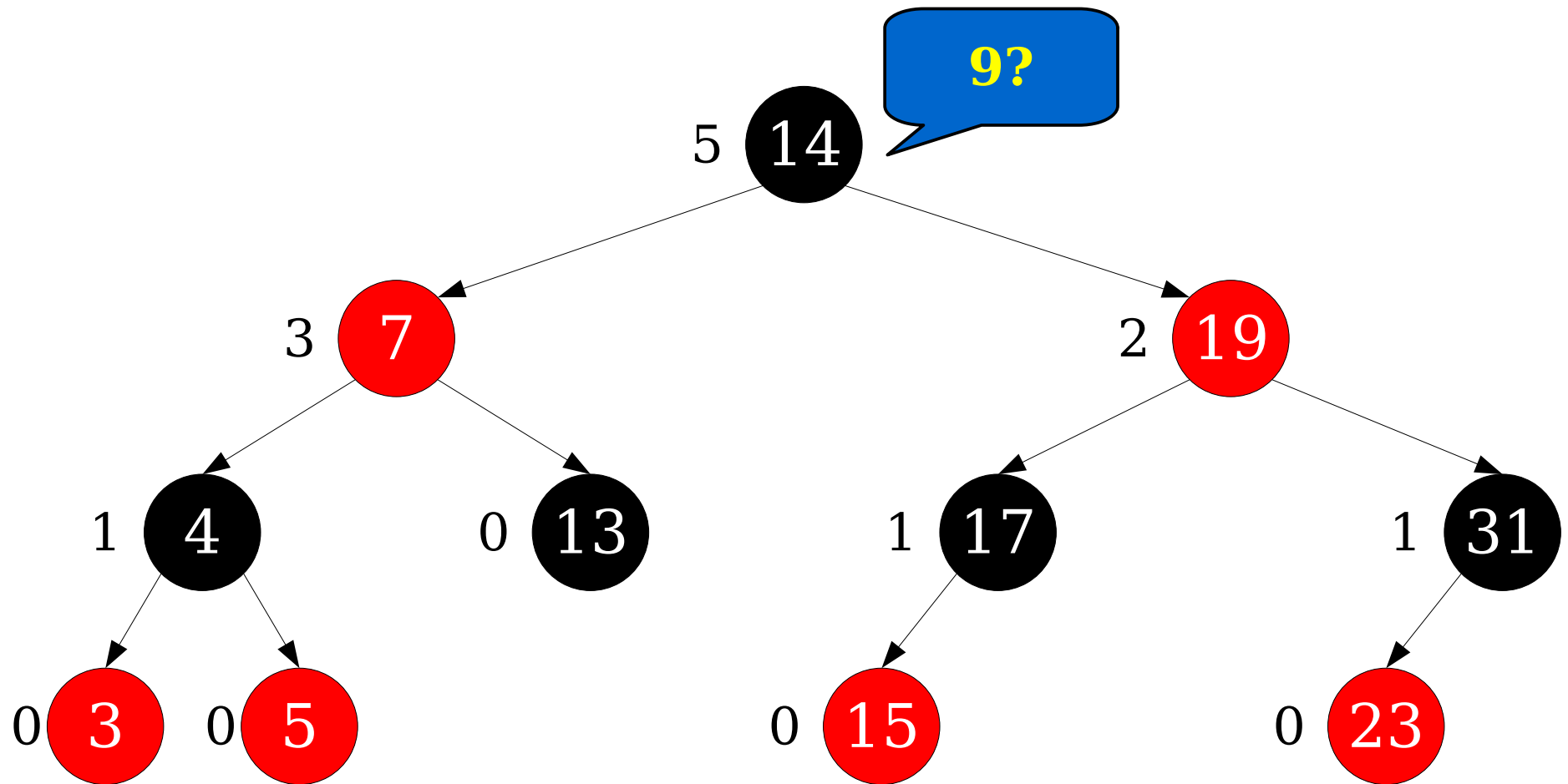
Dynamic Selection



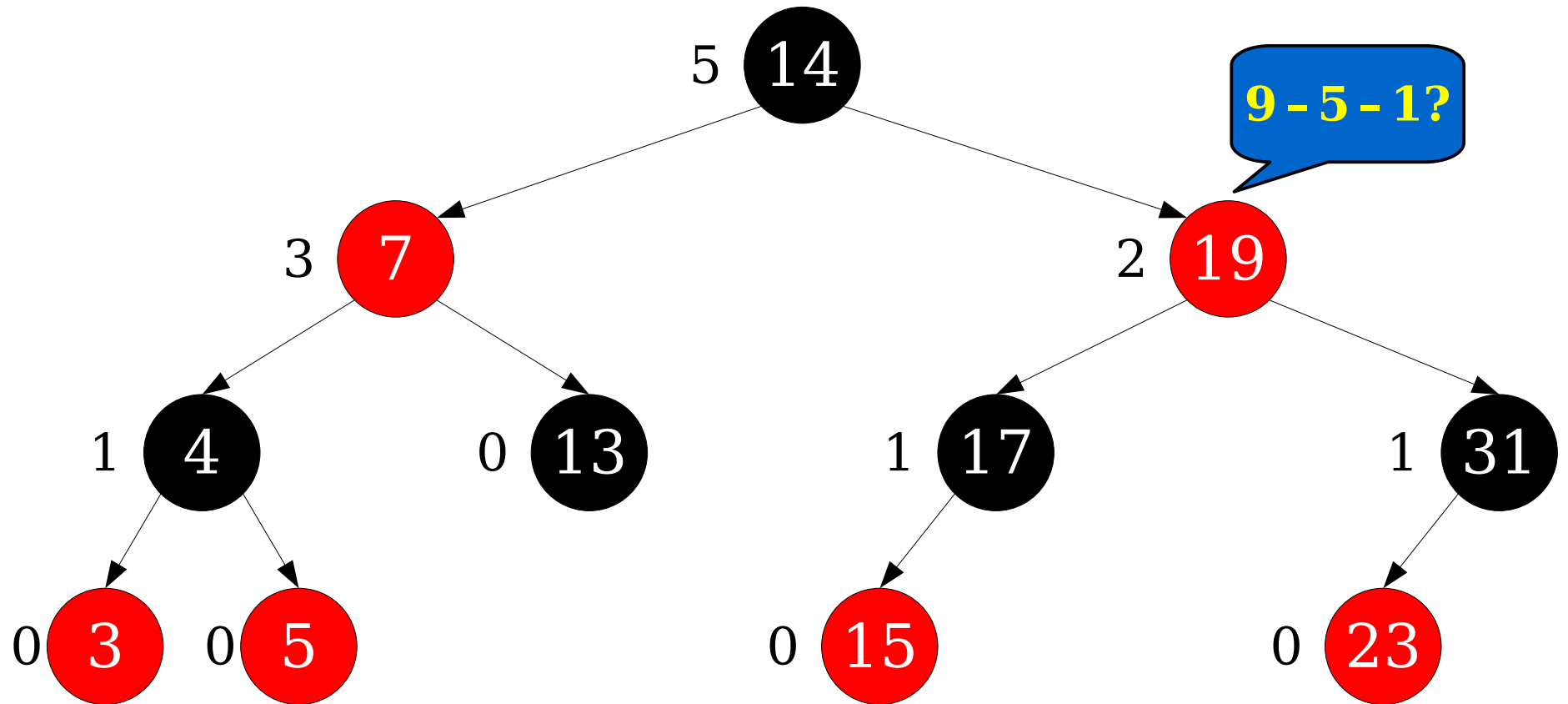
Dynamic Selection



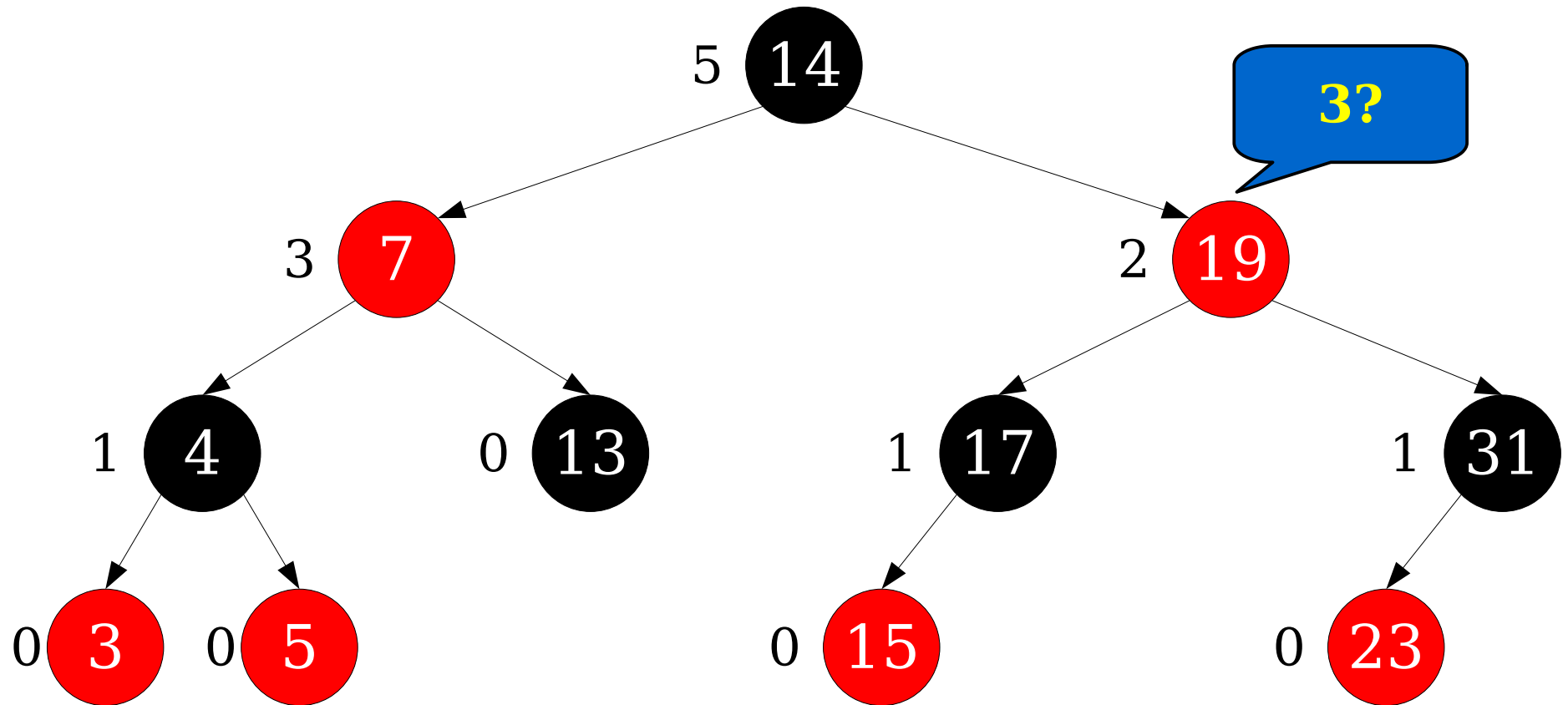
Dynamic Selection



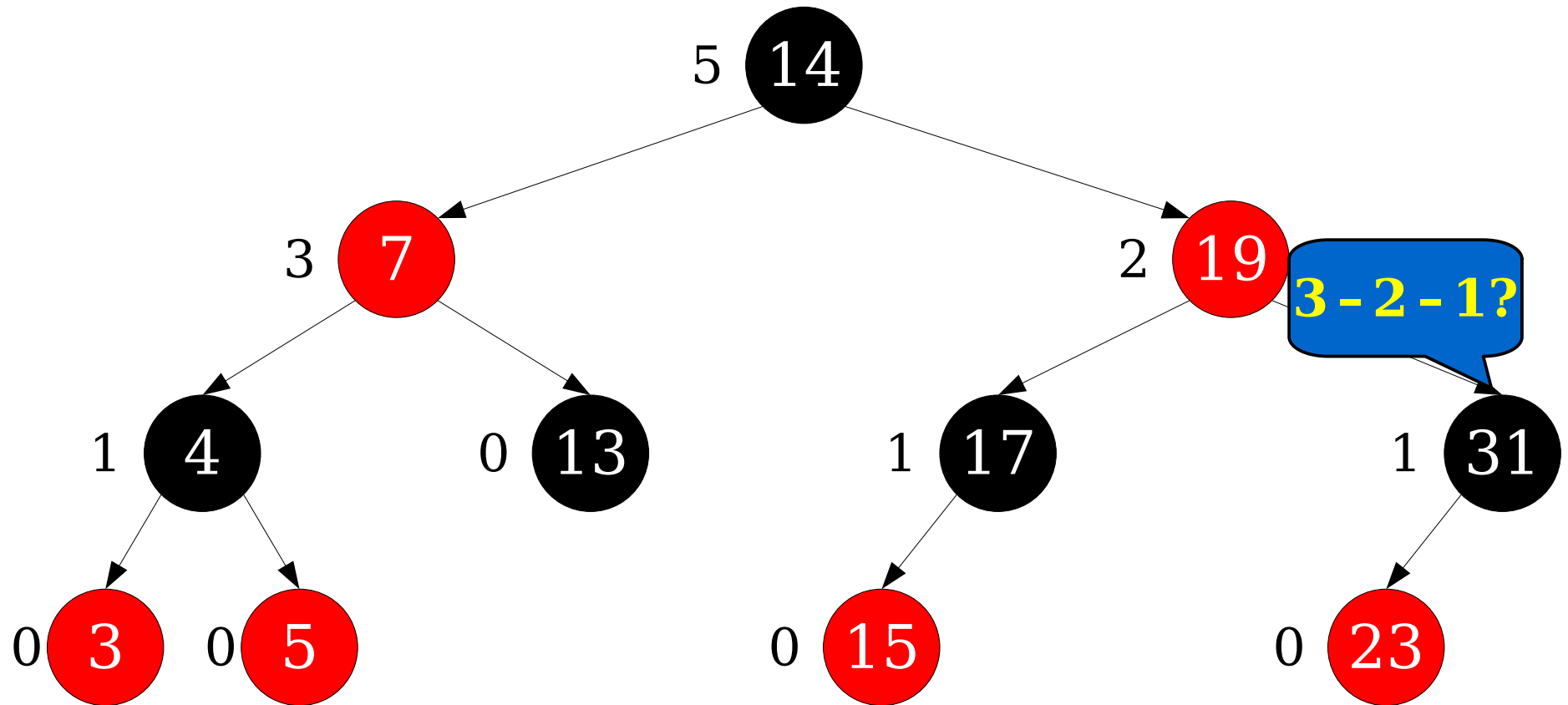
Dynamic Selection



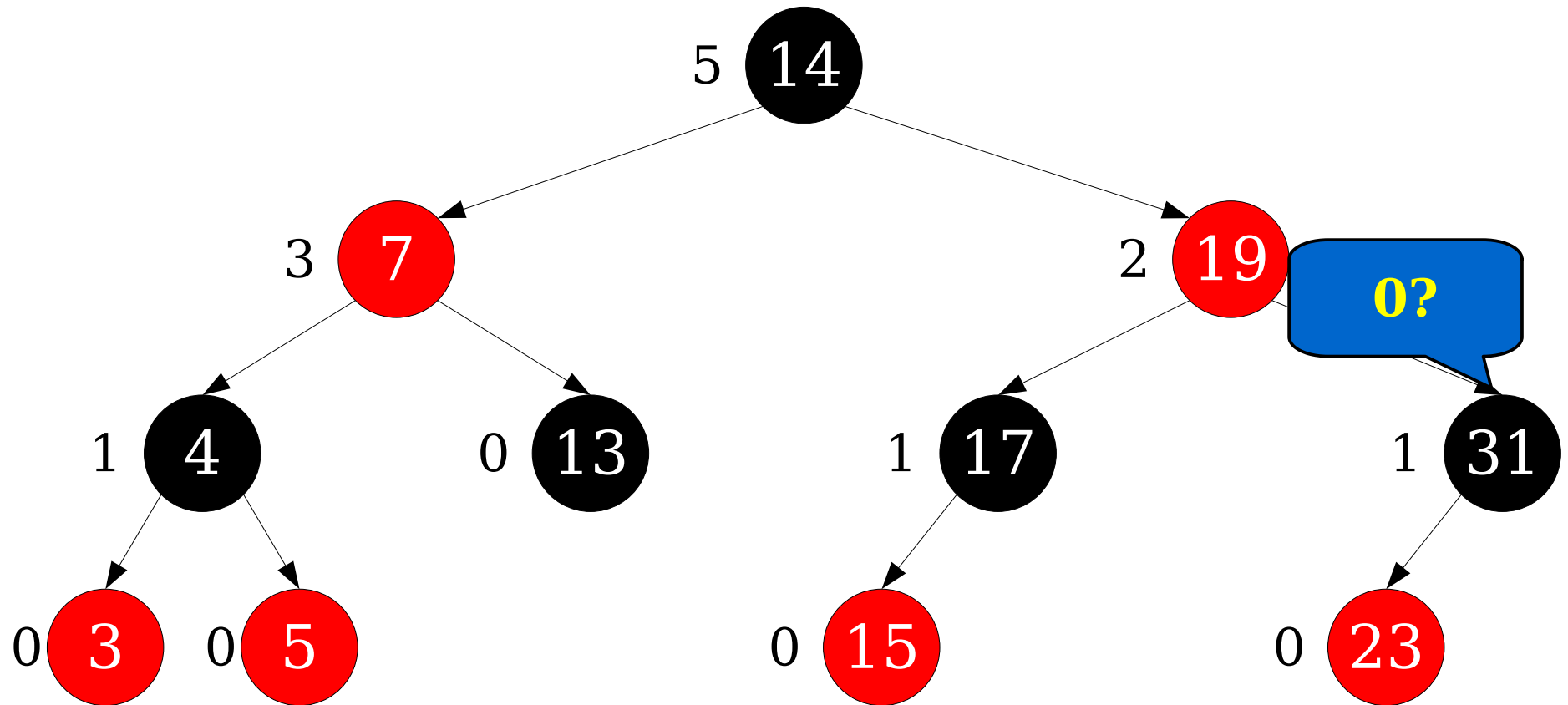
Dynamic Selection



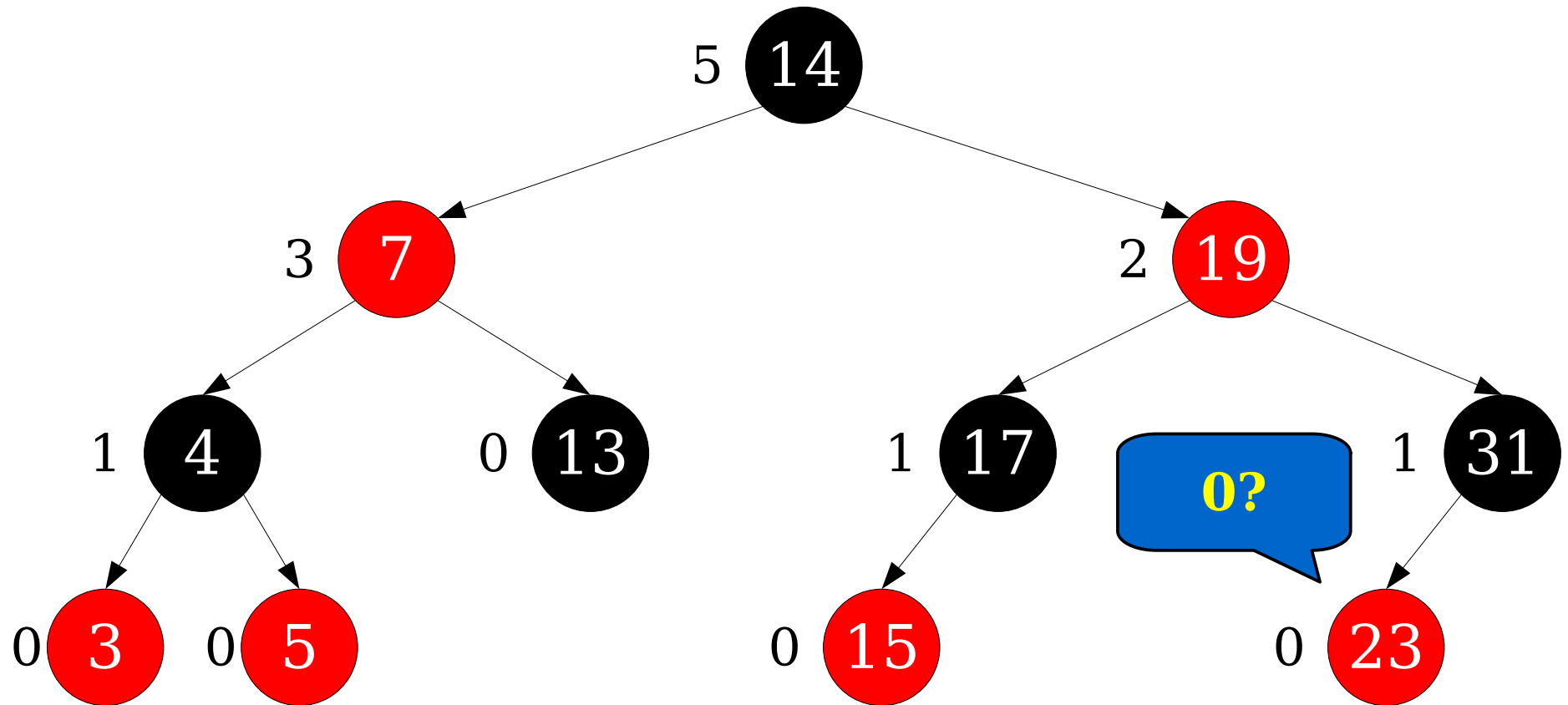
Dynamic Selection



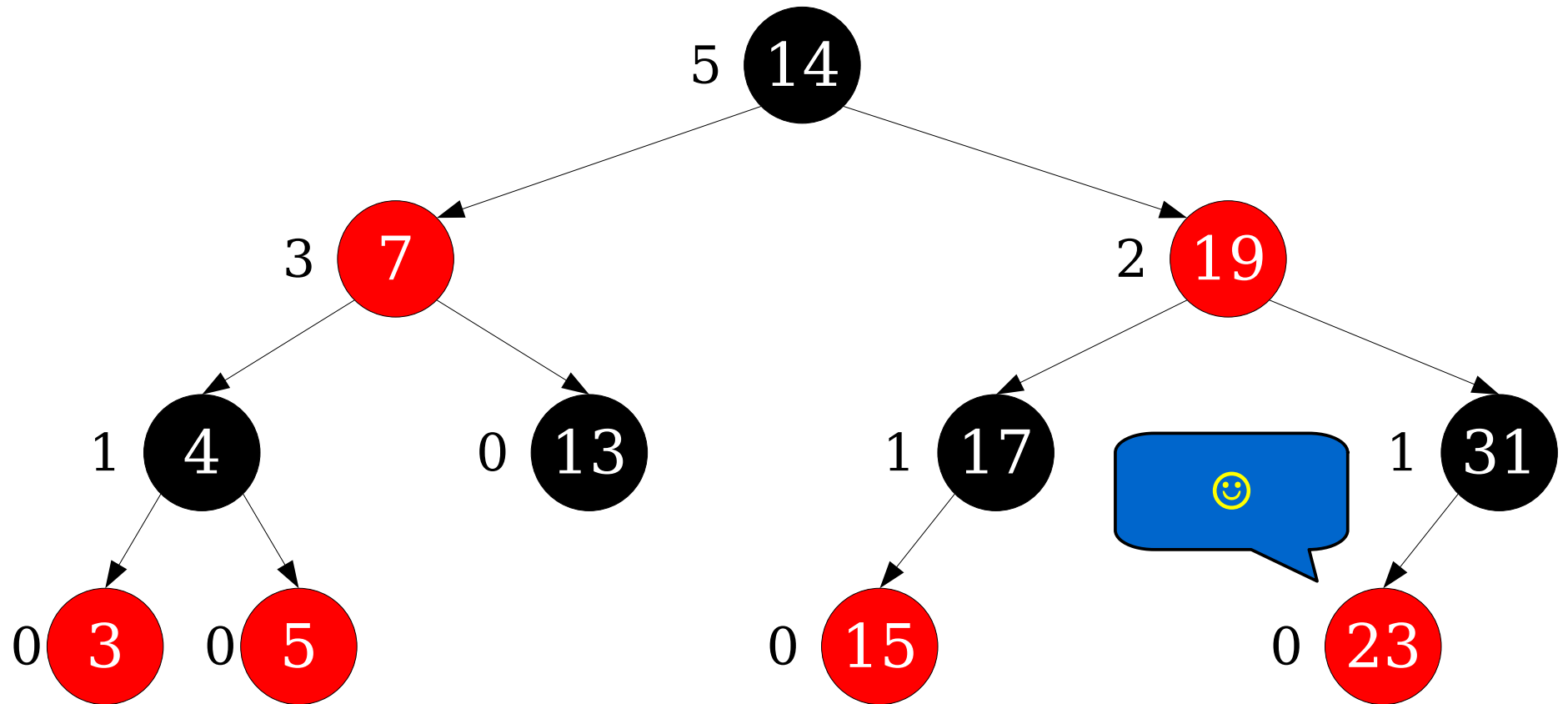
Dynamic Selection



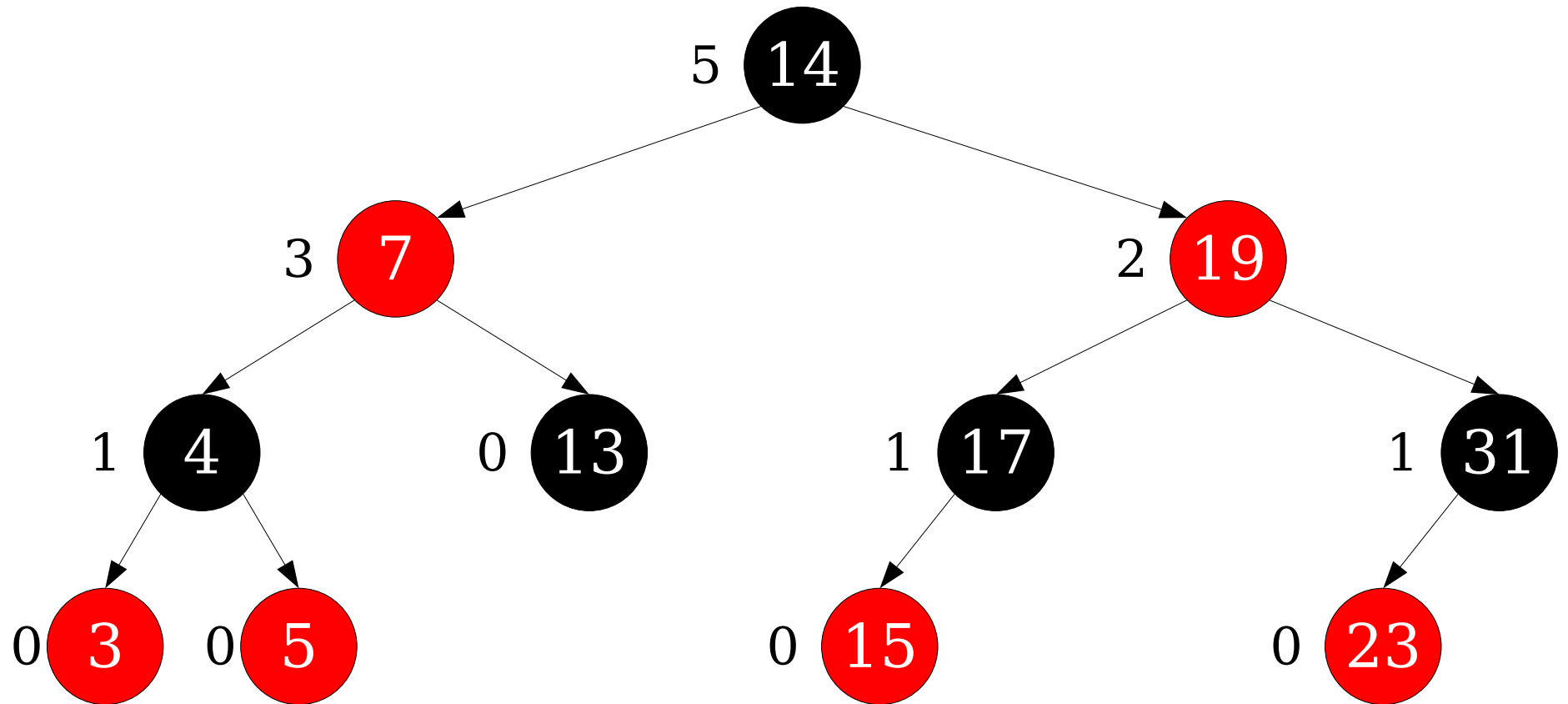
Dynamic Selection



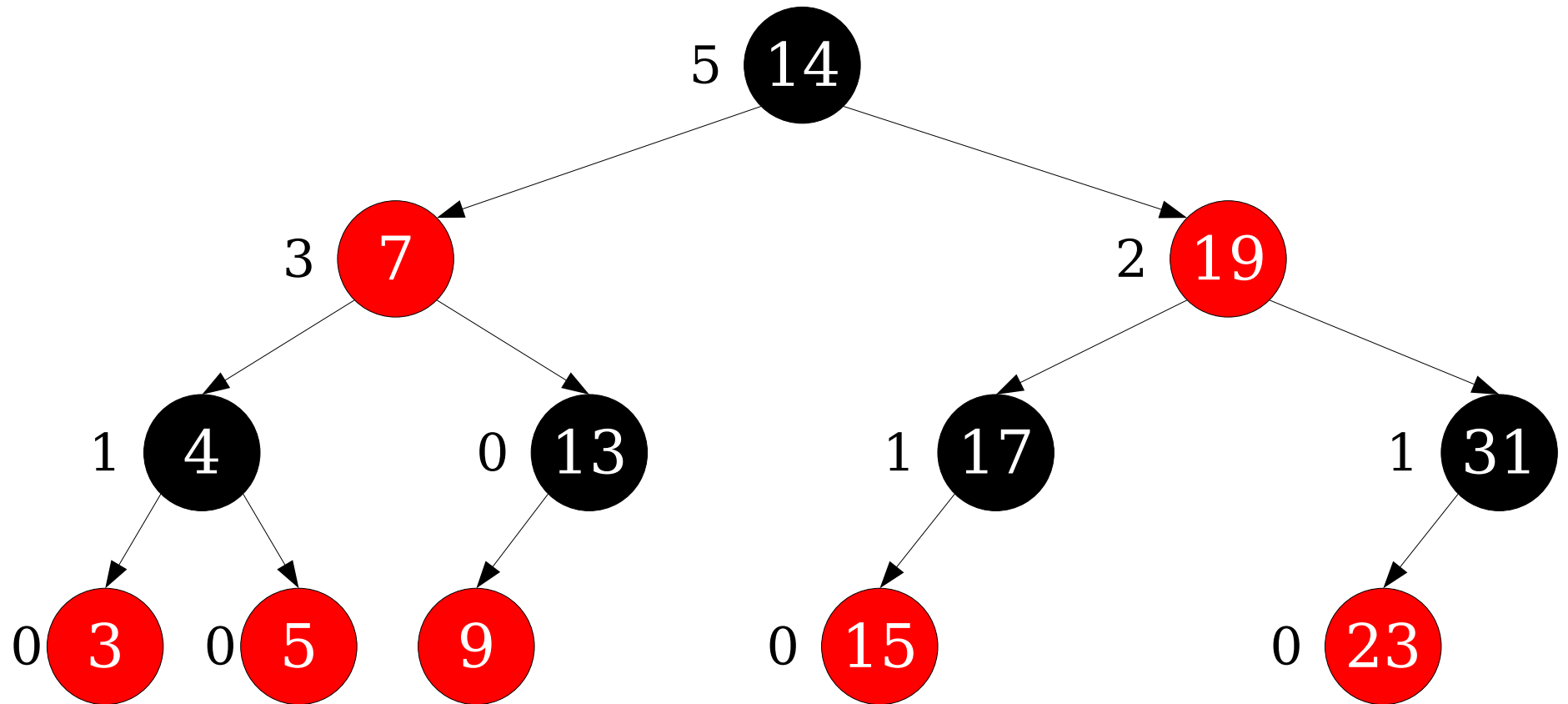
Dynamic Selection



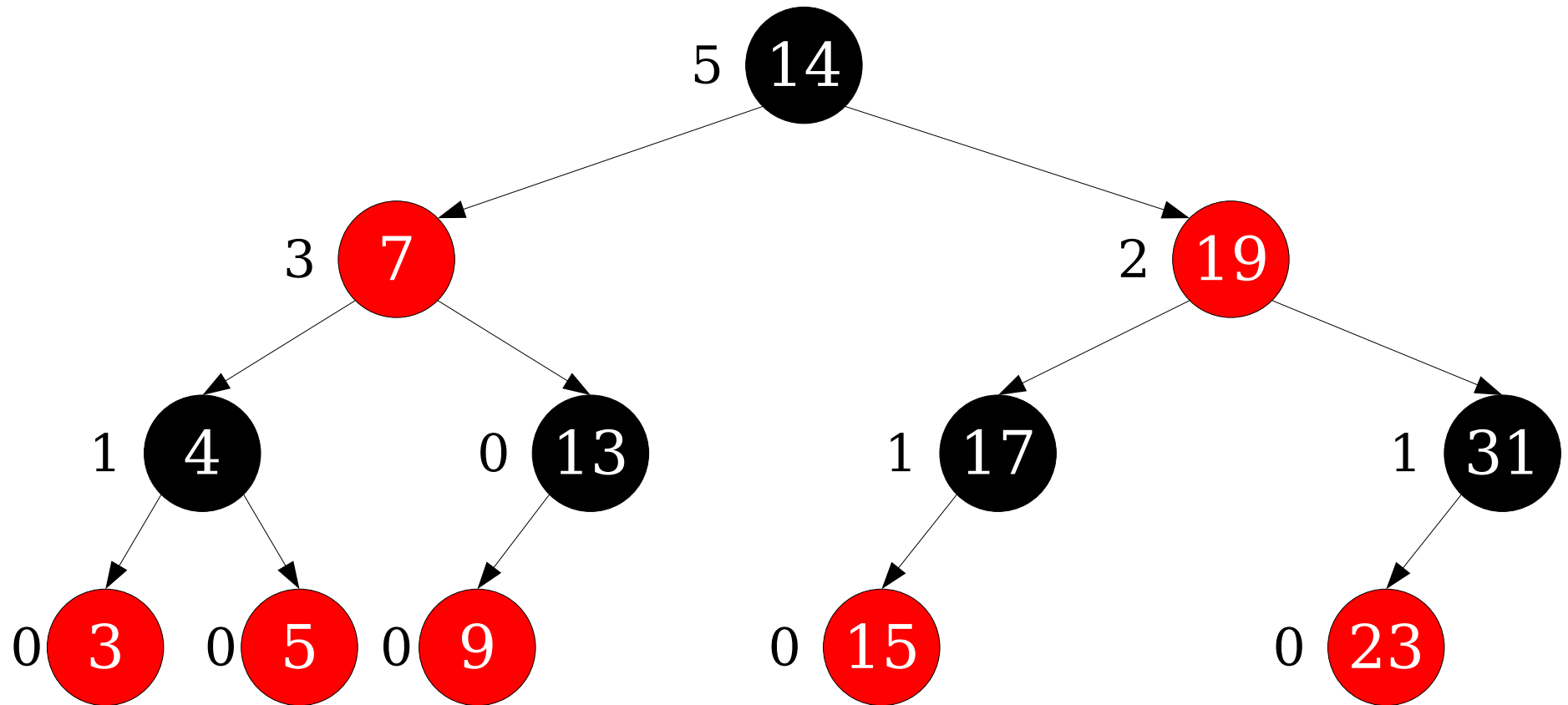
Dynamic Selection



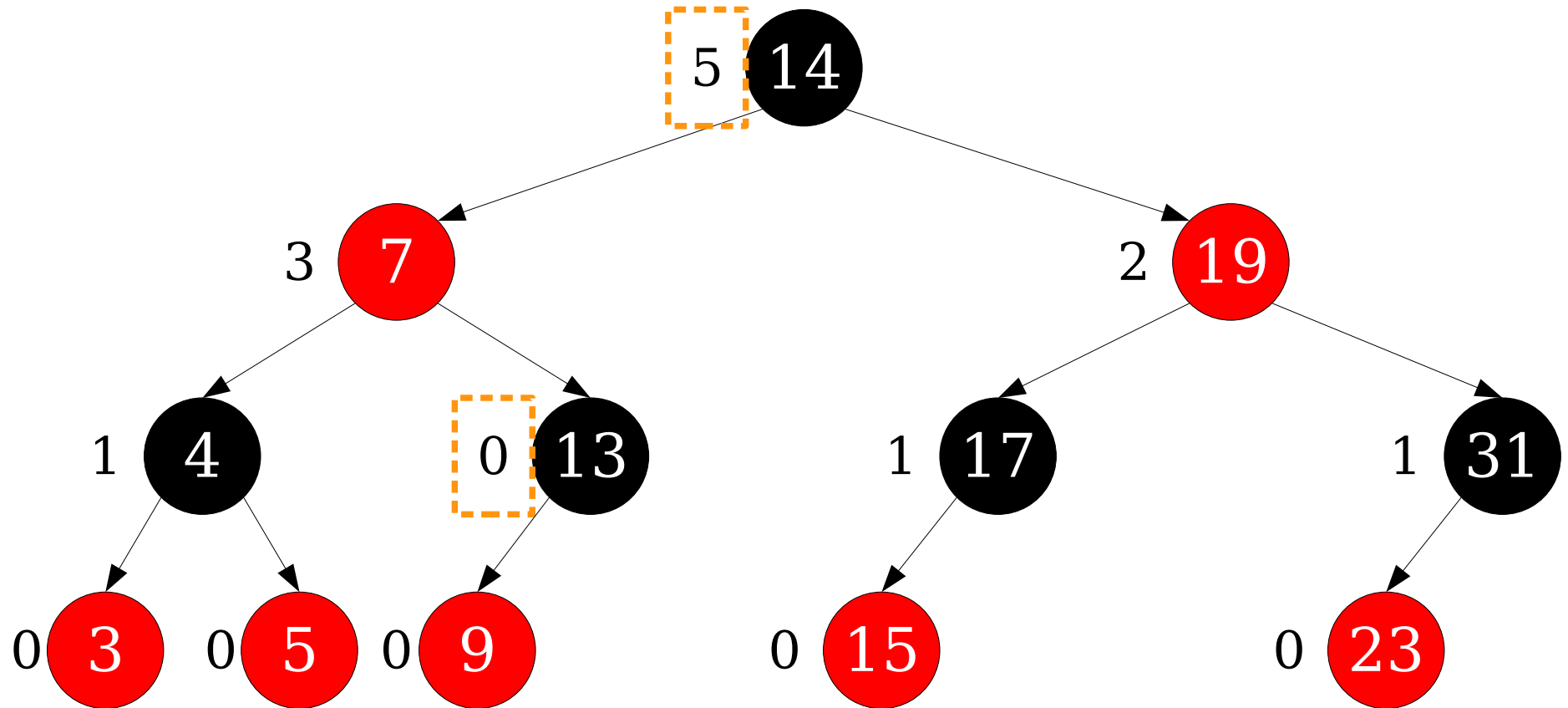
Dynamic Selection



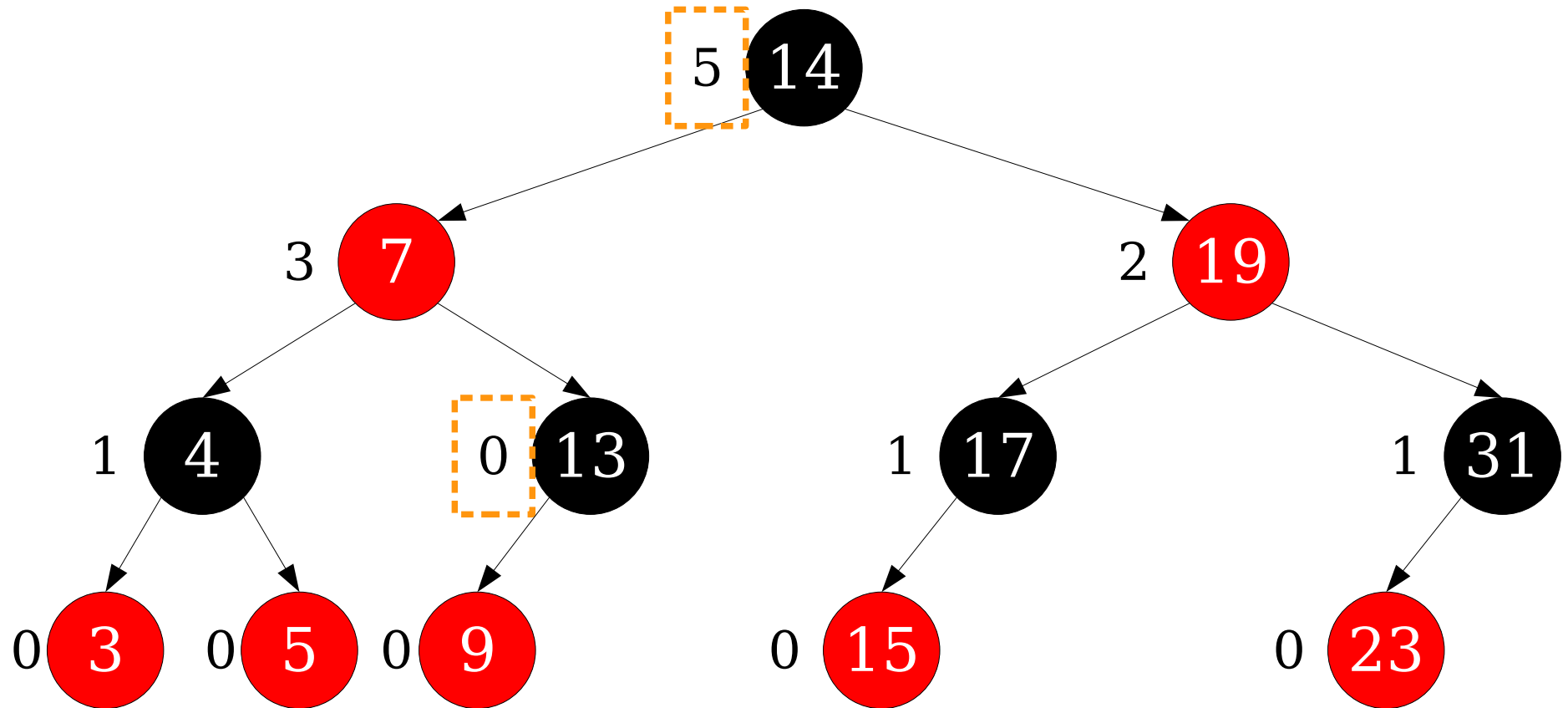
Dynamic Selection



Dynamic Selection

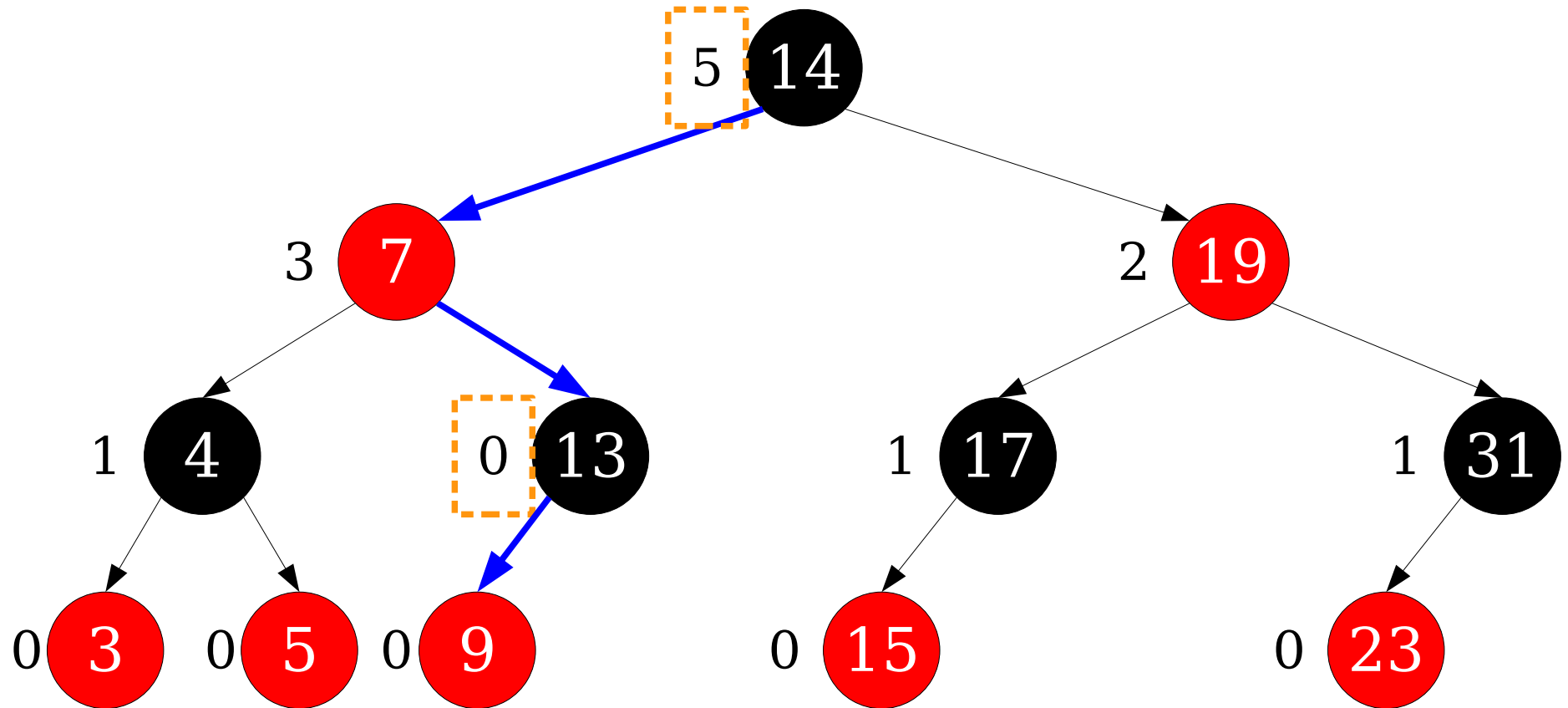


Dynamic Selection



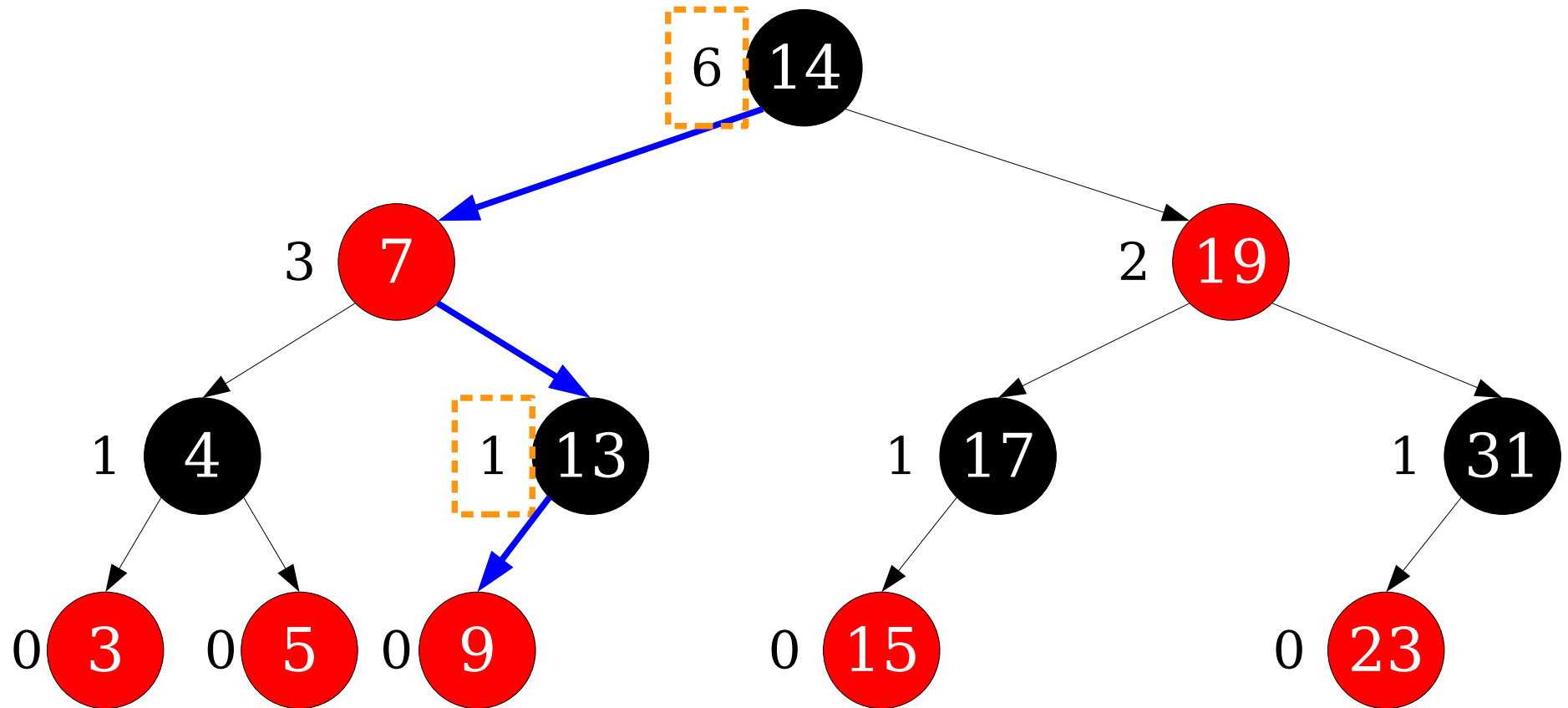
We only update values on nodes that gained a new key in their left subtree. And there are only $O(\log n)$ of these!

Dynamic Selection



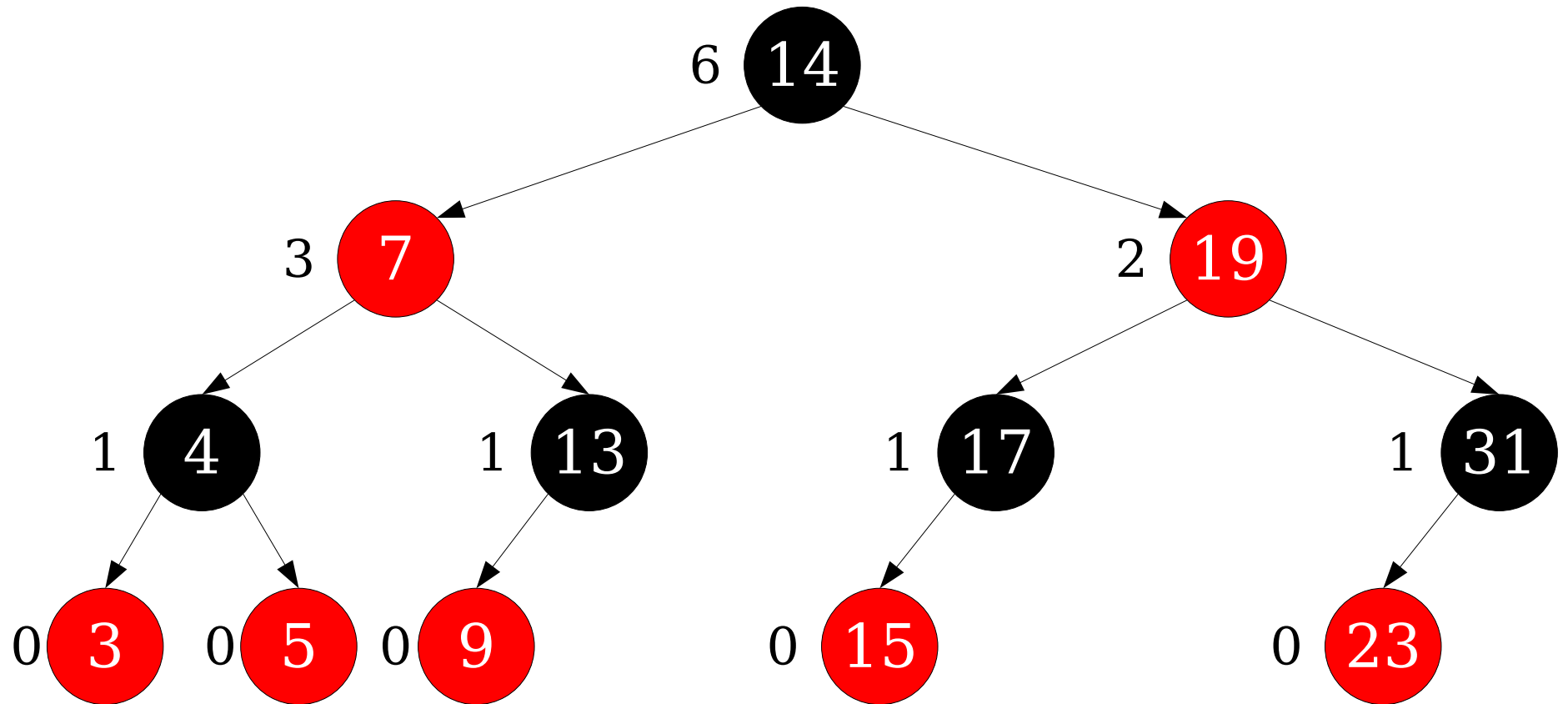
We only update values on nodes that gained a new key in their left subtree. And there are only $O(\log n)$ of these!

Dynamic Selection

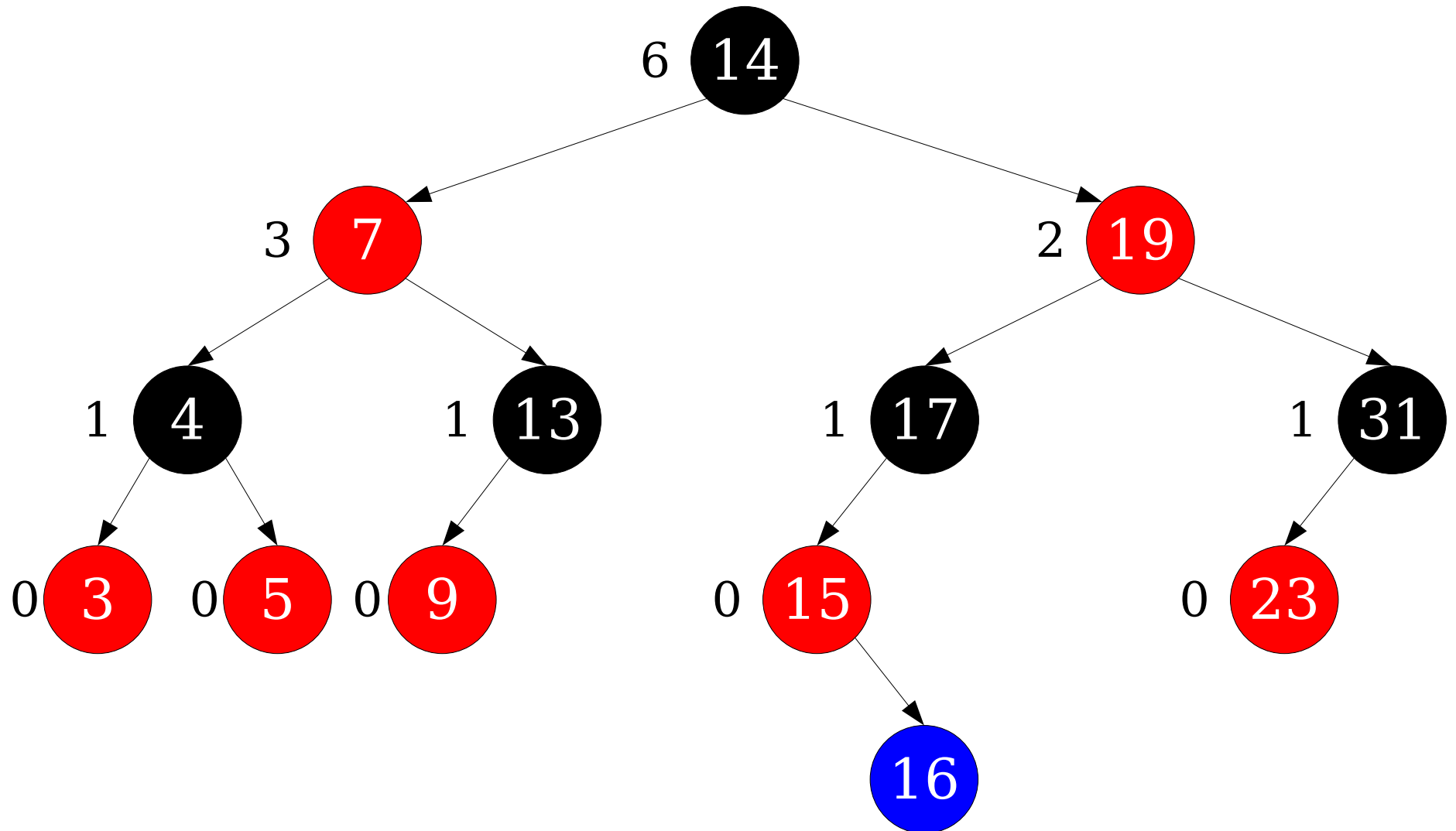


We only update values on nodes that gained a new key in their left subtree. And there are only $O(\log n)$ of these!

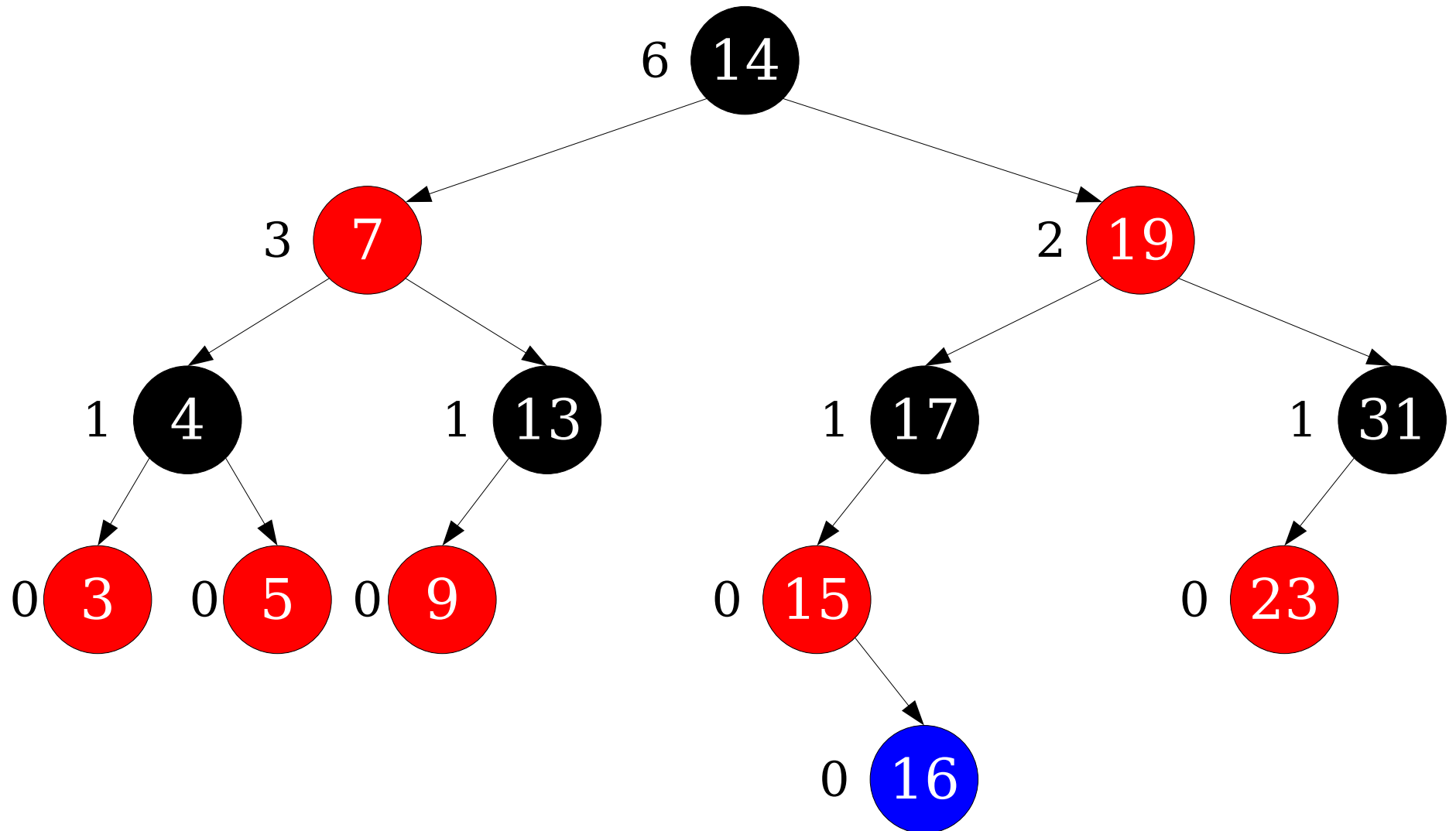
Dynamic Selection



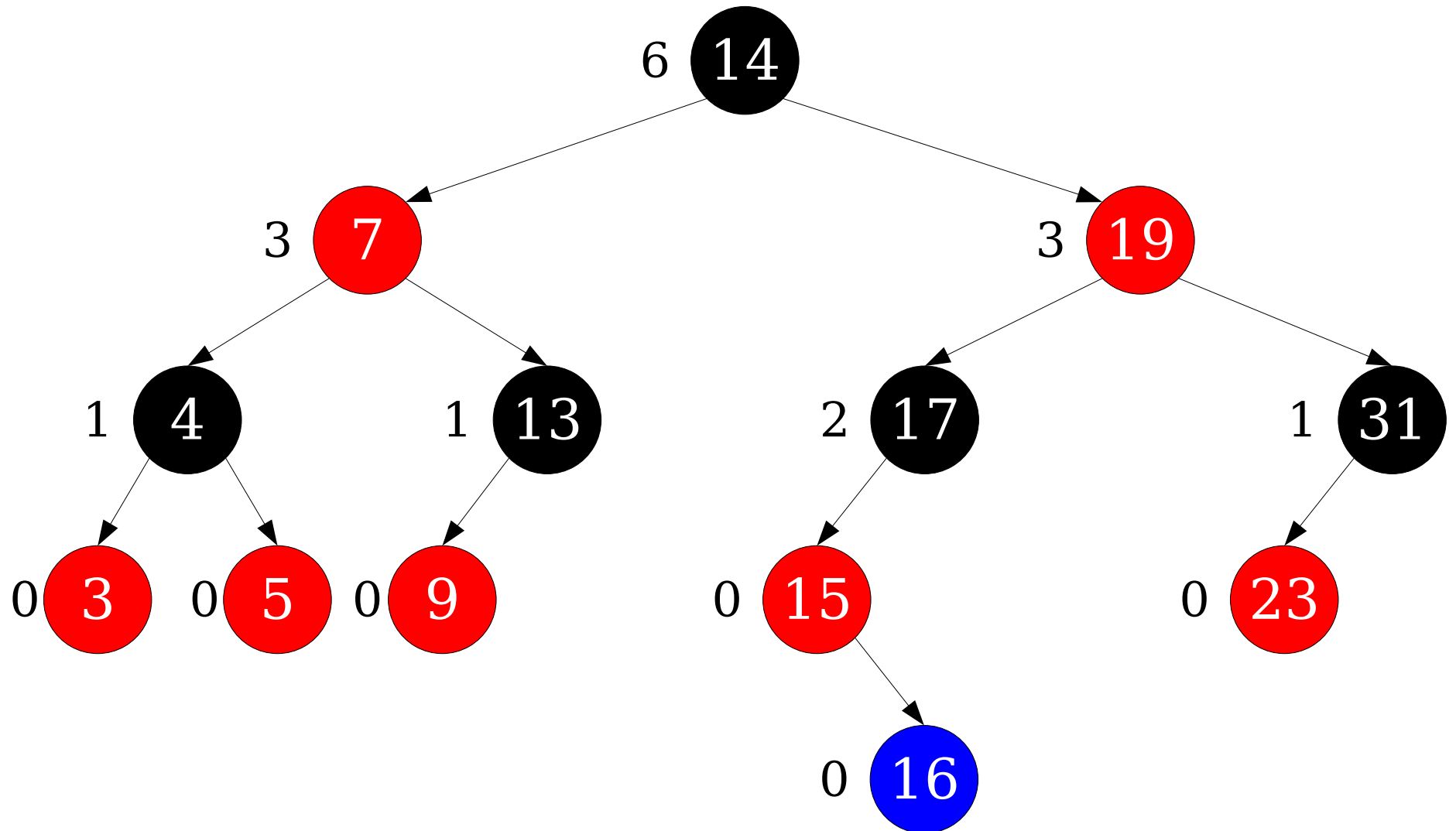
Dynamic Selection



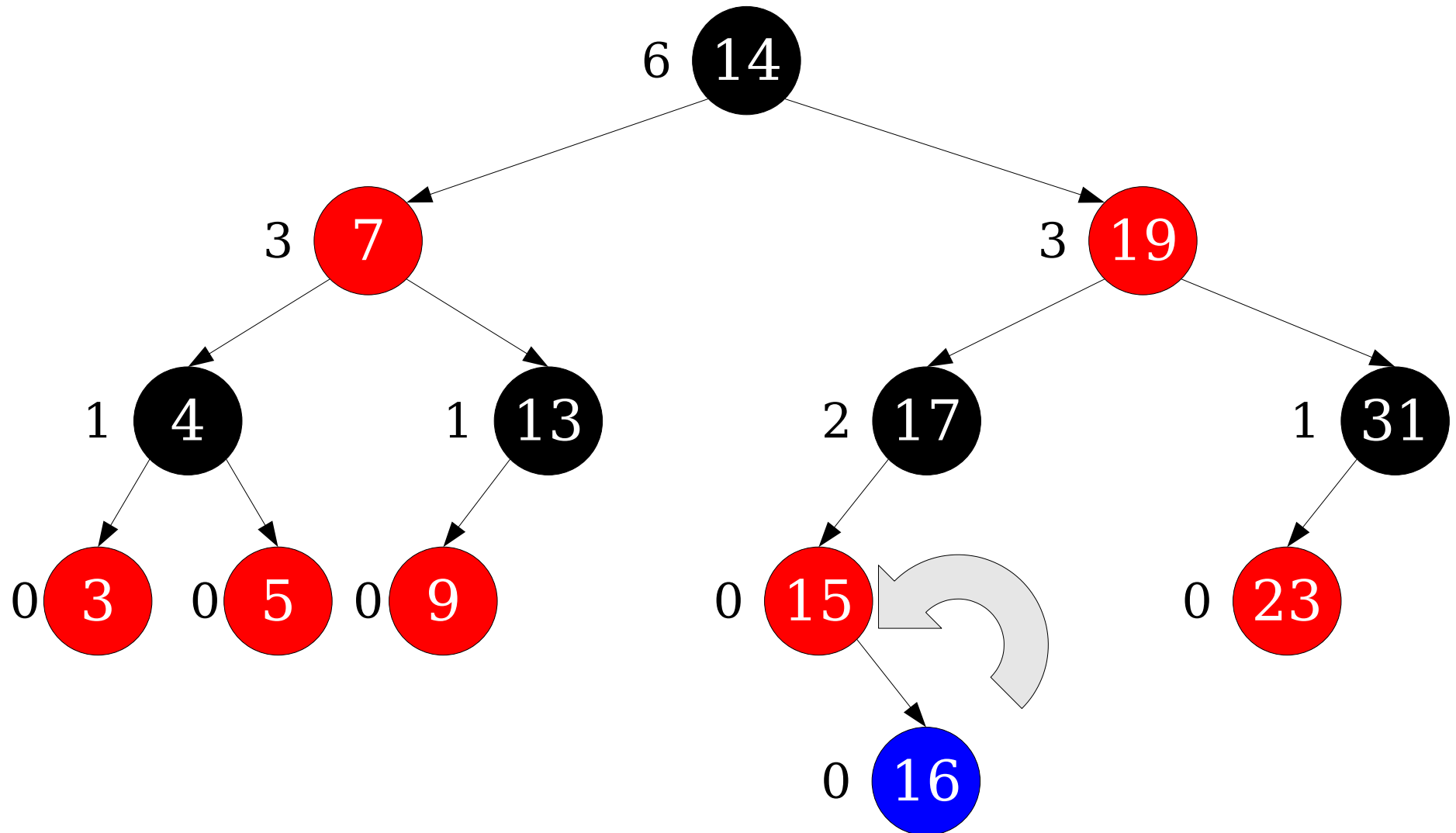
Dynamic Selection



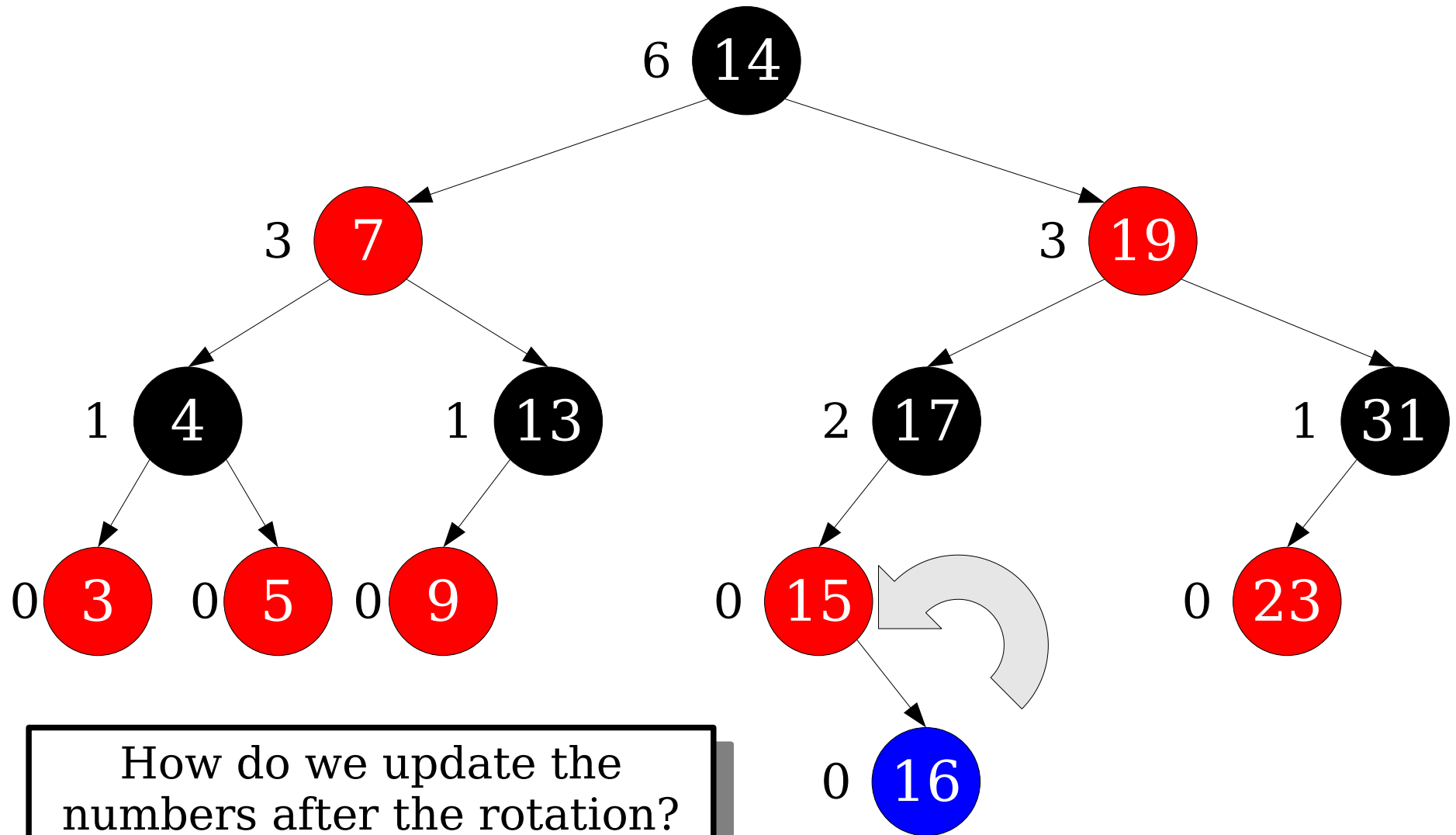
Dynamic Selection

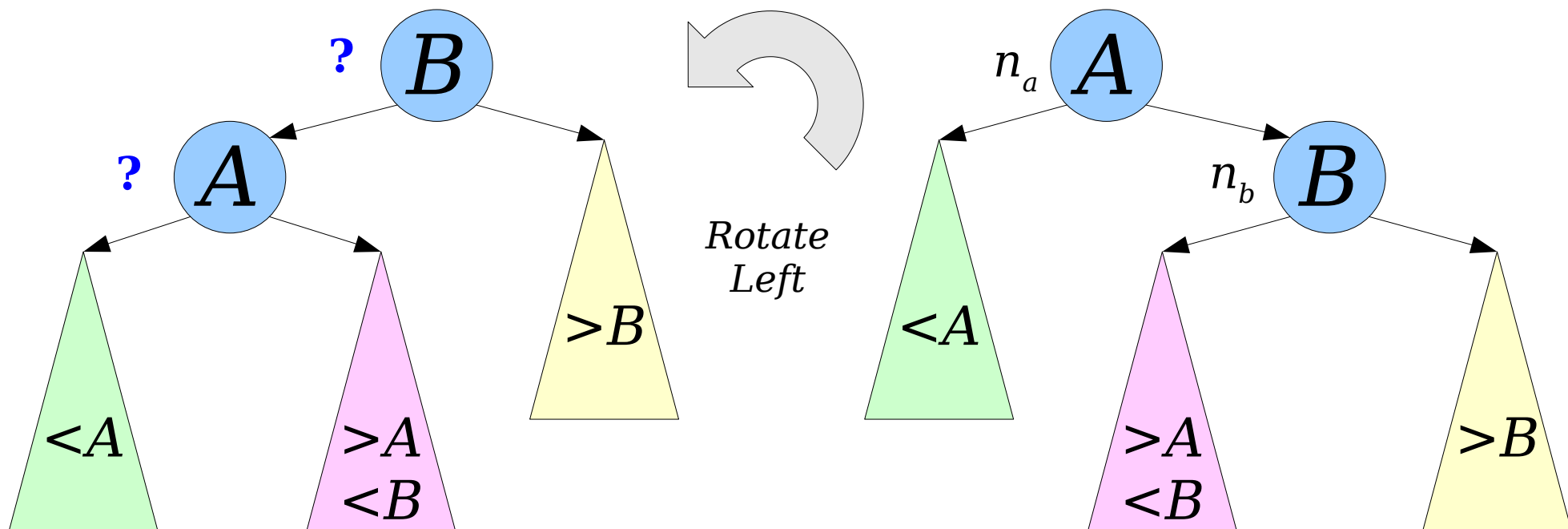
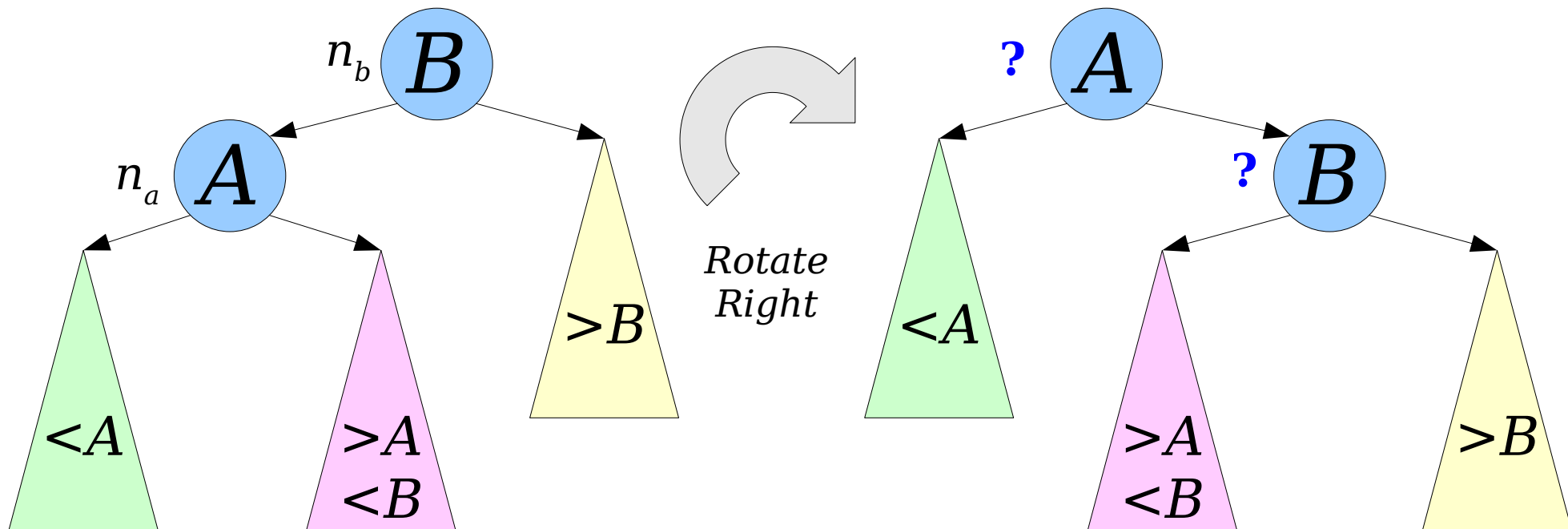


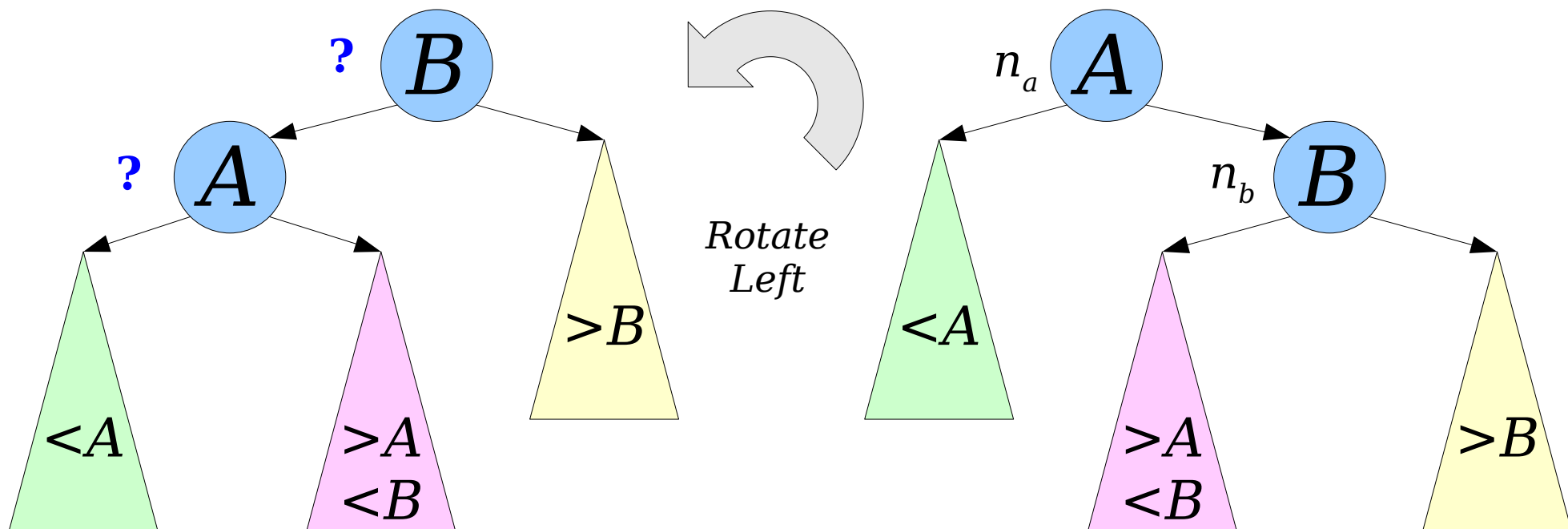
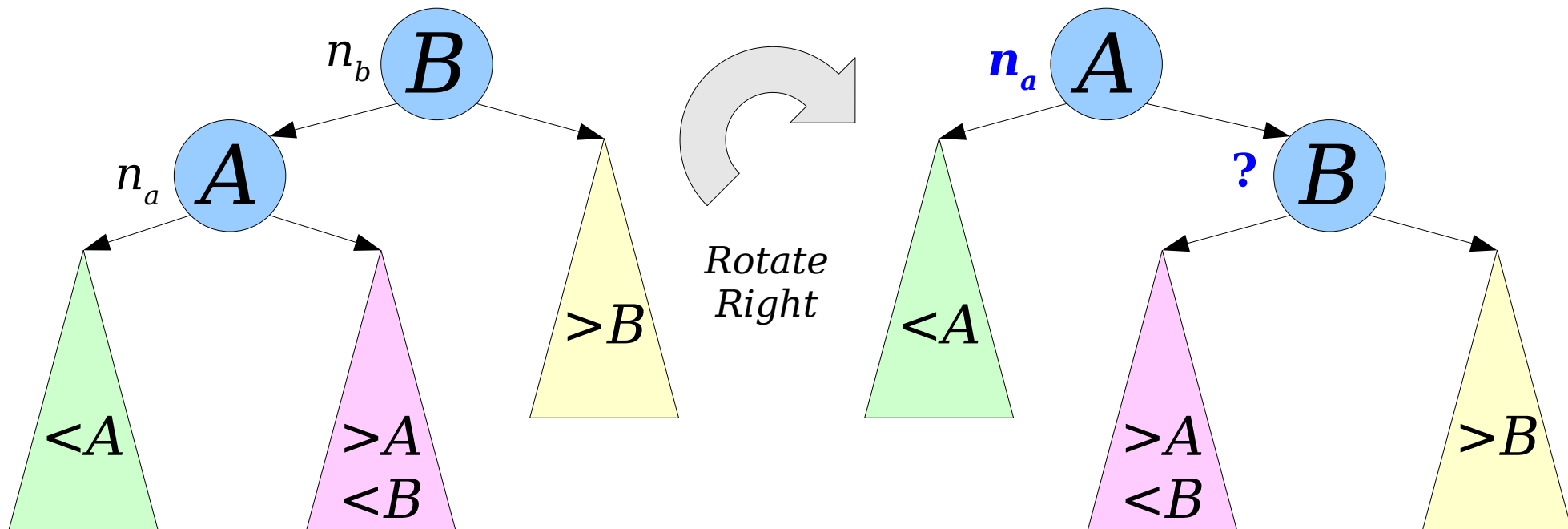
Dynamic Selection

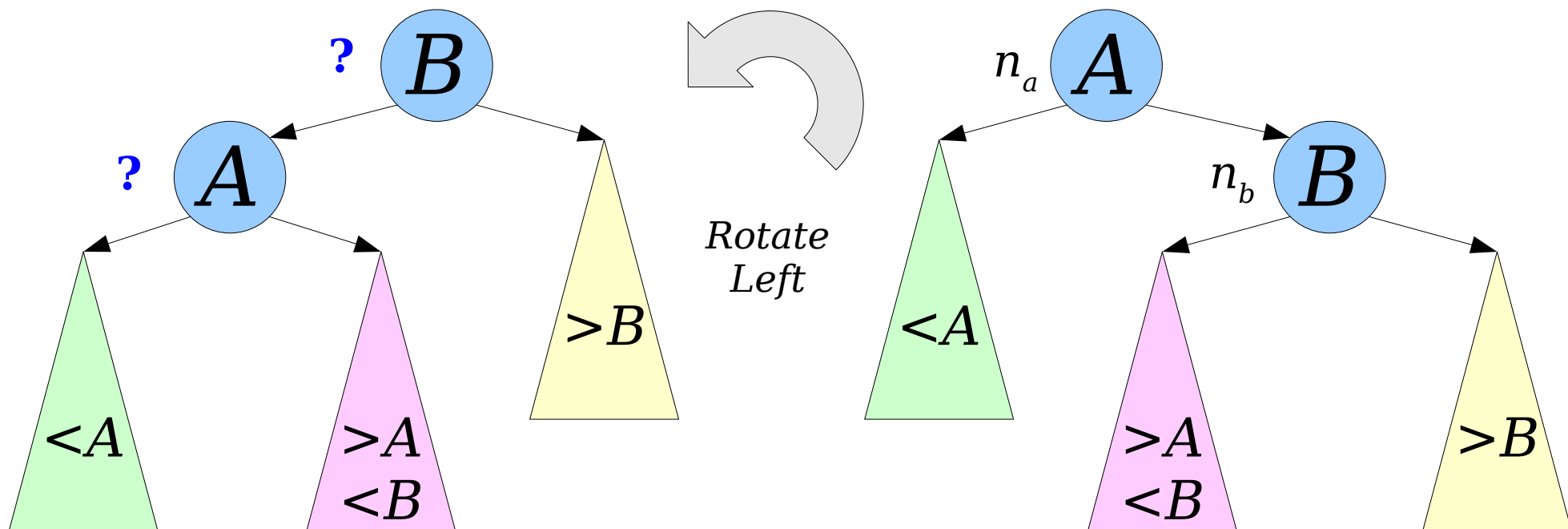
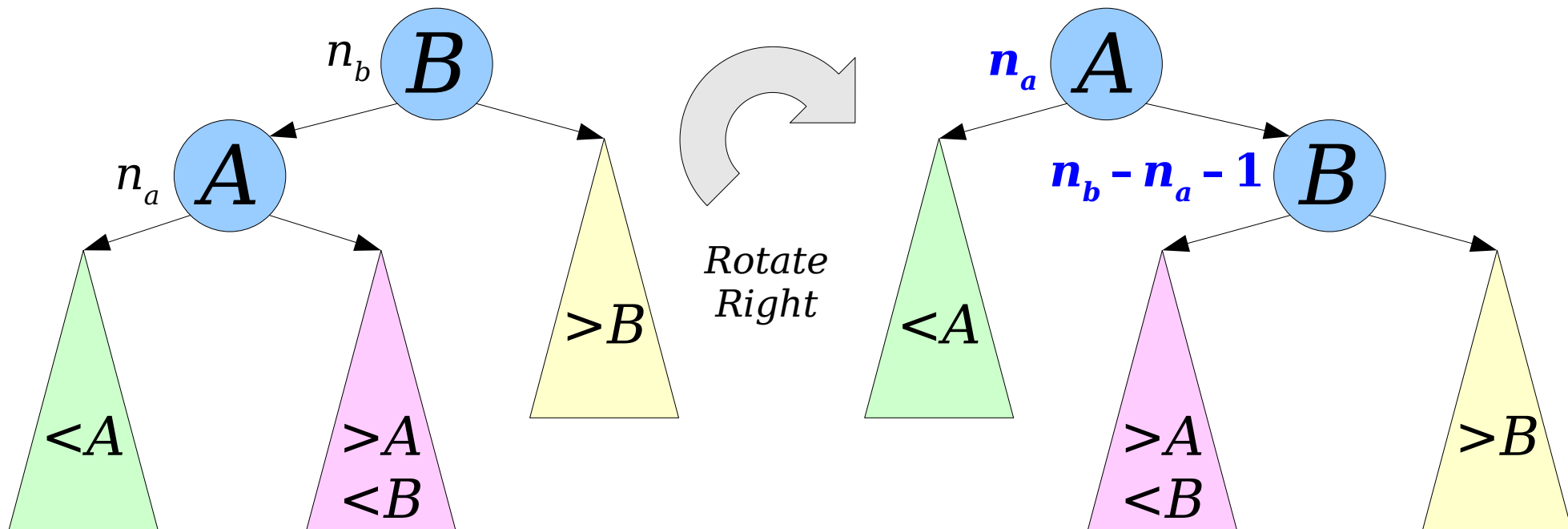


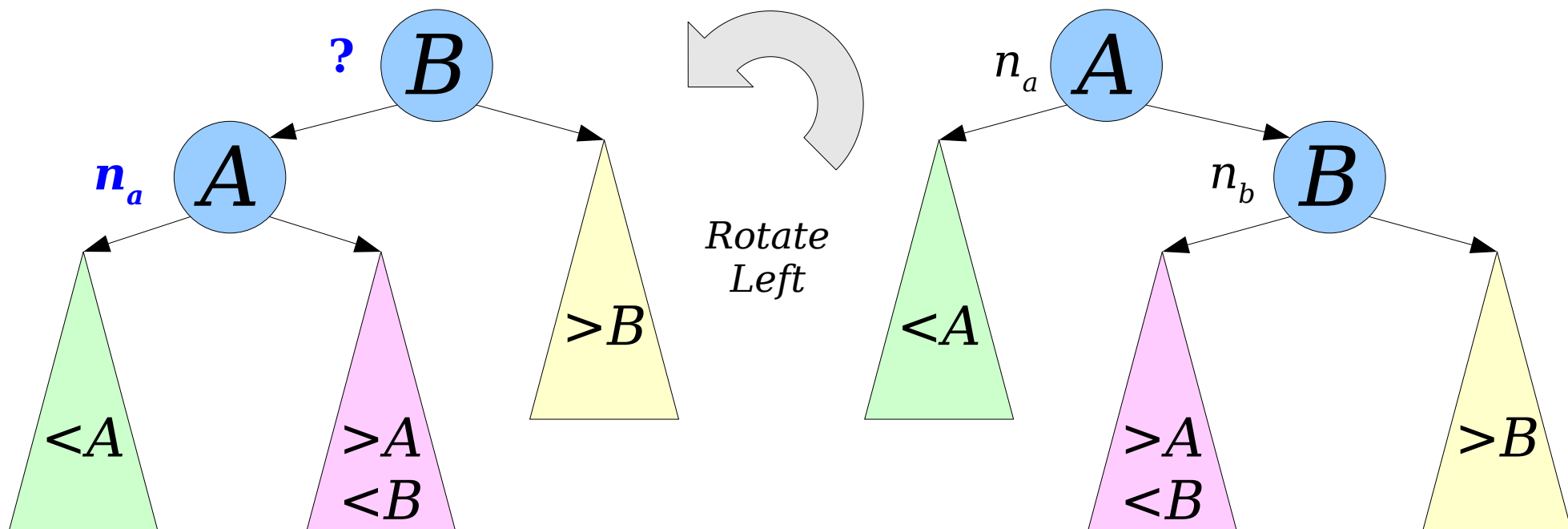
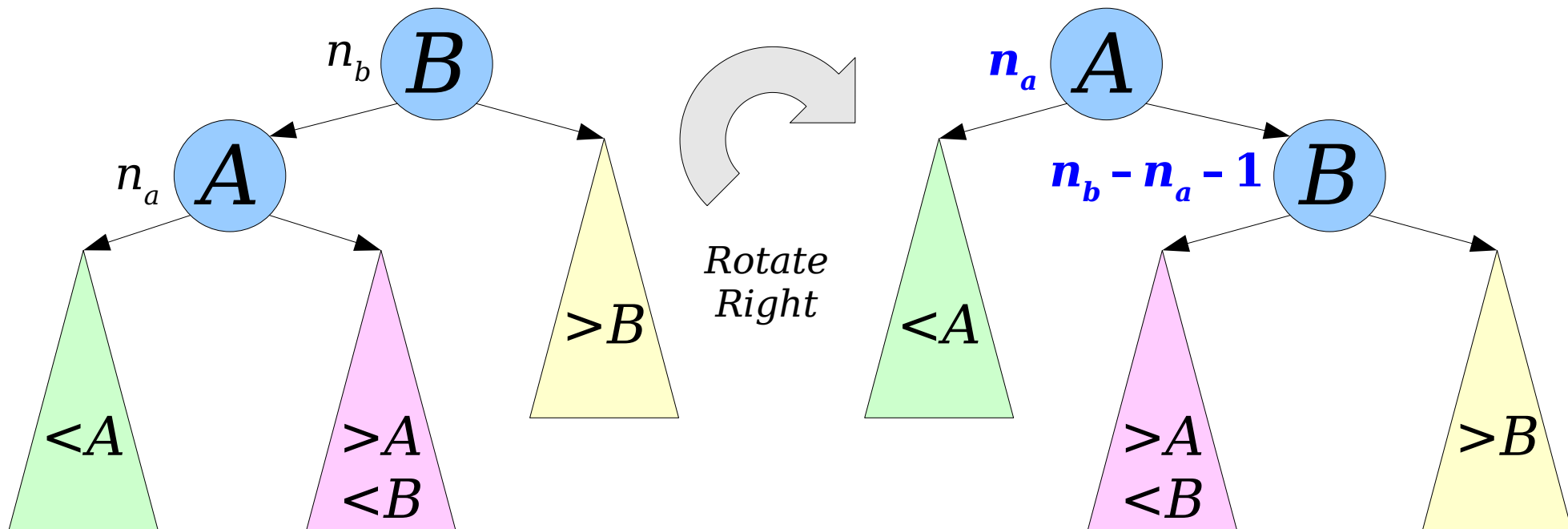
Dynamic Selection

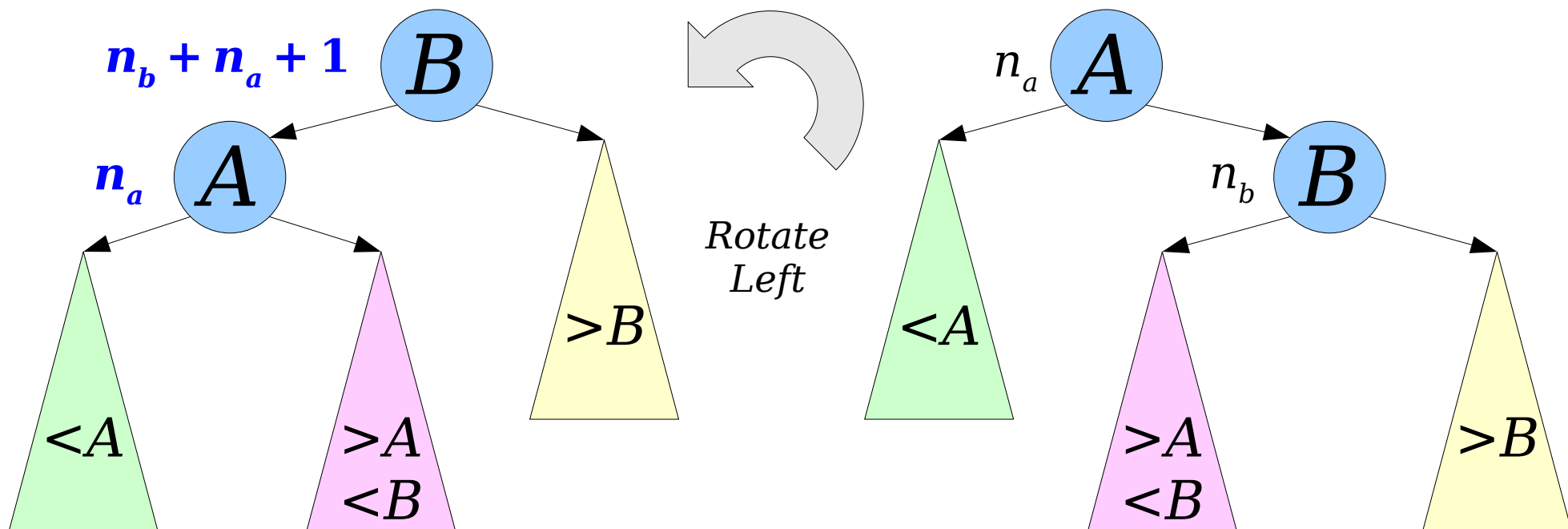
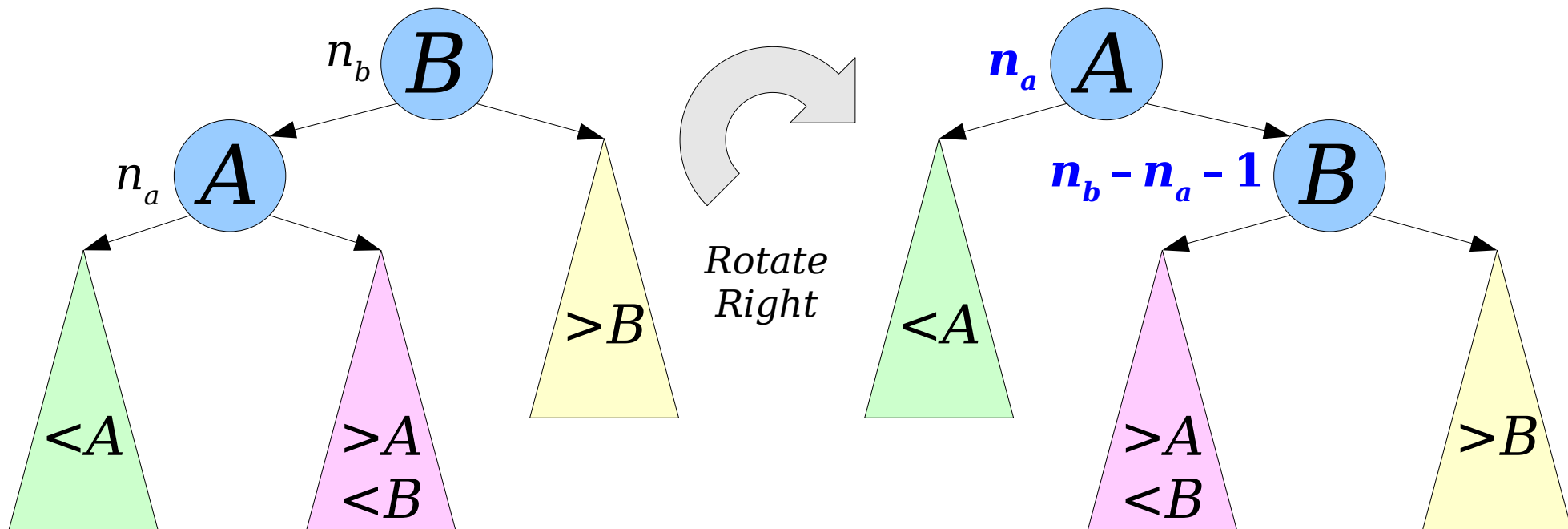




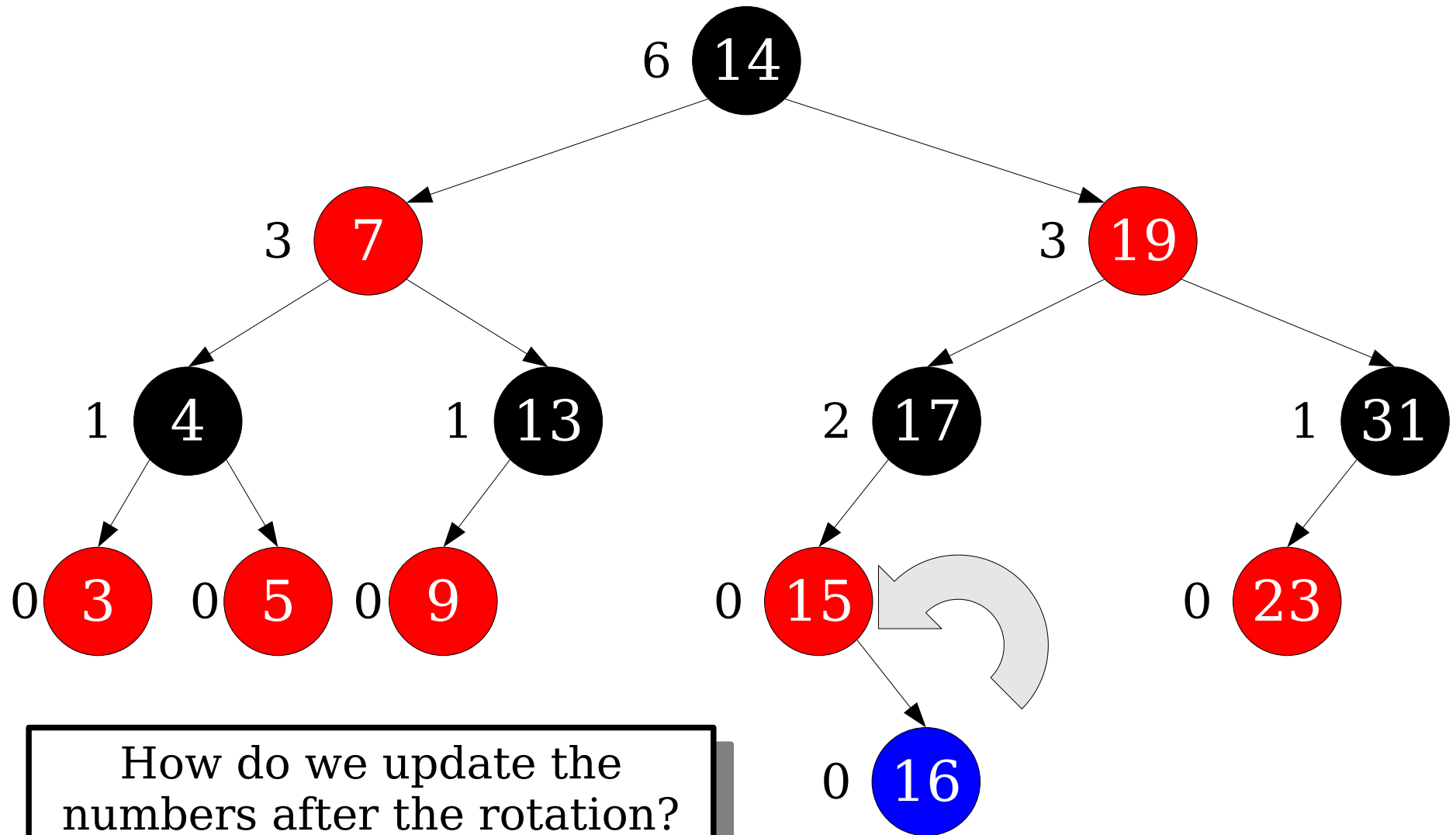




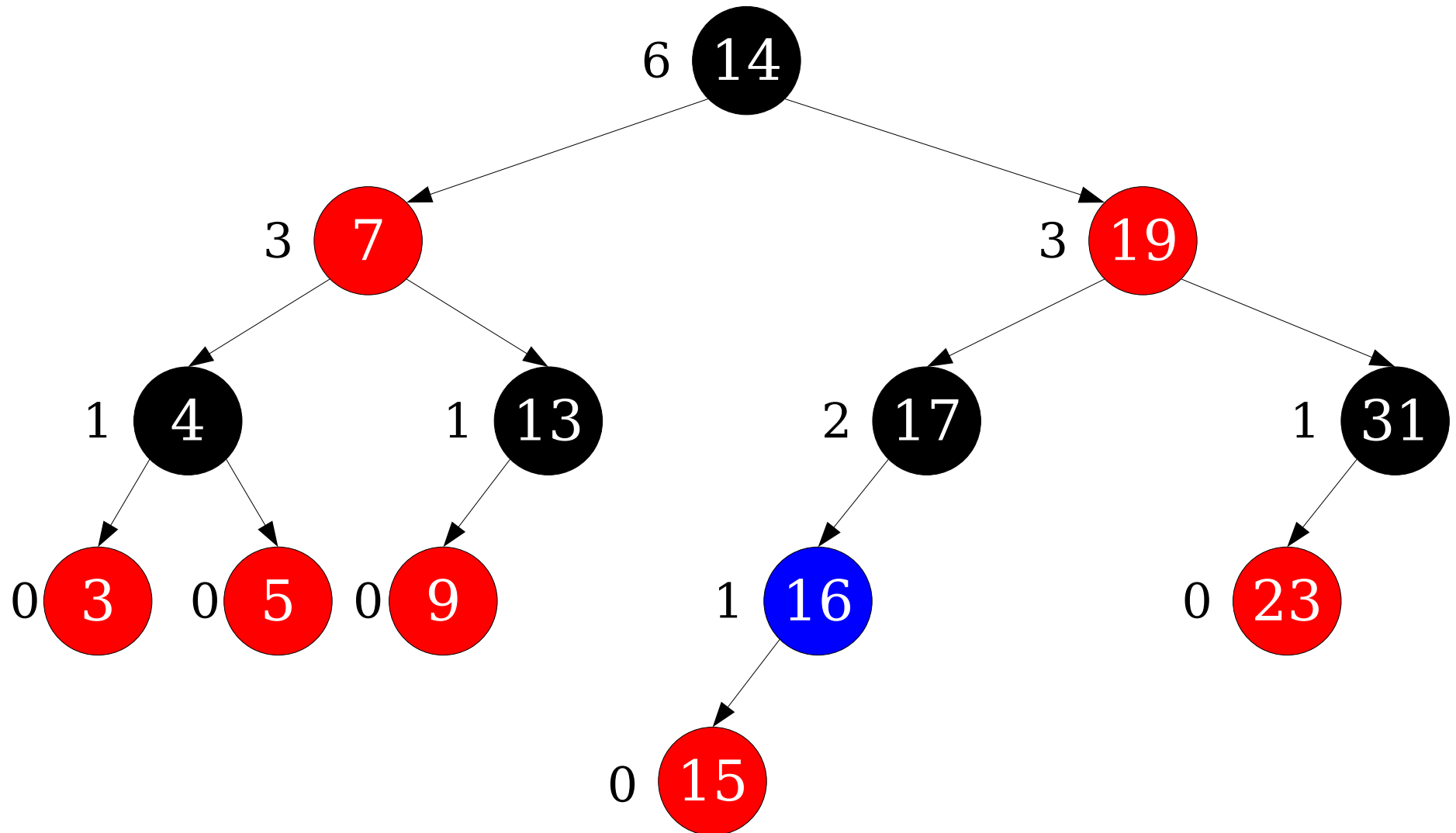




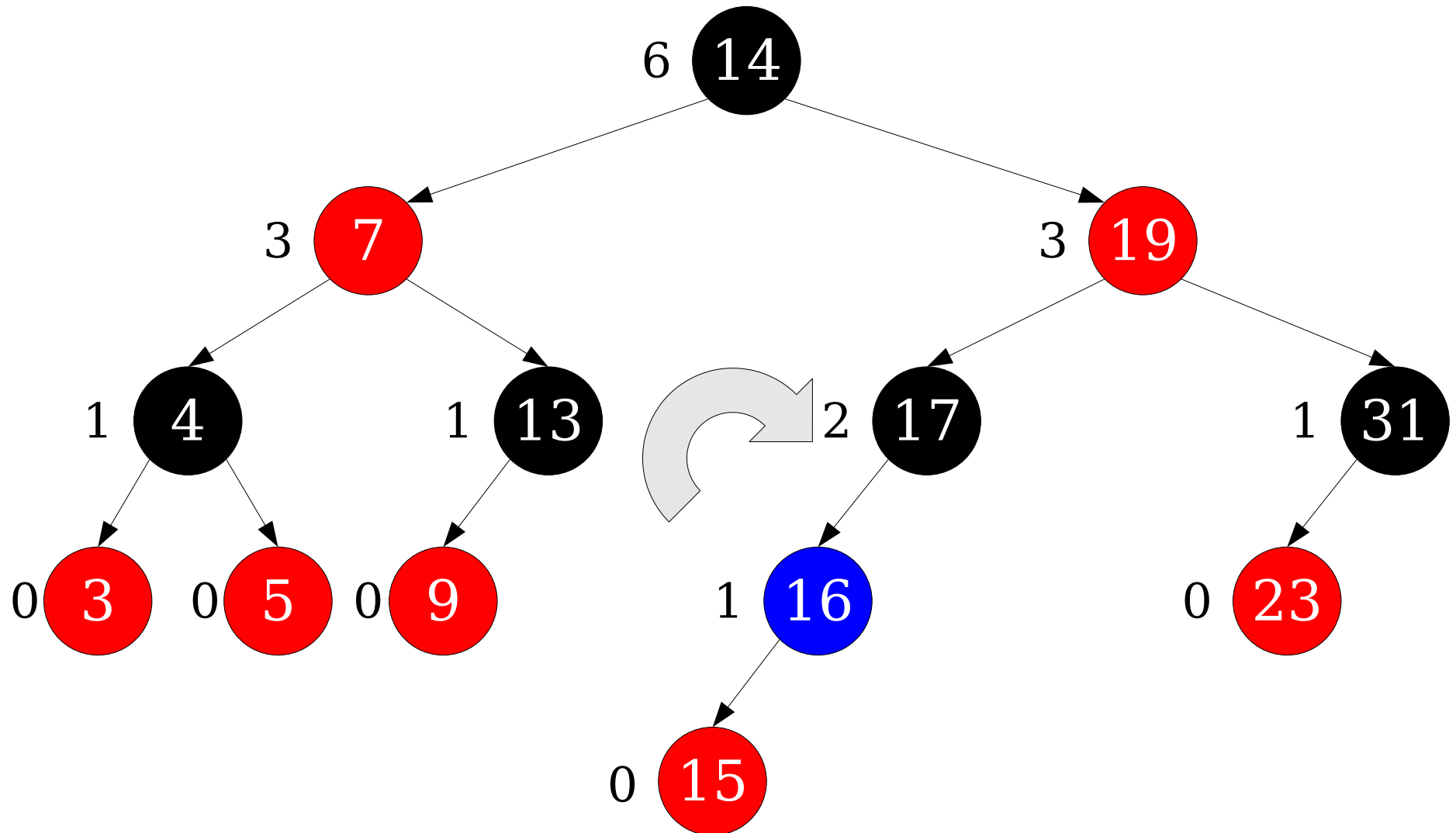
Dynamic Selection



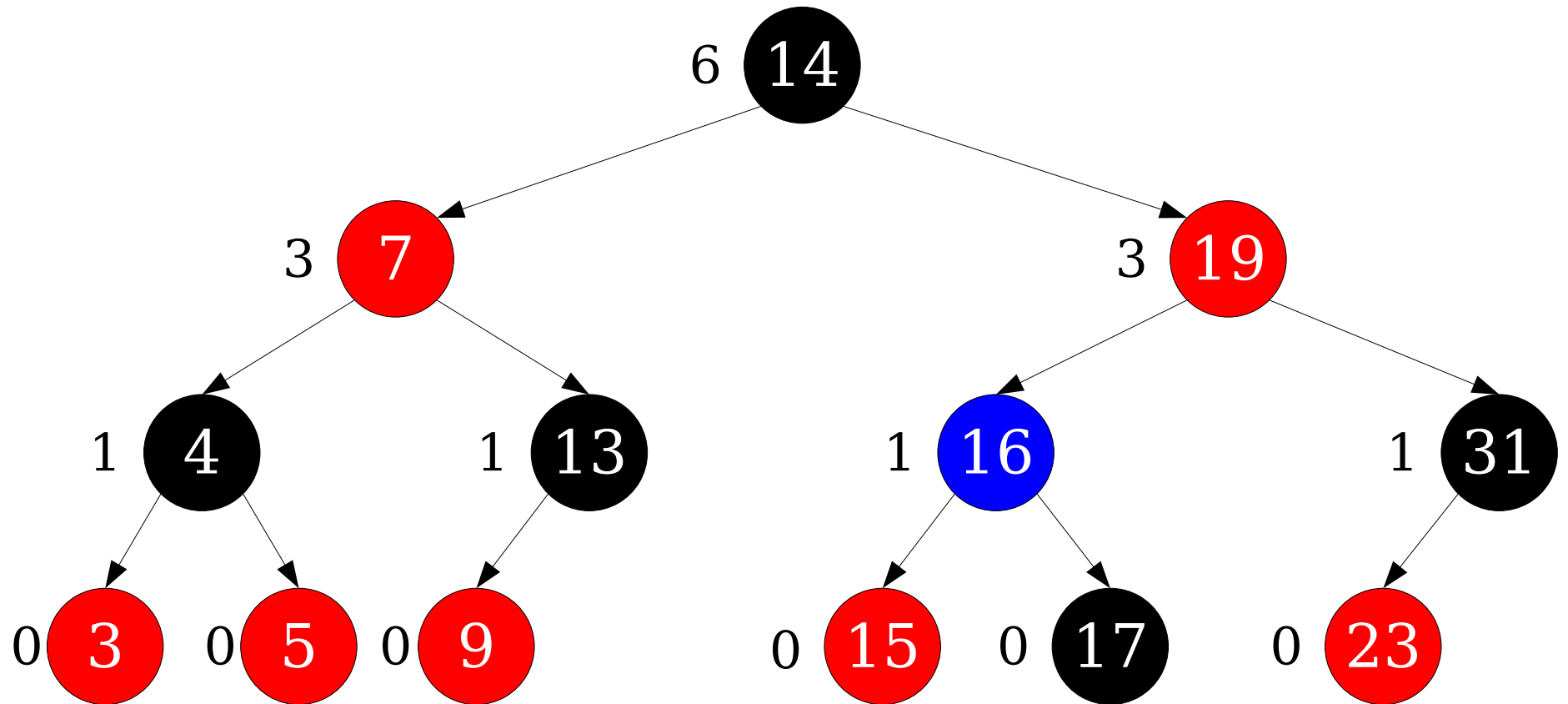
Dynamic Selection



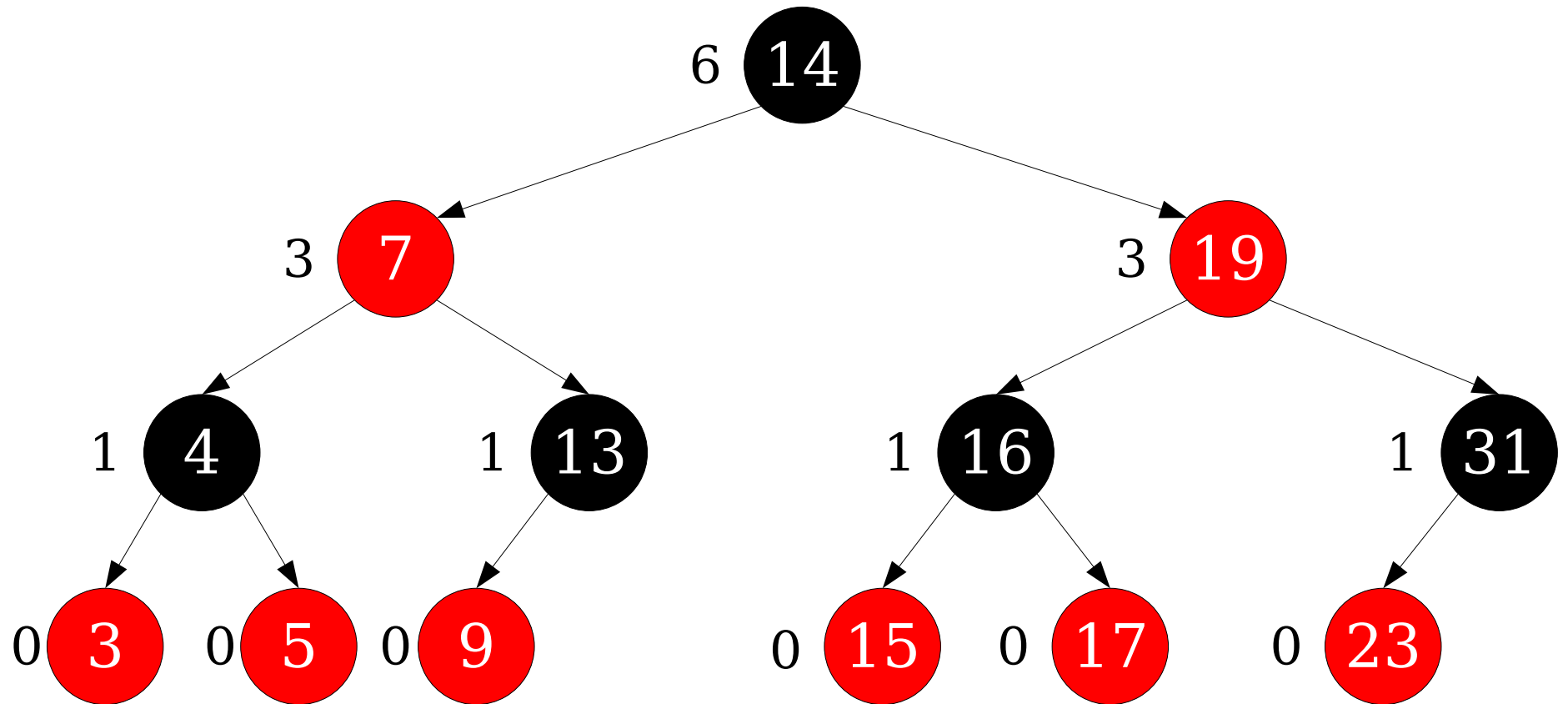
Dynamic Selection



Dynamic Selection



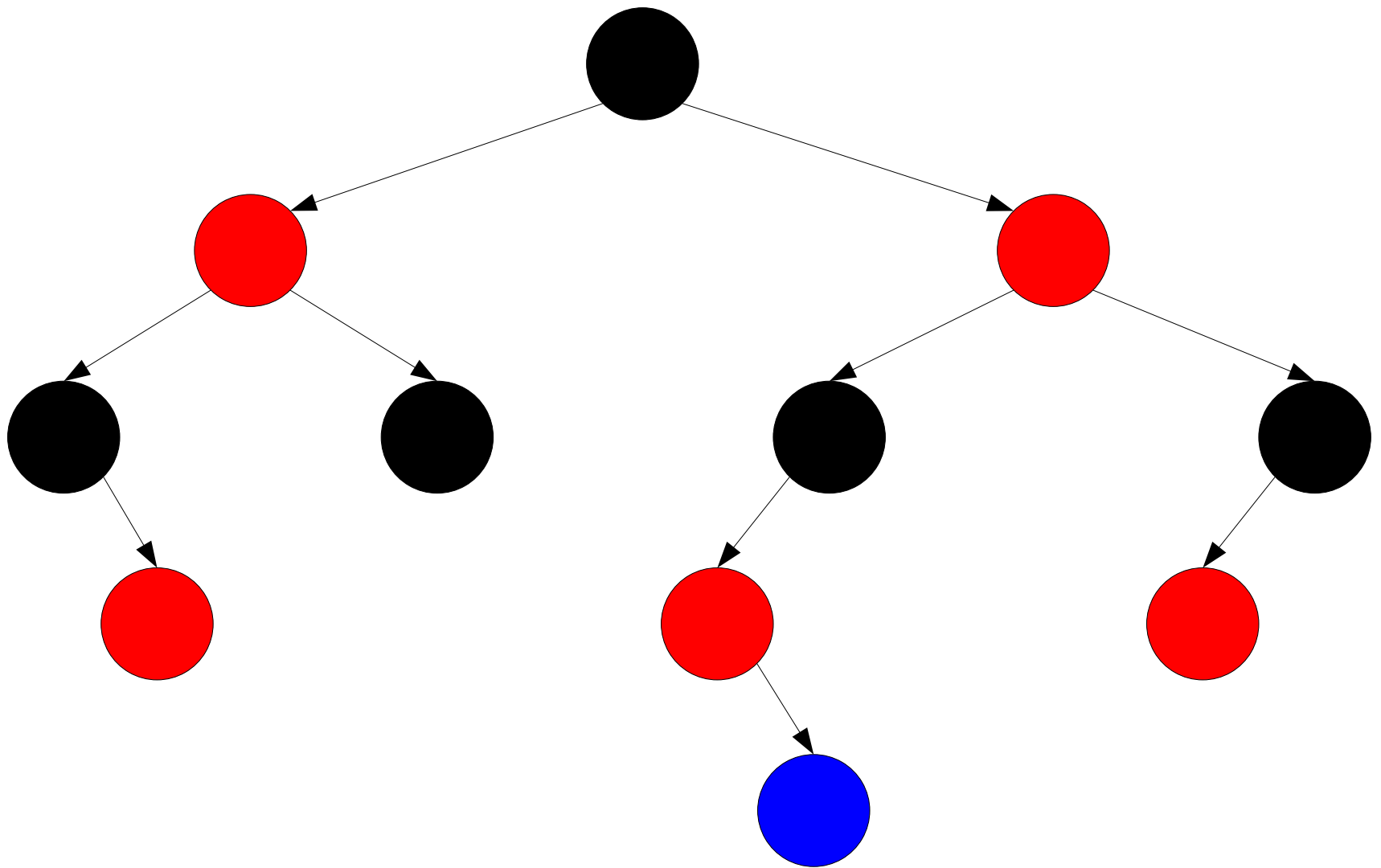
Dynamic Selection



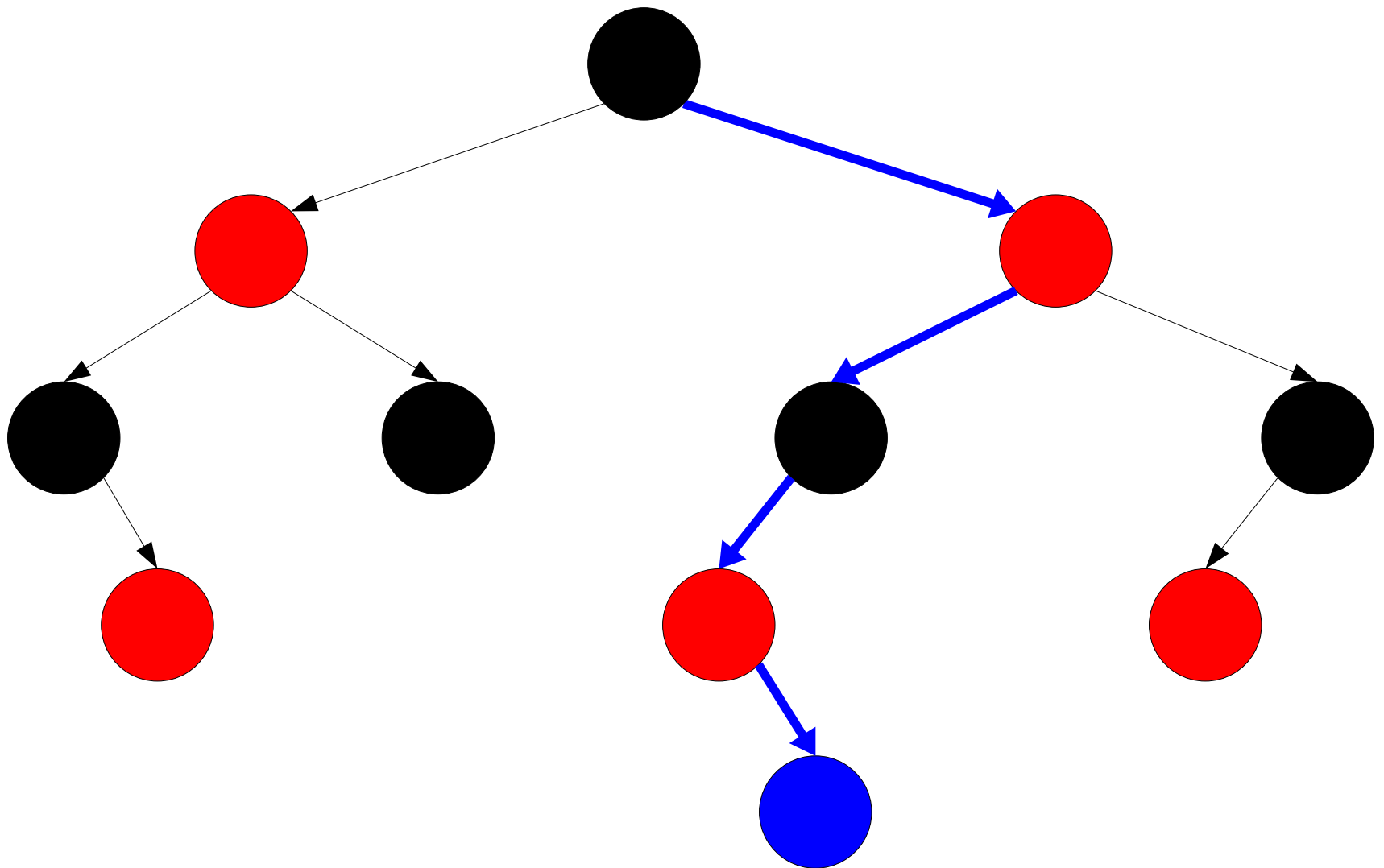
Order Statistic Trees

- This modified red/black tree is called an **order statistics tree**.
 - Start with a red/black tree.
 - Tag each node with the number of nodes in its left subtree.
 - Use the preceding update rules to preserve values during rotations.
 - Propagate other changes up to the root of the tree.
- Only $O(\log n)$ values must be updated on an insertion or deletion and each can be updated in time $O(1)$.
- Supports all BST operations plus **select** (find k th order statistic) and **rank** (given a key, report its order statistic) in time $O(\log n)$.

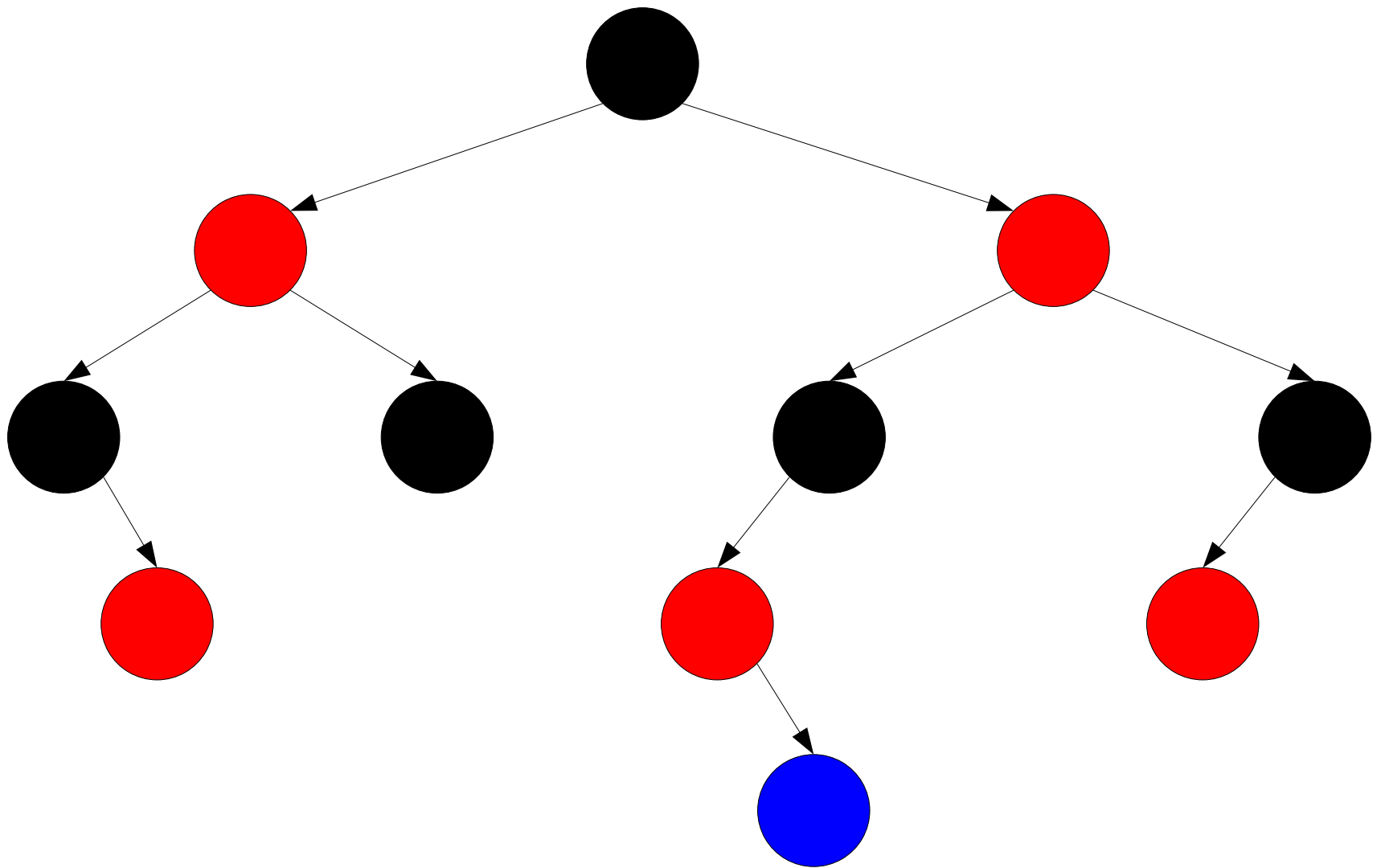
Generalizing our Idea



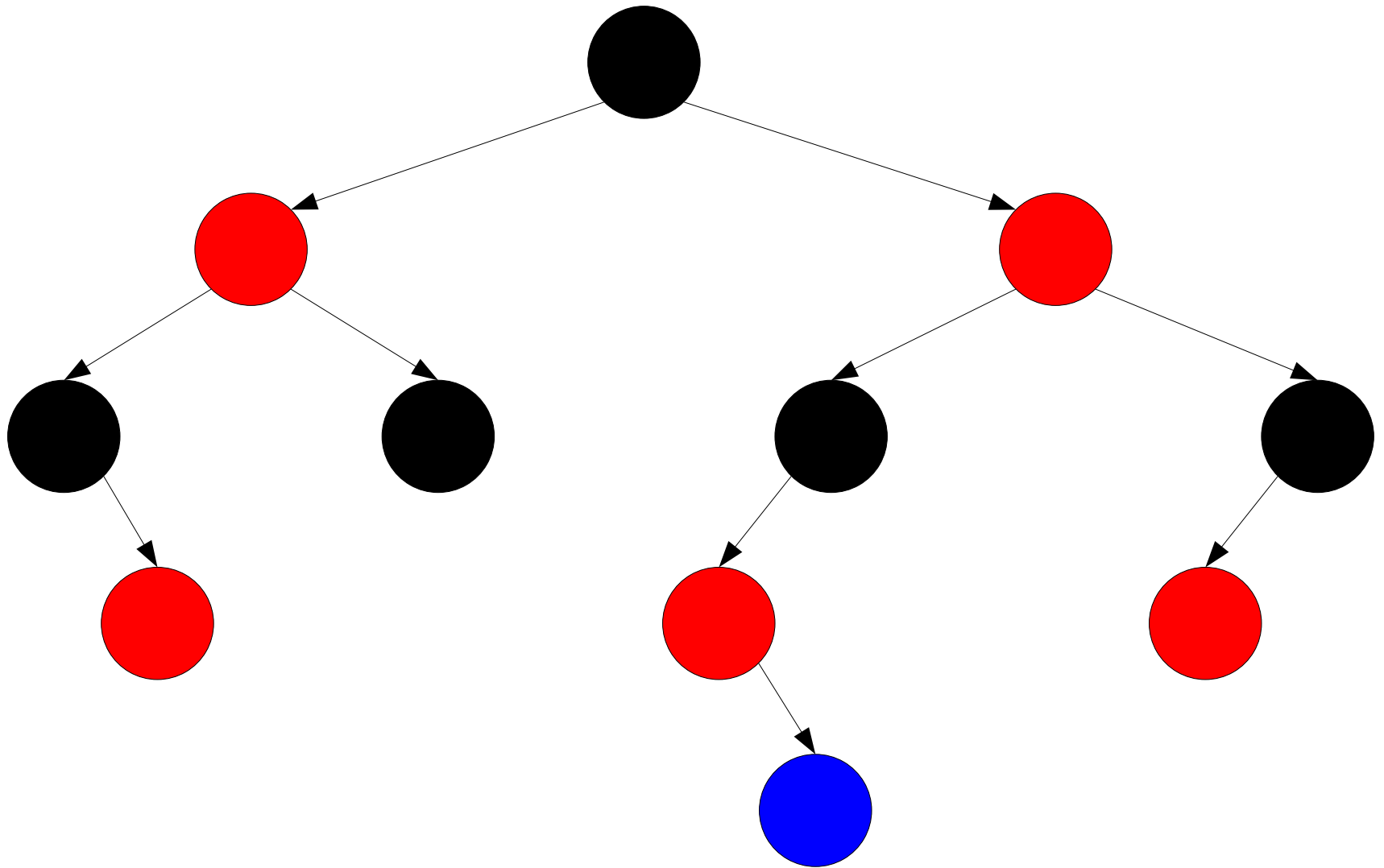
Edits to values are localized along the access path.



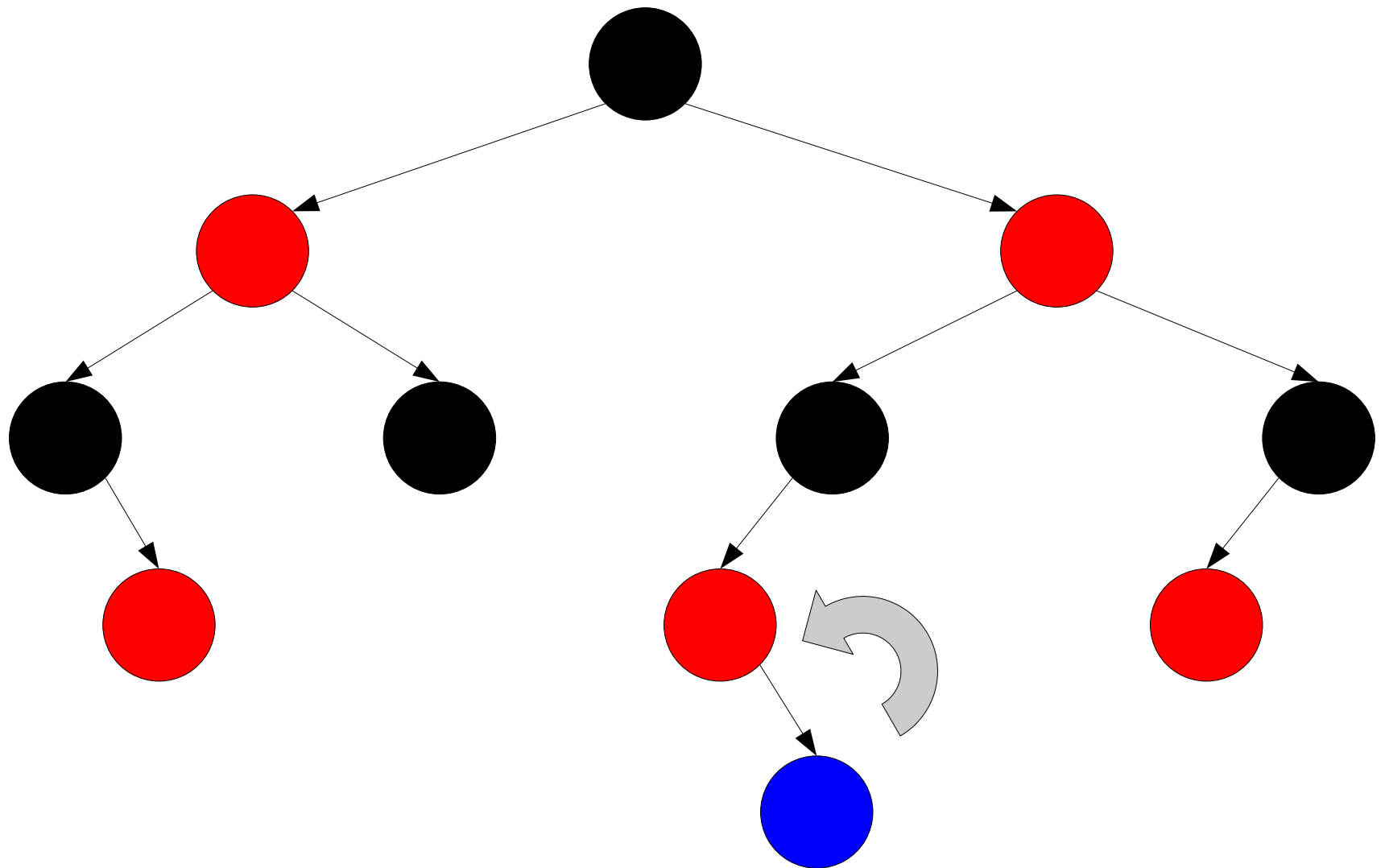
Edits to values are localized along the access path.



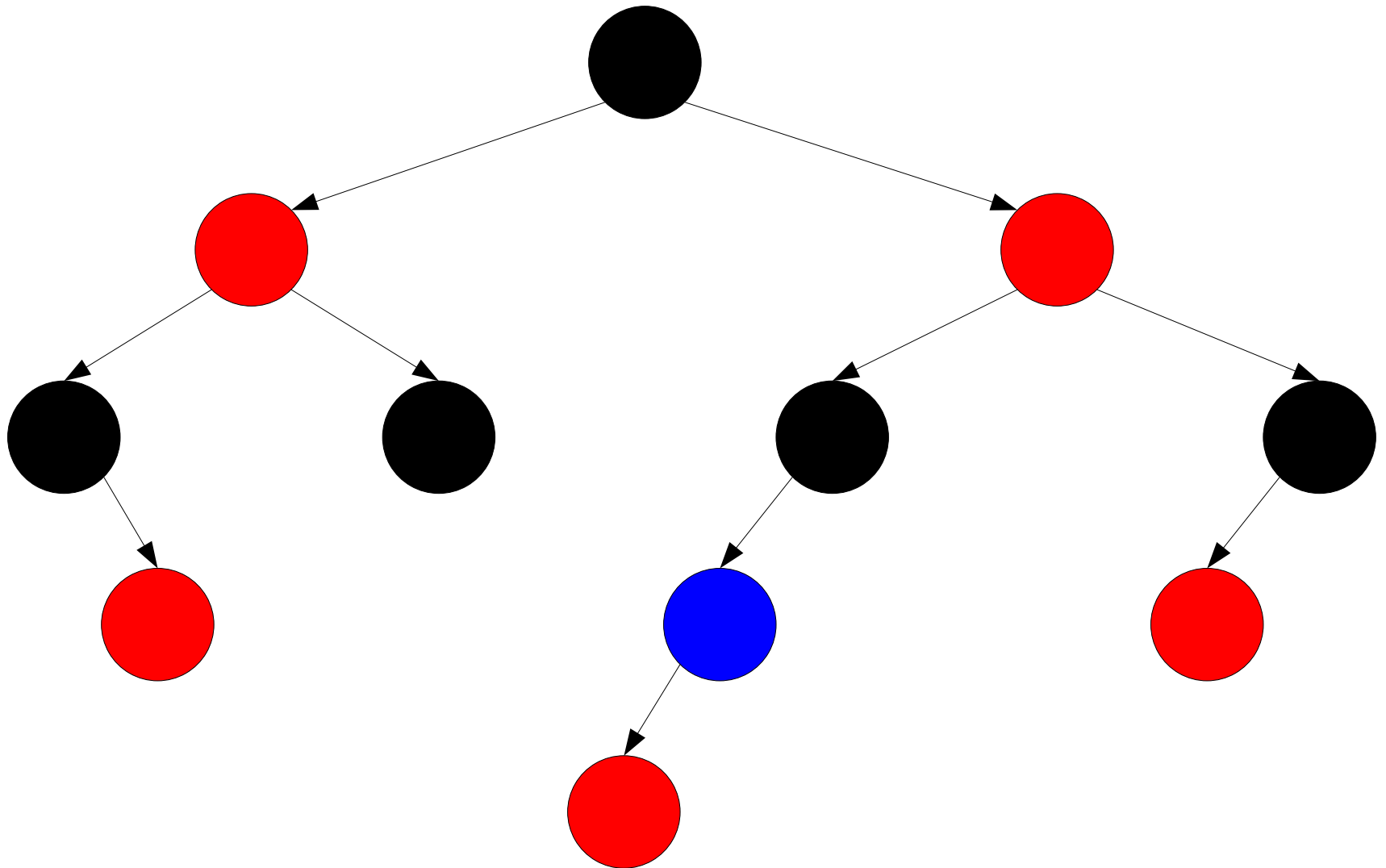
Edits to values are localized along the access path.



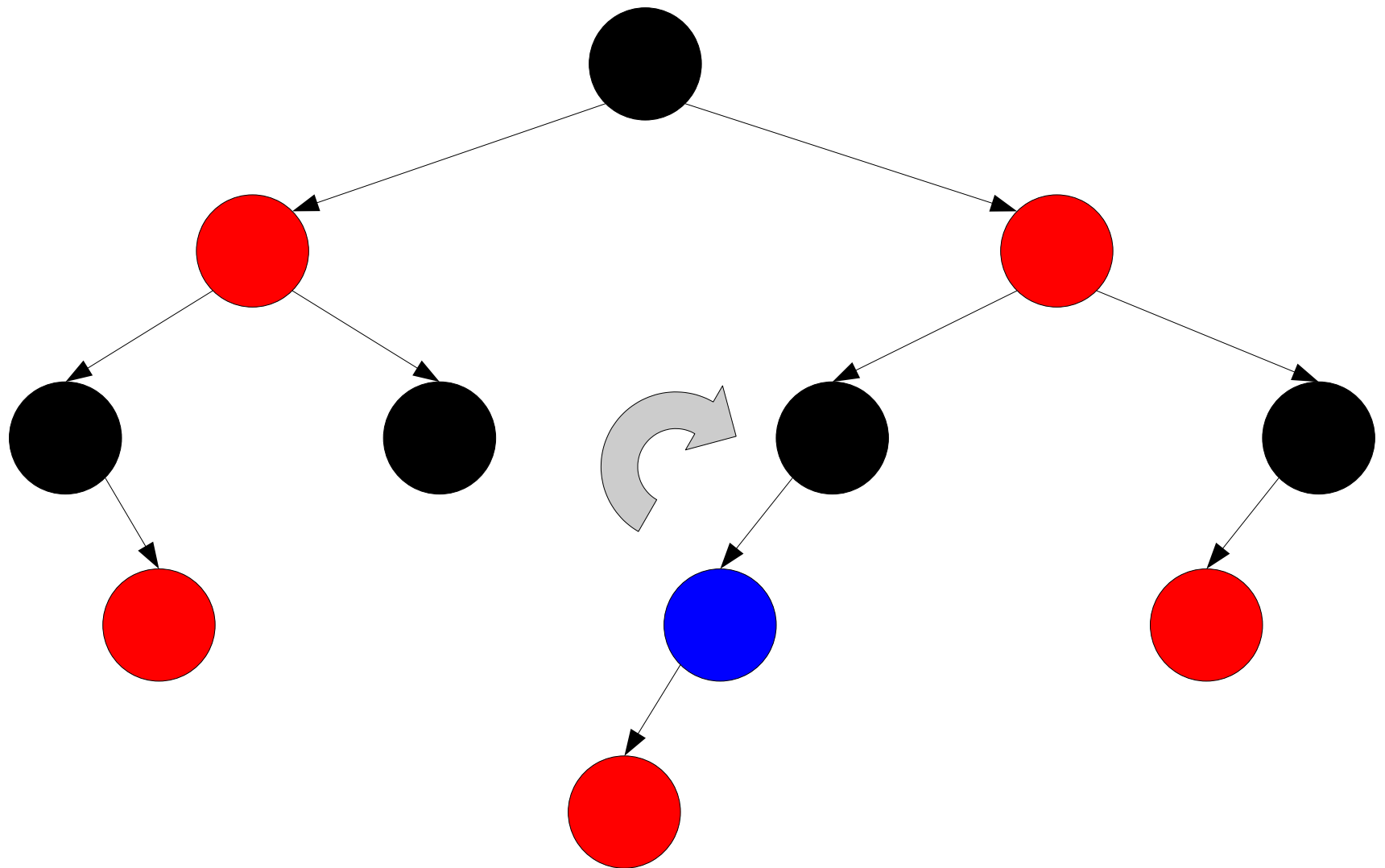
Edits to values are localized along the access path.
We can recompute values after a rotation.



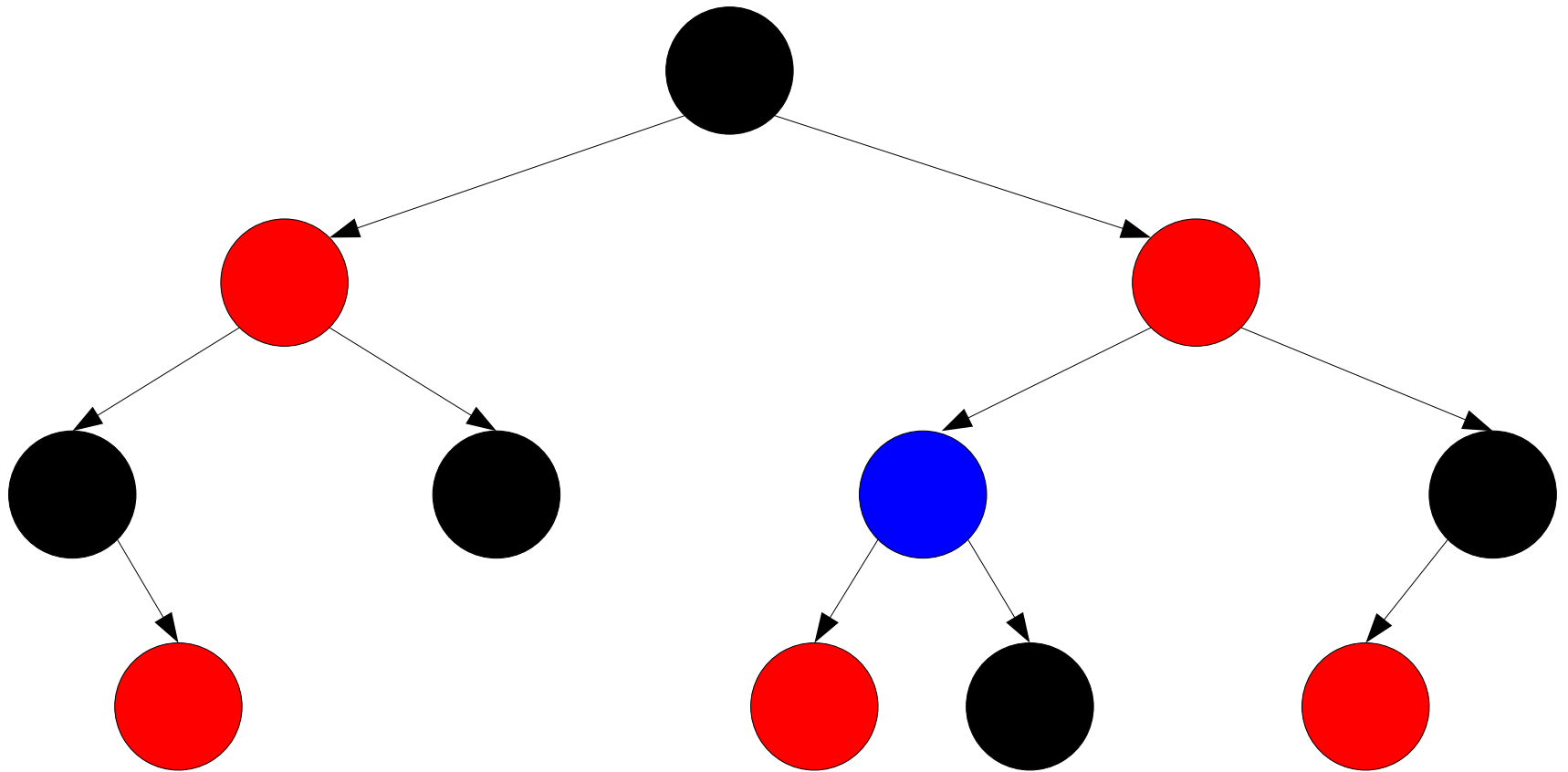
Edits to values are localized along the access path.
We can recompute values after a rotation.



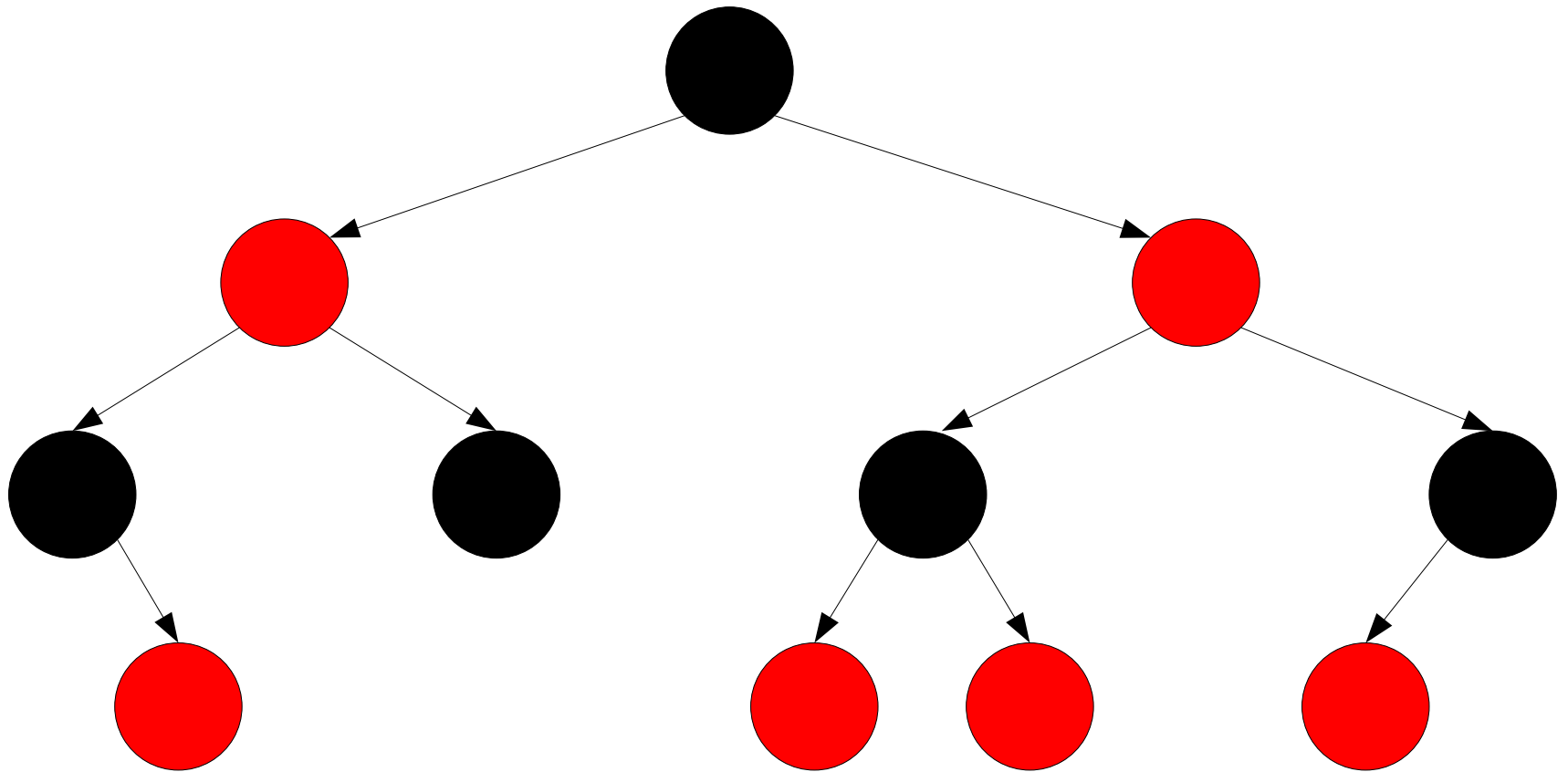
Edits to values are localized along the access path.
We can recompute values after a rotation.



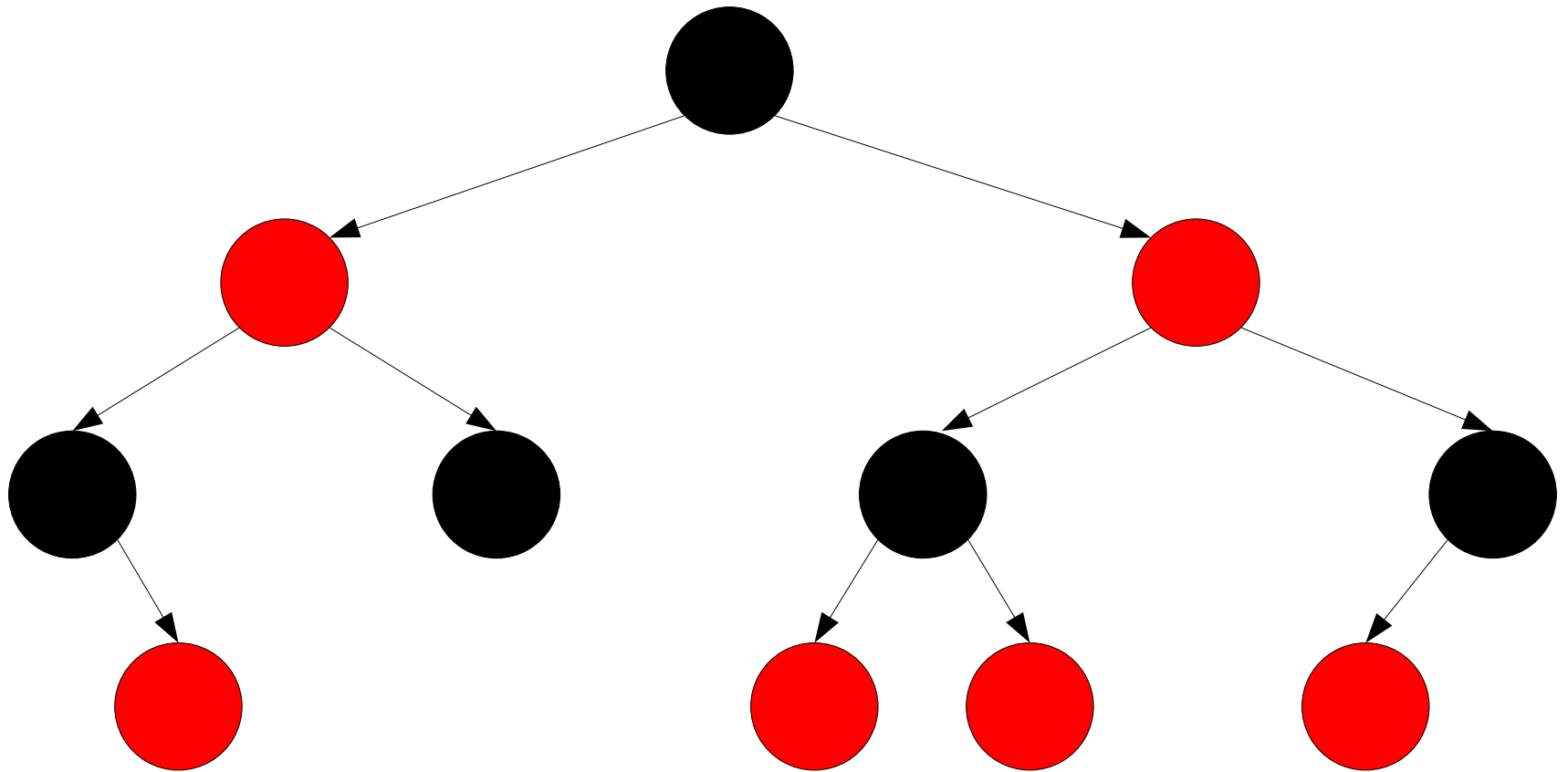
Edits to values are localized along the access path.
We can recompute values after a rotation.



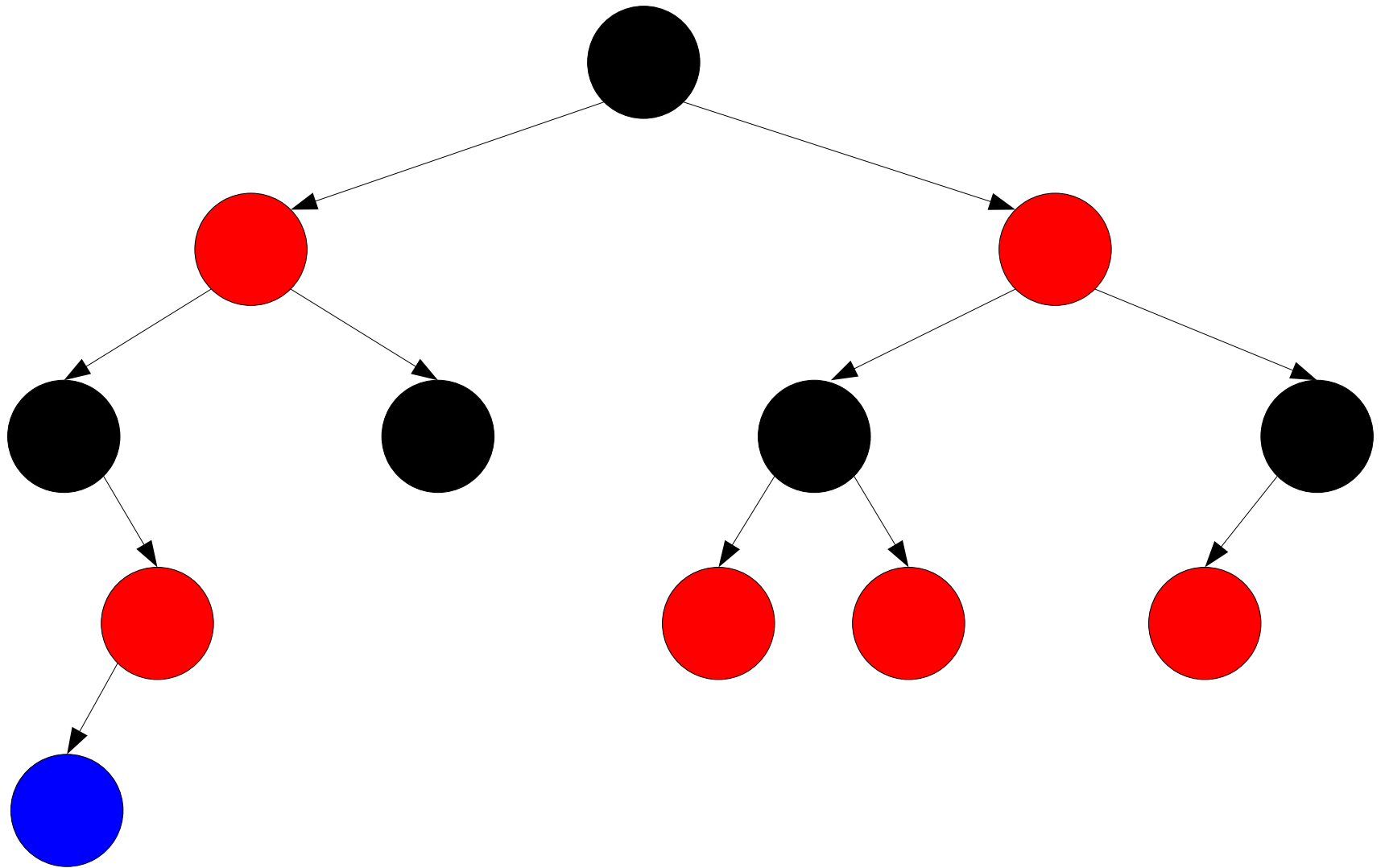
Edits to values are localized along the access path.
We can recompute values after a rotation.



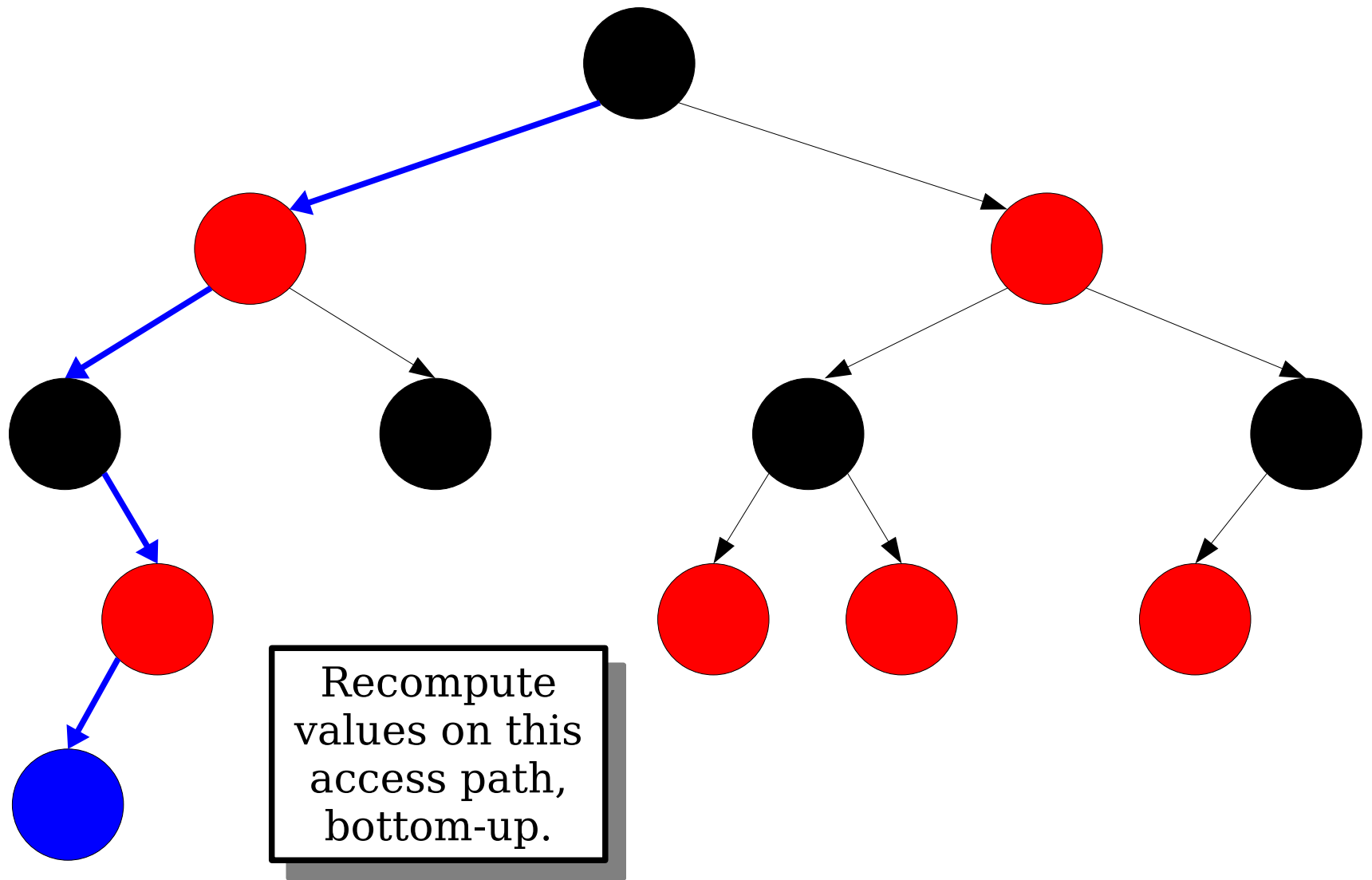
Edits to values are localized along the access path.
We can recompute values after a rotation.



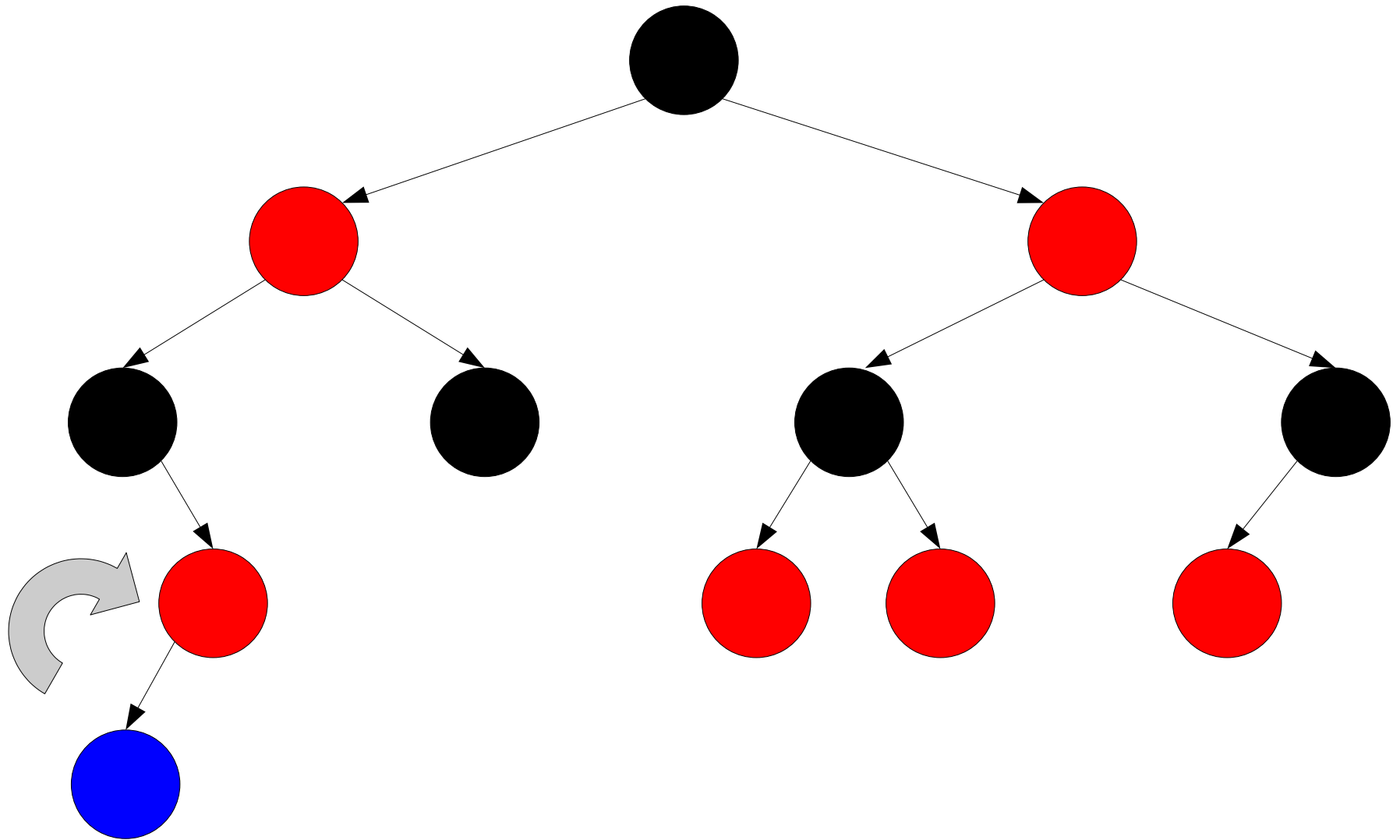
Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



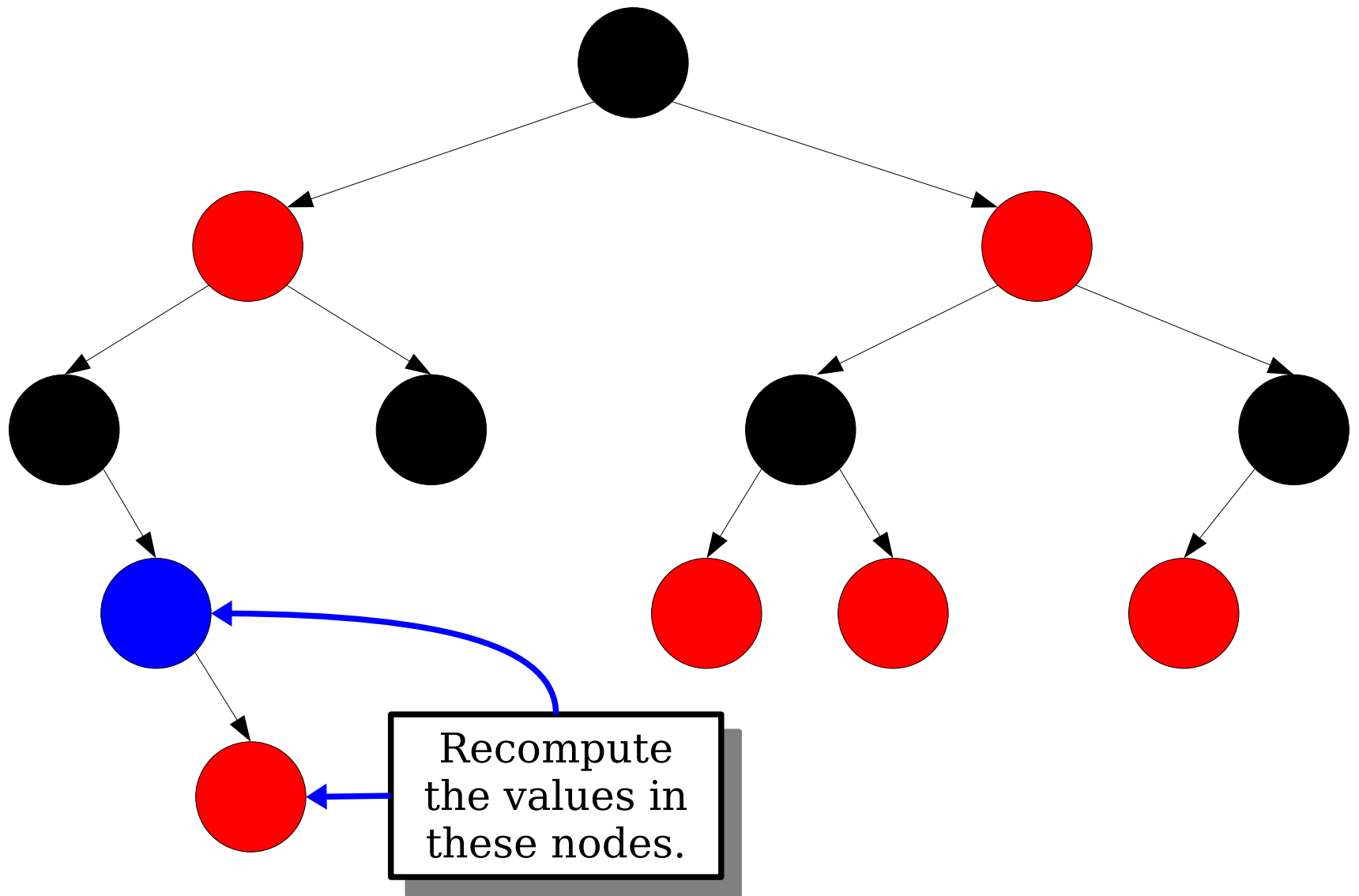
Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



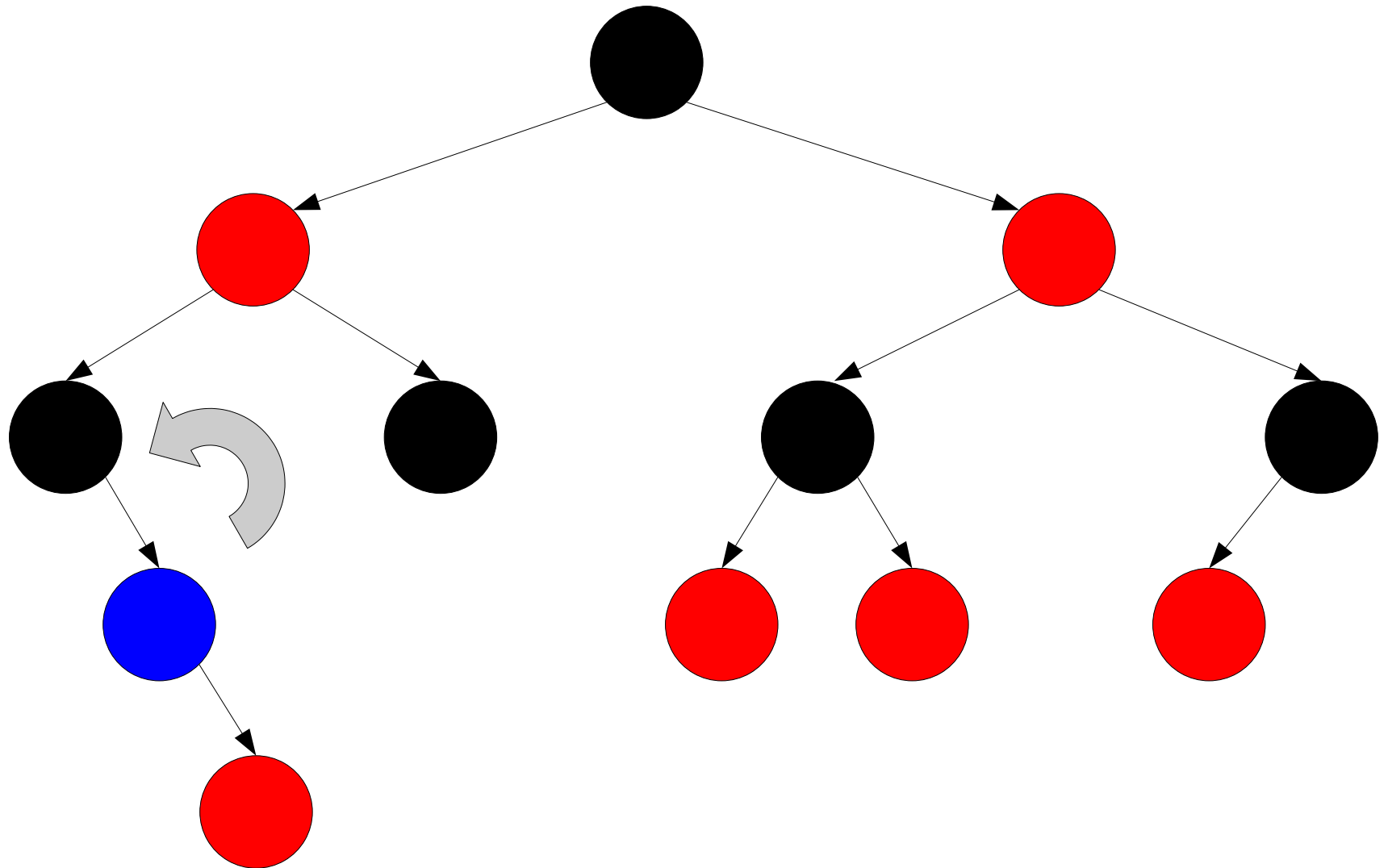
Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



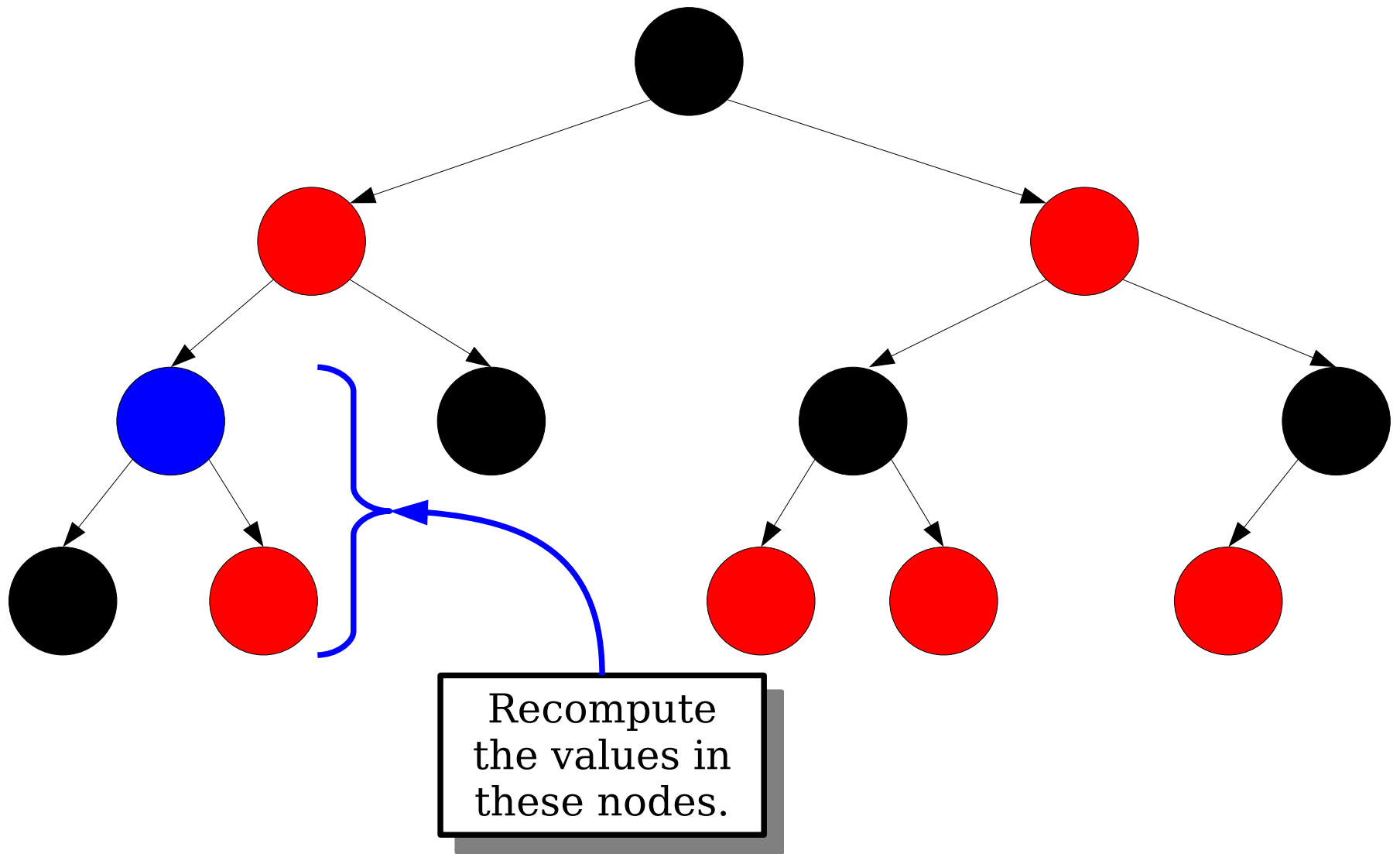
Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



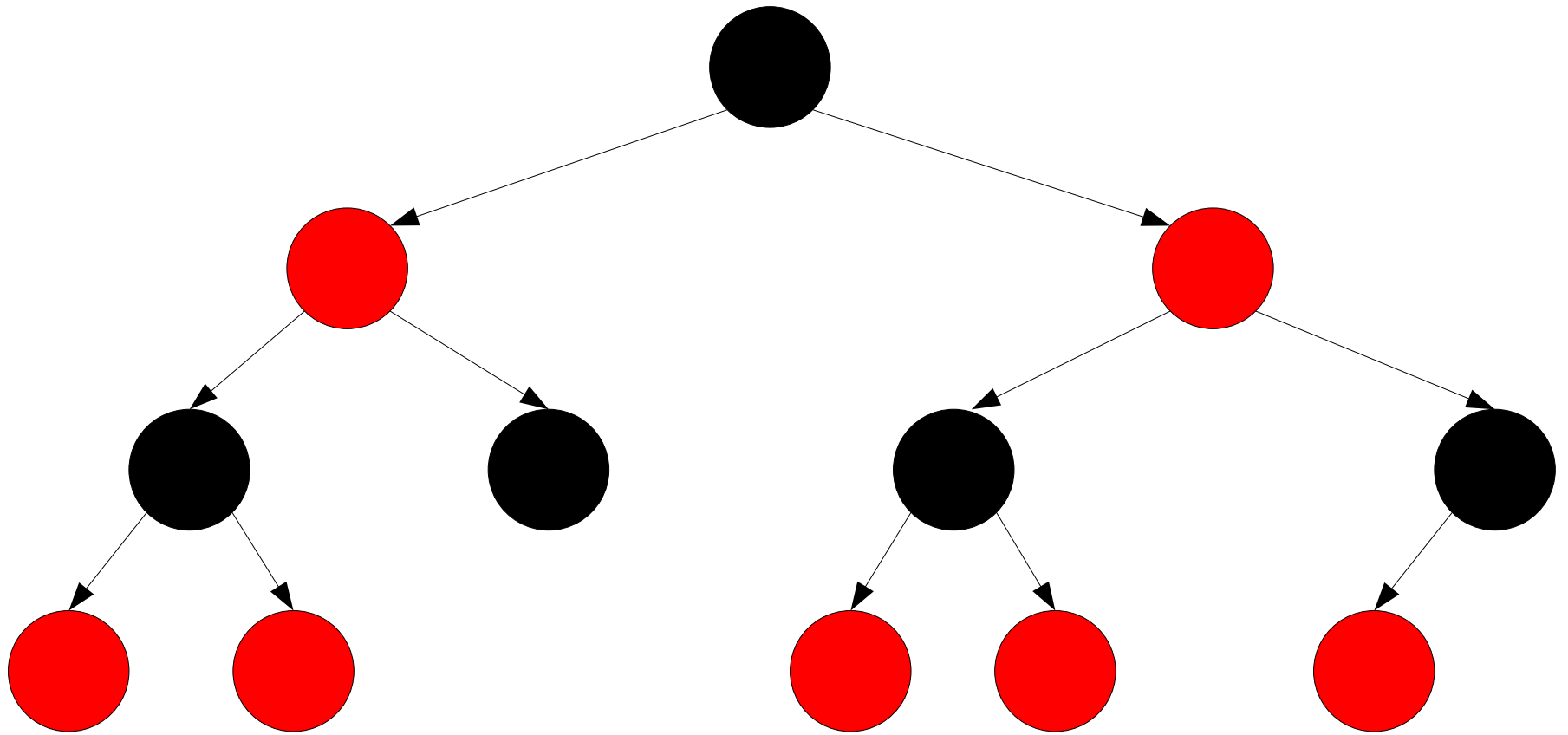
Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



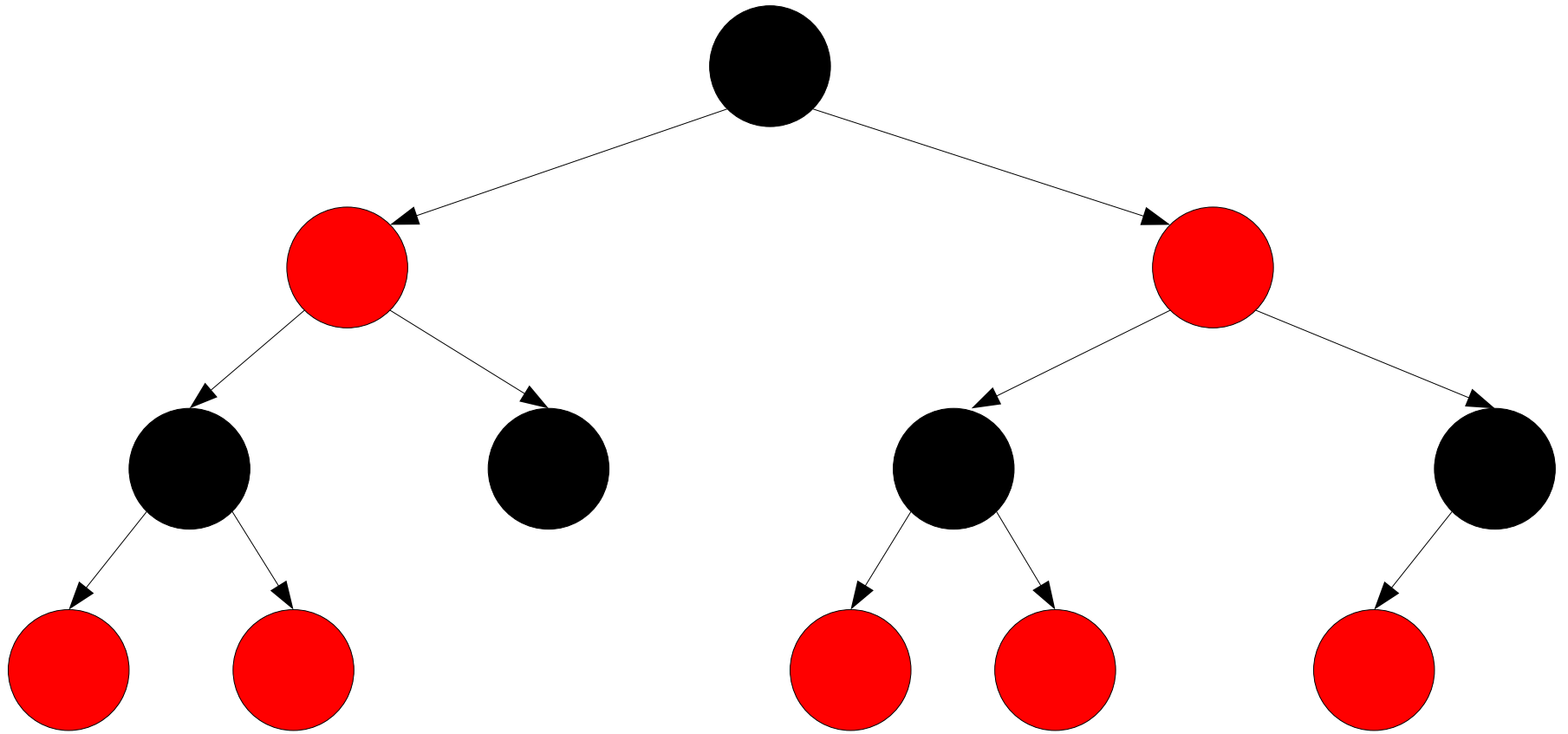
Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



Imagine we cache some value in each node that can be computed just from (1) the node itself and (2) its children's values.



Theorem: Suppose we want to cache some computed value in each node of a red/black tree. Provided that the value can be recomputed purely from the node's value and from its children's values, and provided that each value can be computed in time $O(1)$, then these values can be cached in each node with insertions, lookups, and deletions still taking time $O(\log n)$.

Example: ***Hierarchical Clustering***

1D Hierarchical Clustering

20

42

44

60

66

71

86

92

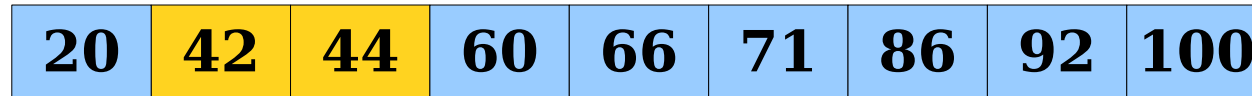
100

1D Hierarchical Clustering

20	42	44	60	66	71	86	92	100
-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	------------

20	42	44	60	66	71	86	92	100
-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	------------

1D Hierarchical Clustering



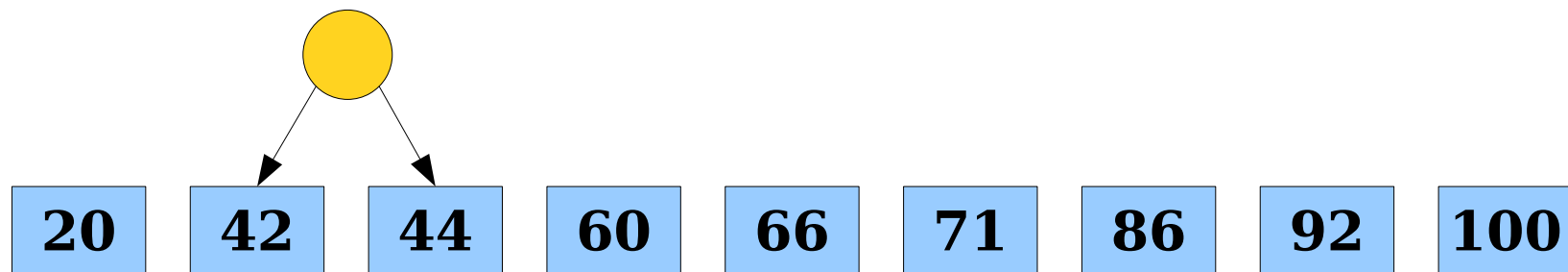
1D Hierarchical Clustering

20	43	60	66	71	86	92	100
	42	44					

20	42	44	60	66	71	86	92	100
----	----	----	----	----	----	----	----	-----

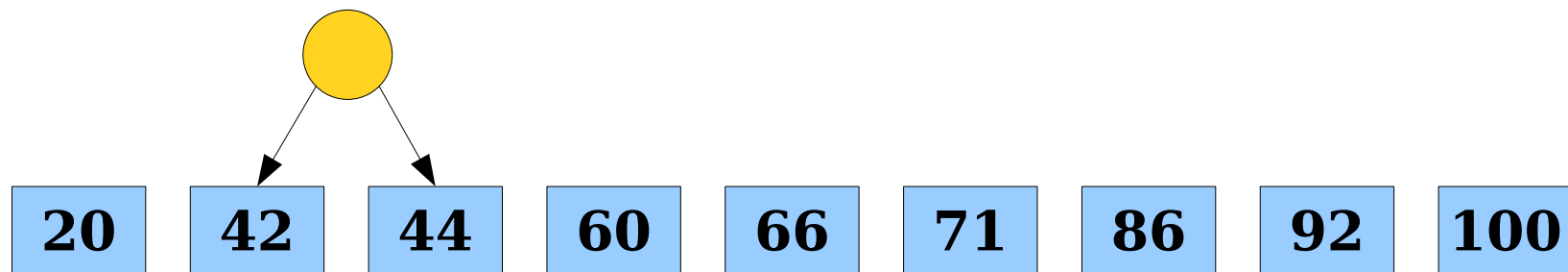
1D Hierarchical Clustering

20	43	60	66	71	86	92	100
	42	44					

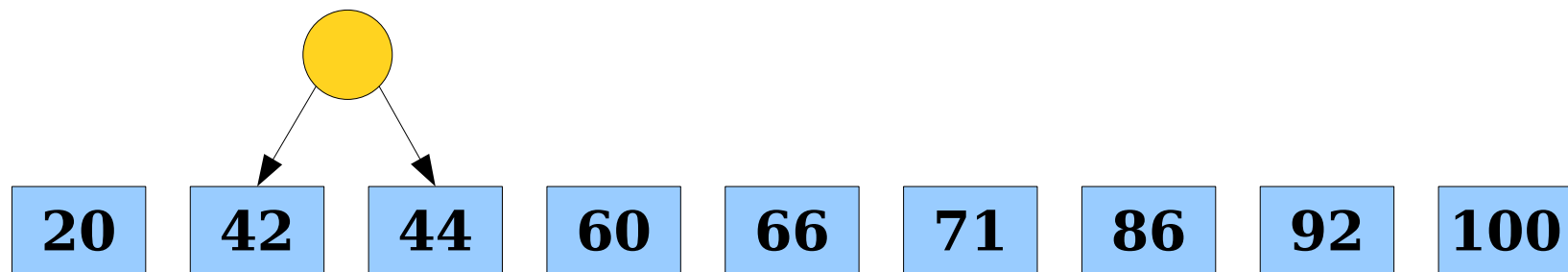
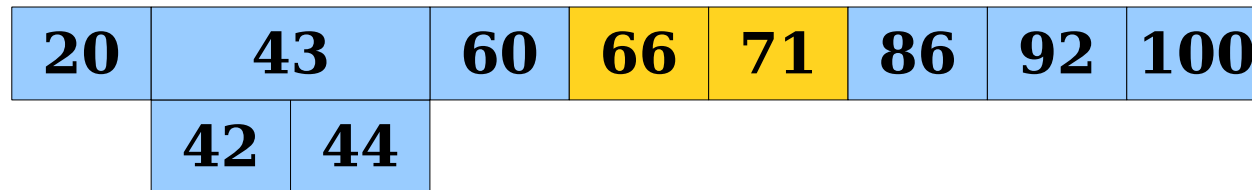


1D Hierarchical Clustering

20	43	60	66	71	86	92	100
	42	44					

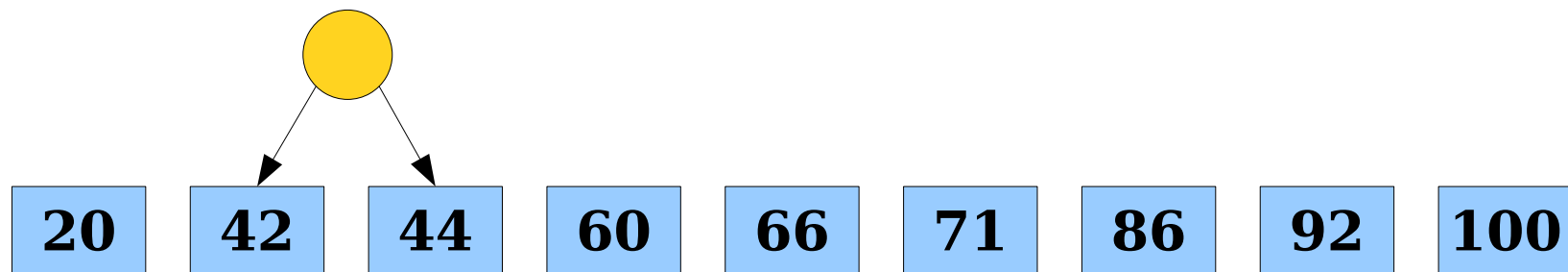


1D Hierarchical Clustering



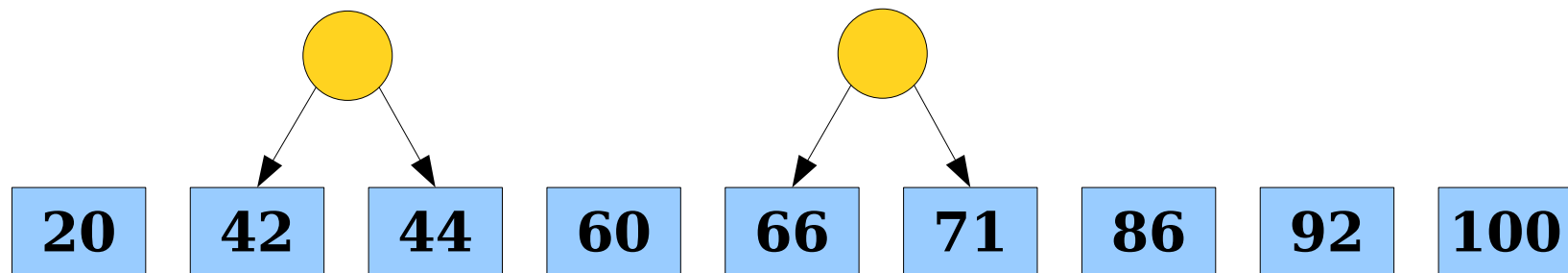
1D Hierarchical Clustering

20	43		60	68.5	86	92	100
	42	44		66	71		



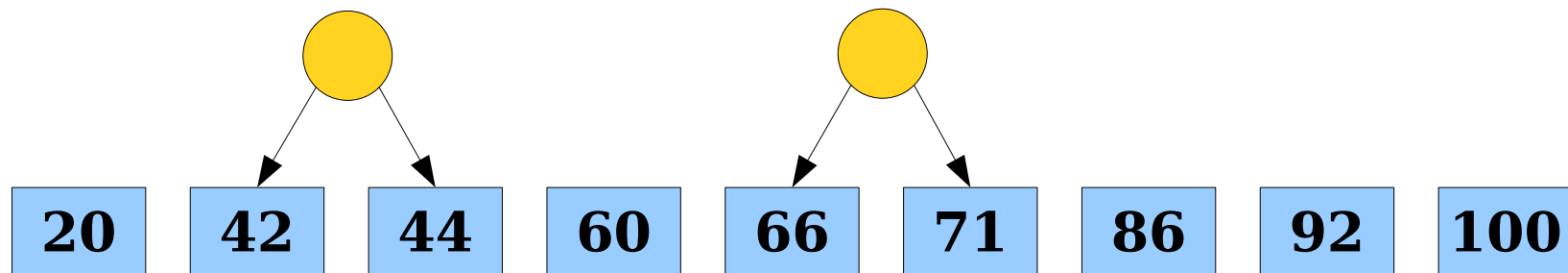
1D Hierarchical Clustering

20	43		60	68.5	86	92	100
	42	44		66	71		



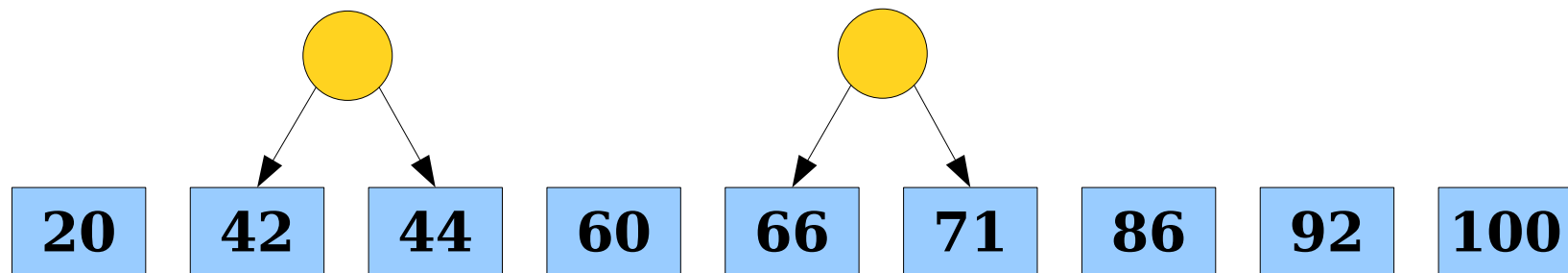
1D Hierarchical Clustering

20	43		60	68.5		86	92	100
	42	44		66	71			



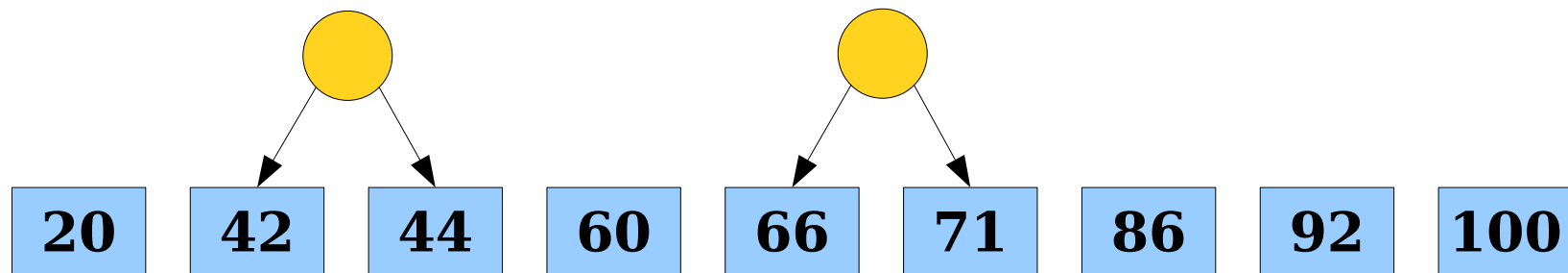
1D Hierarchical Clustering

20	43		60	68.5		86	92	100
	42	44		66	71			



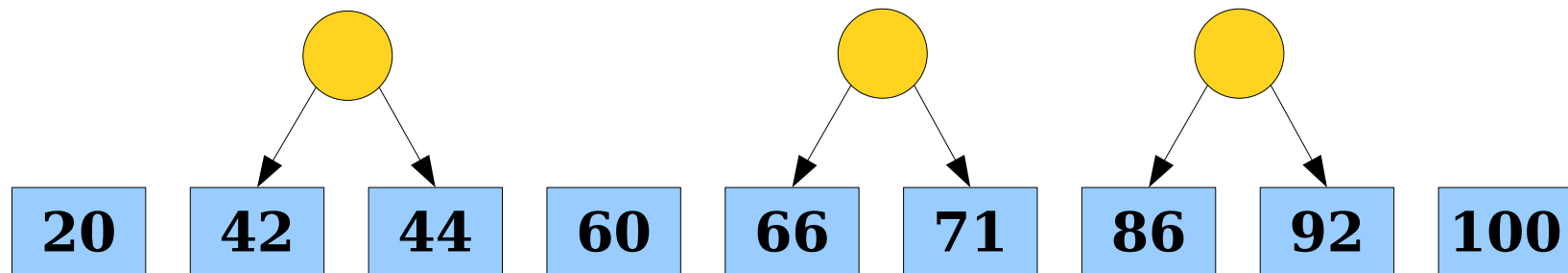
1D Hierarchical Clustering

20	43		60	68.5		89		100
	42	44		66	71	86	92	



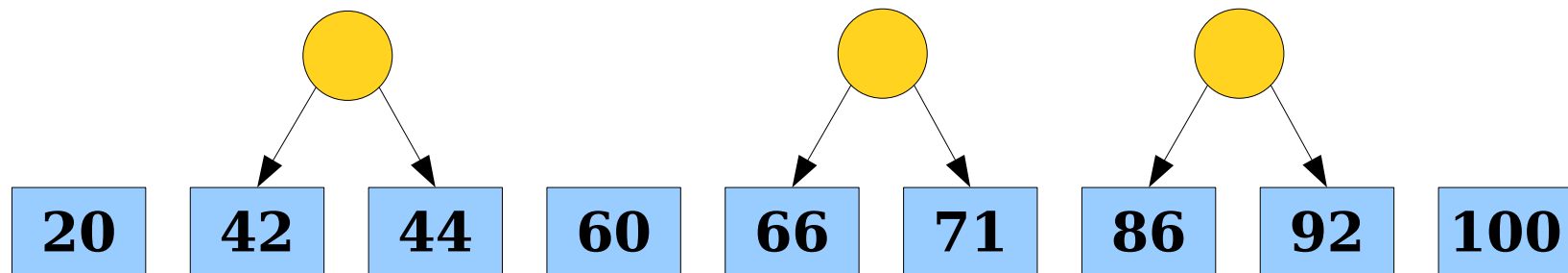
1D Hierarchical Clustering

20	43		60	68.5		89		100
	42	44		66	71	86	92	



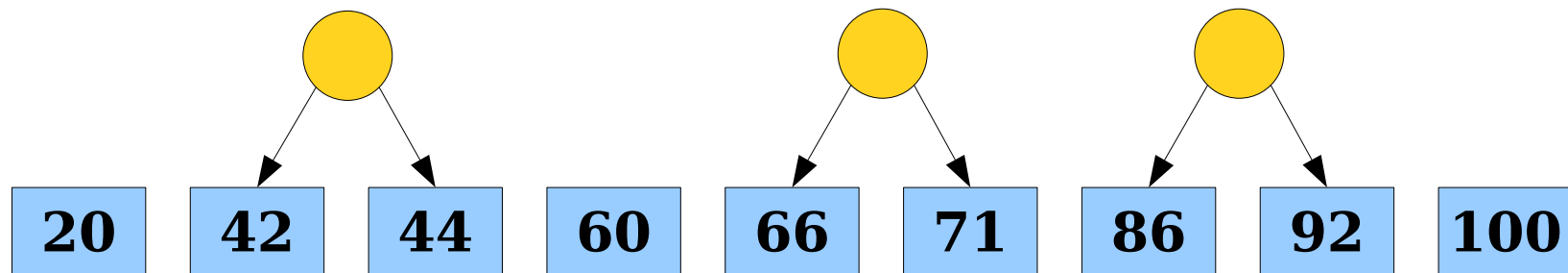
1D Hierarchical Clustering

20	43		60	68.5		89		100
	42	44		66	71	86	92	



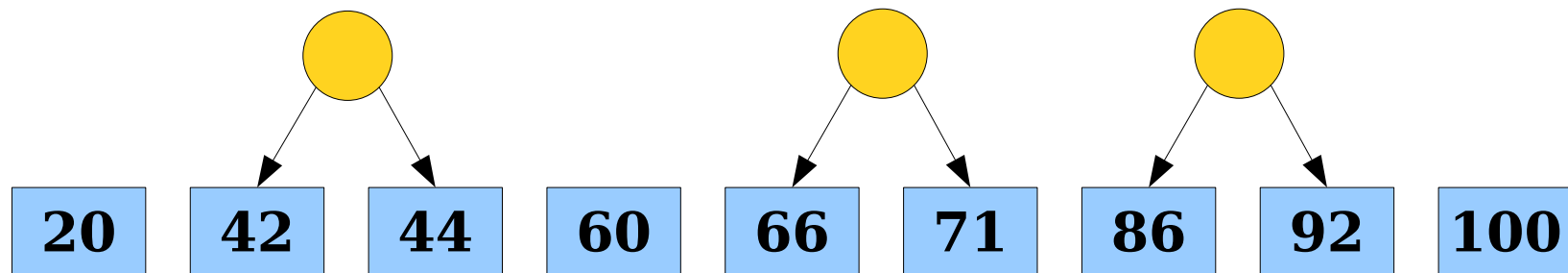
1D Hierarchical Clustering

20	43		60	68.5		89		100
	42	44		66	71	86	92	



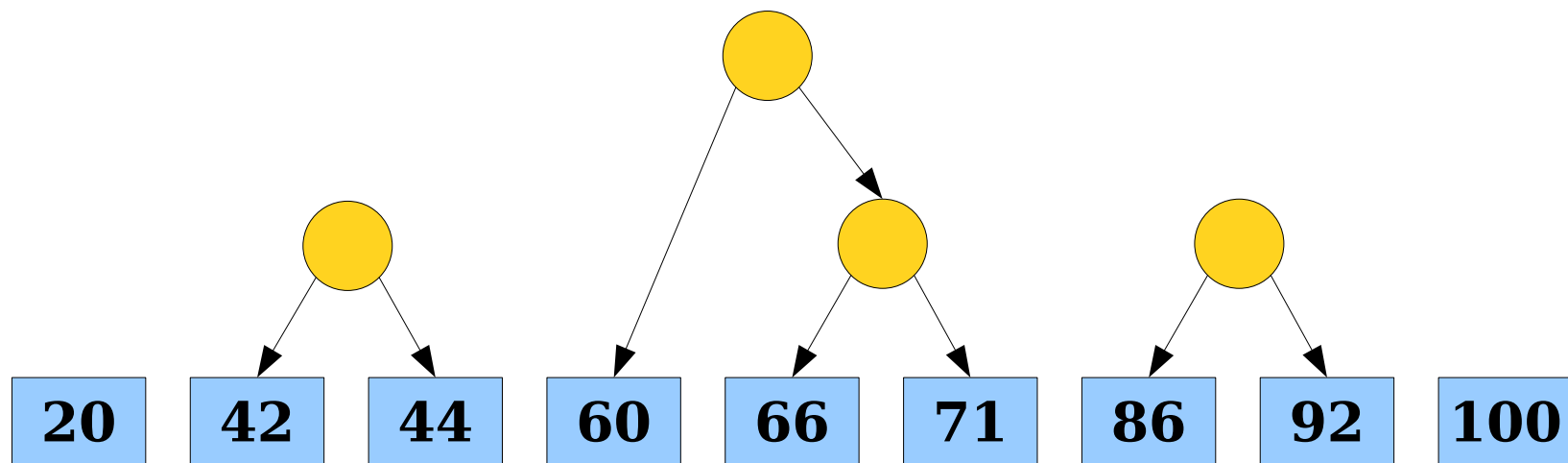
1D Hierarchical Clustering

20	43		65.67			89		100
	42	44	60	66	71	86	92	



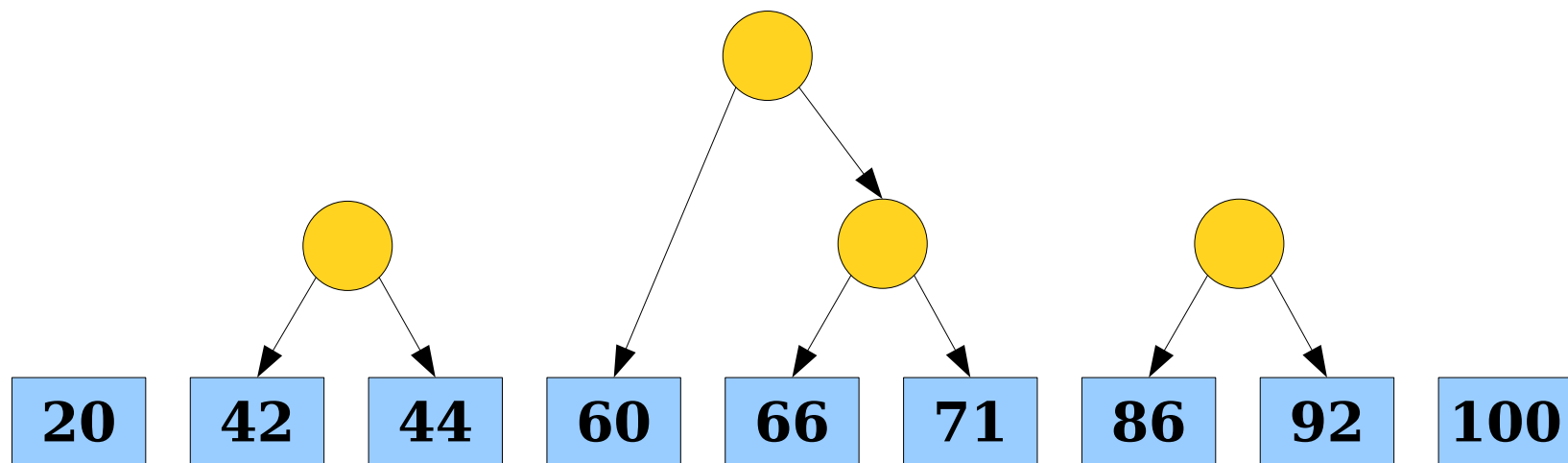
1D Hierarchical Clustering

20	43		65.67			89		100
	42	44	60	66	71	86	92	



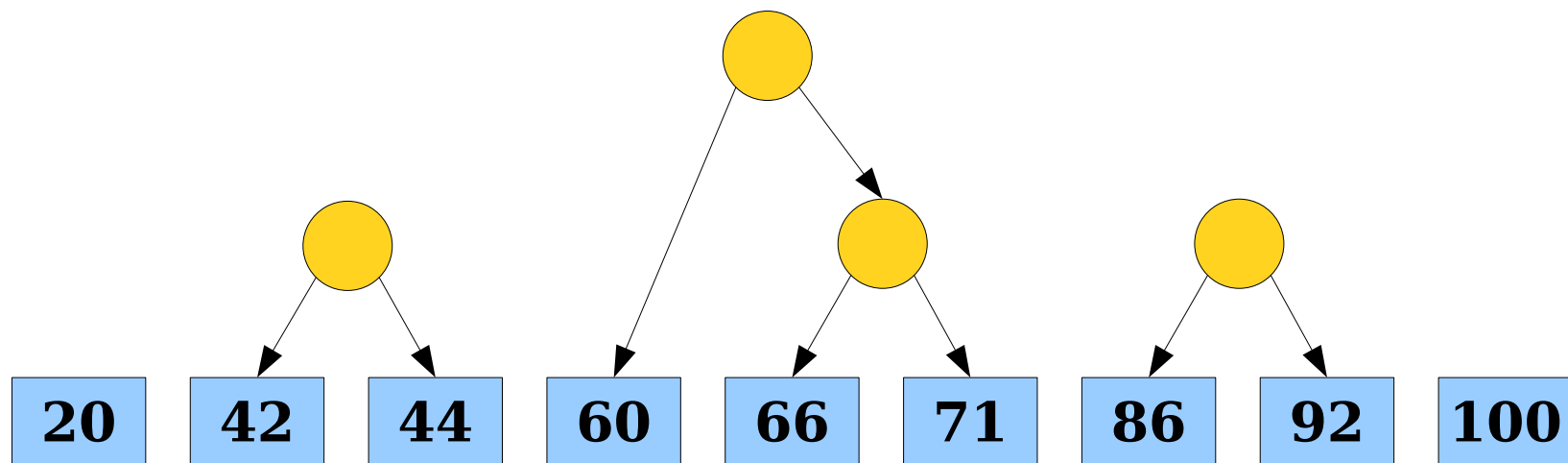
1D Hierarchical Clustering

20	43		65.67			89		100
	42	44	60	66	71	86	92	



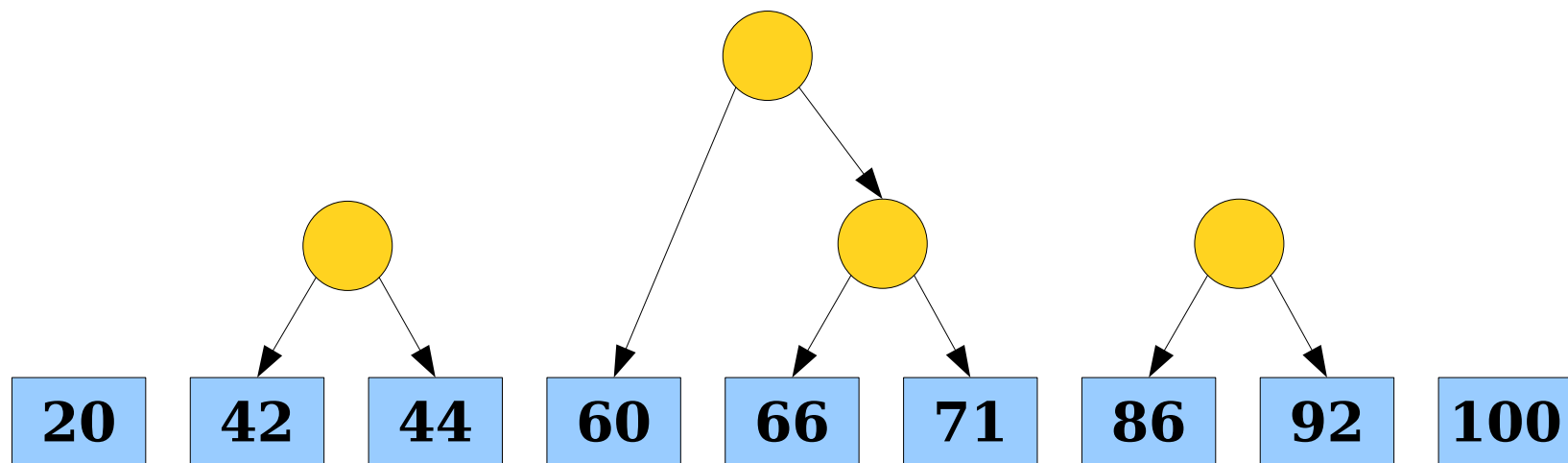
1D Hierarchical Clustering

20	43		65.67			89		100
	42	44	60	66	71	86	92	



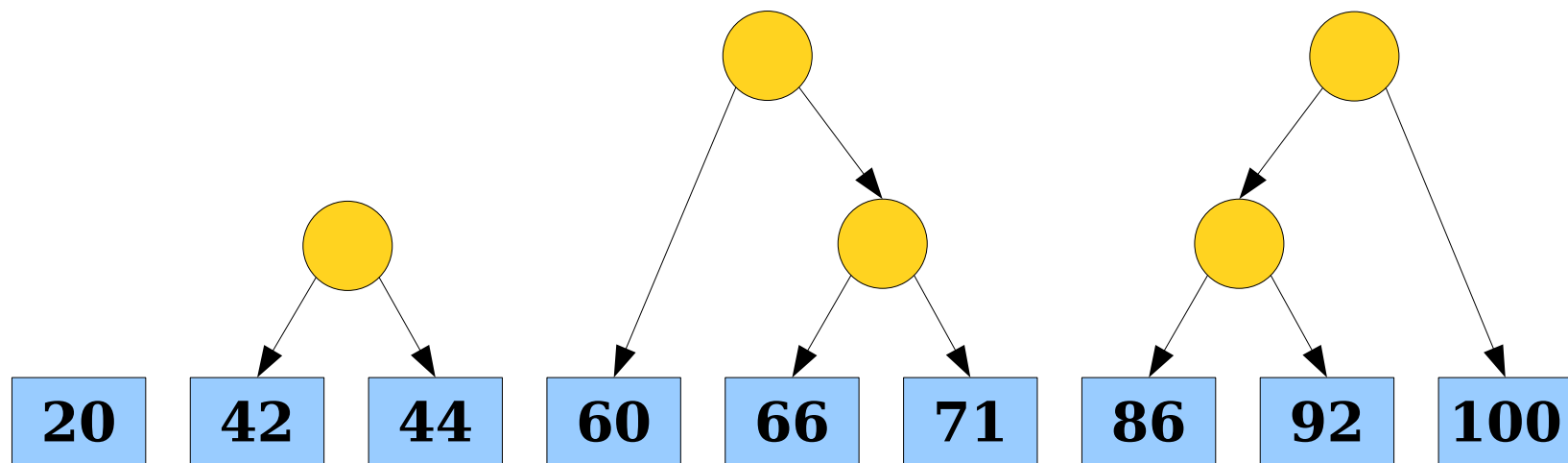
1D Hierarchical Clustering

20	43		65.67			92.67		
	42	44	60	66	71	86	92	100



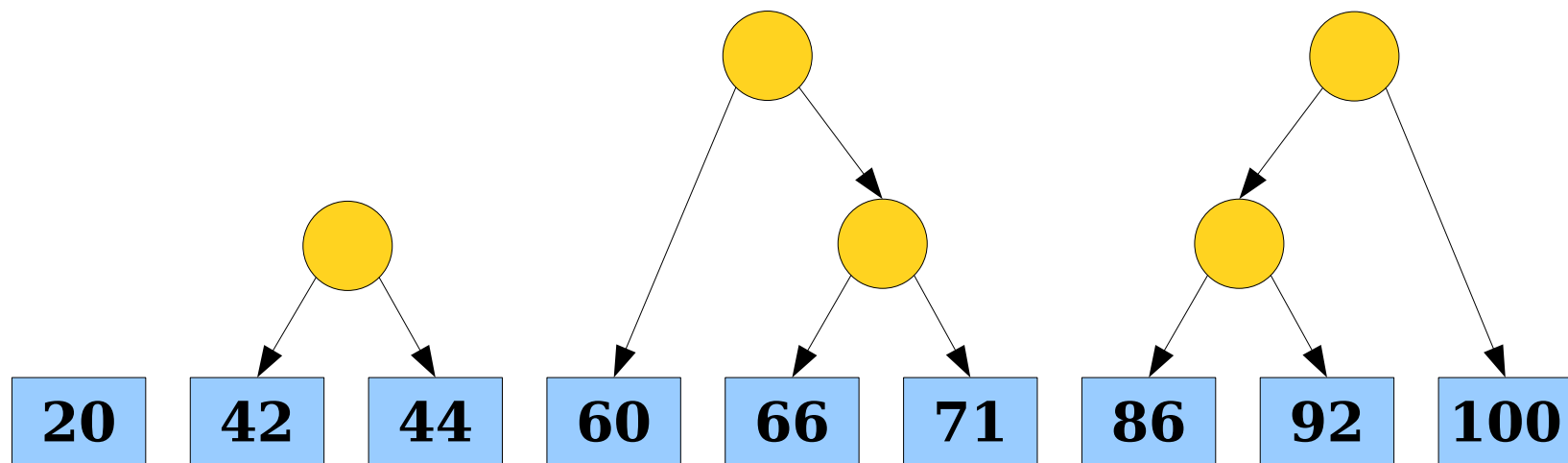
1D Hierarchical Clustering

20	43		65.67			92.67		
	42	44	60	66	71	86	92	100



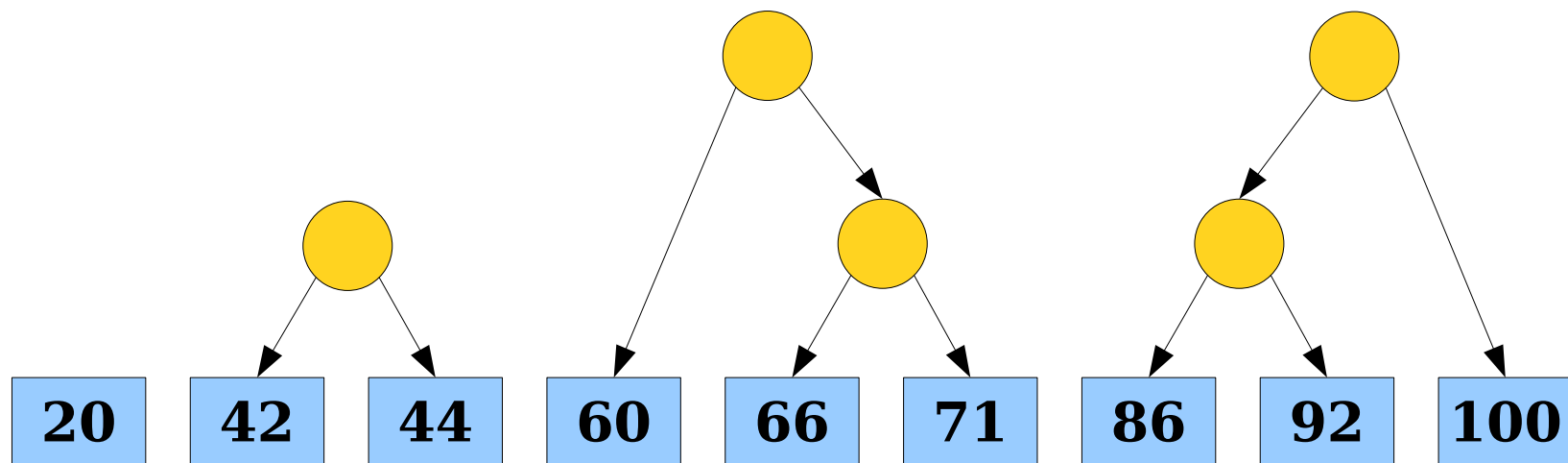
1D Hierarchical Clustering

20	43		65.67			92.67		
	42	44	60	66	71	86	92	100



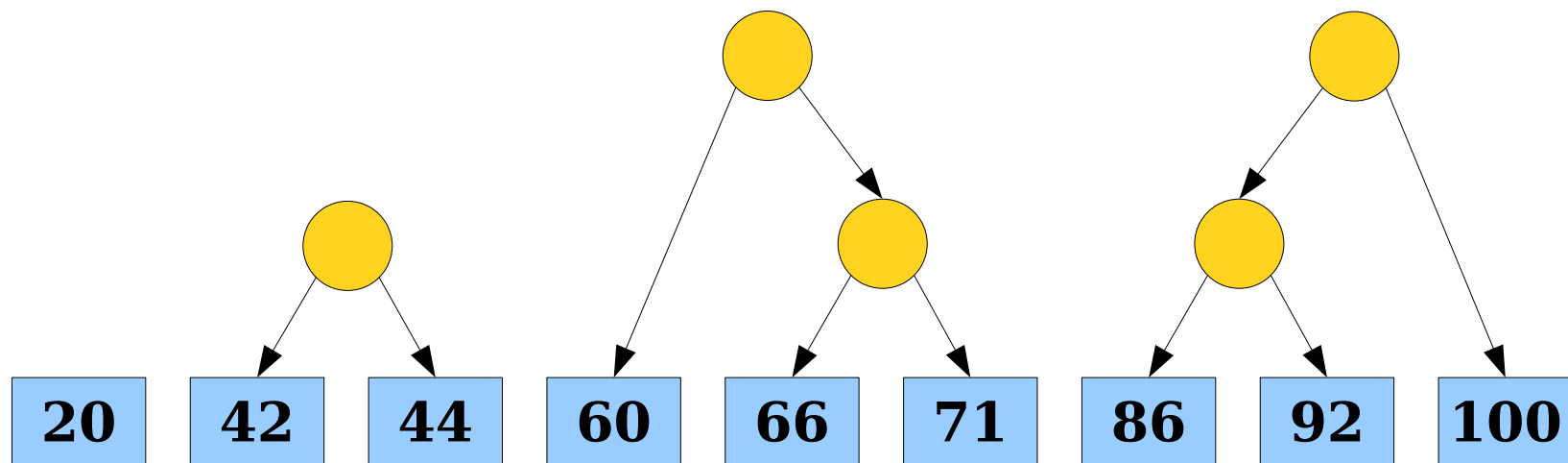
1D Hierarchical Clustering

20	43		65.67			92.67		
	42	44	60	66	71	86	92	100



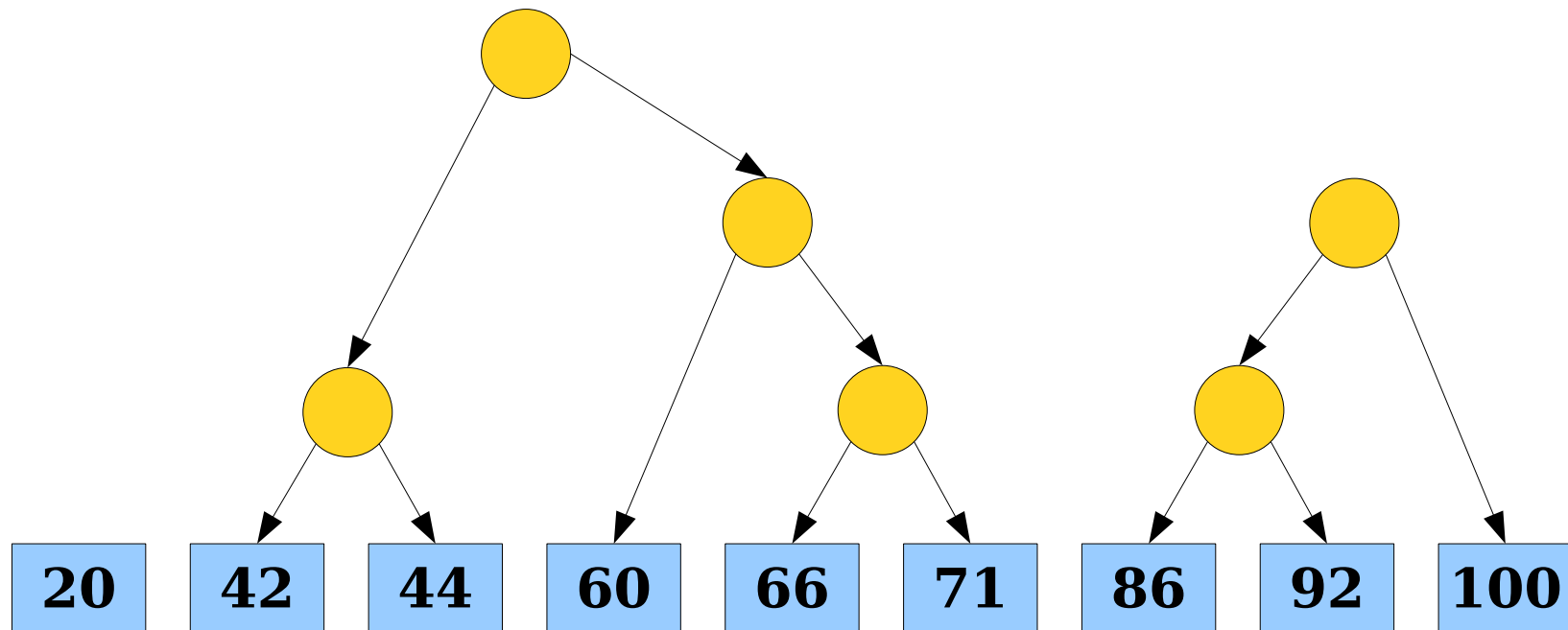
1D Hierarchical Clustering

20	56.6					92.67		
	42	44	60	66	71	86	92	100



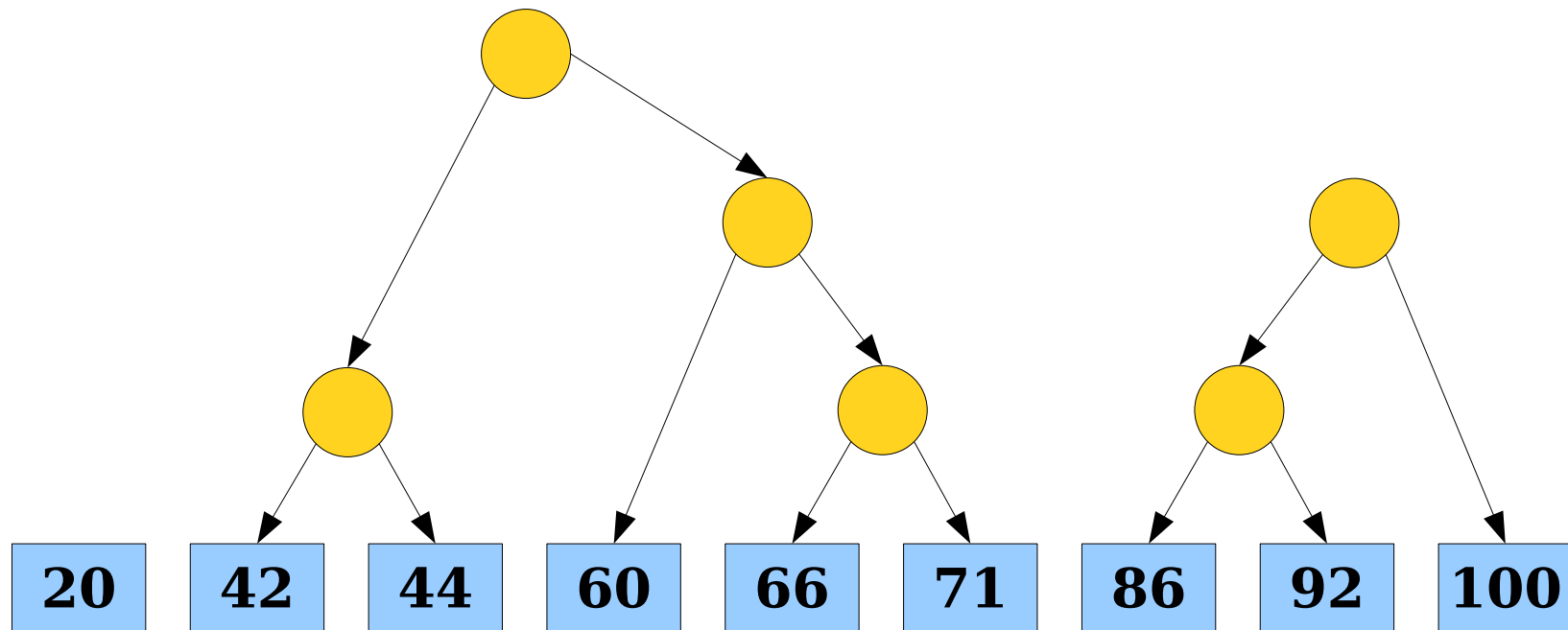
1D Hierarchical Clustering

20	56.6					92.67		
	42	44	60	66	71	86	92	100



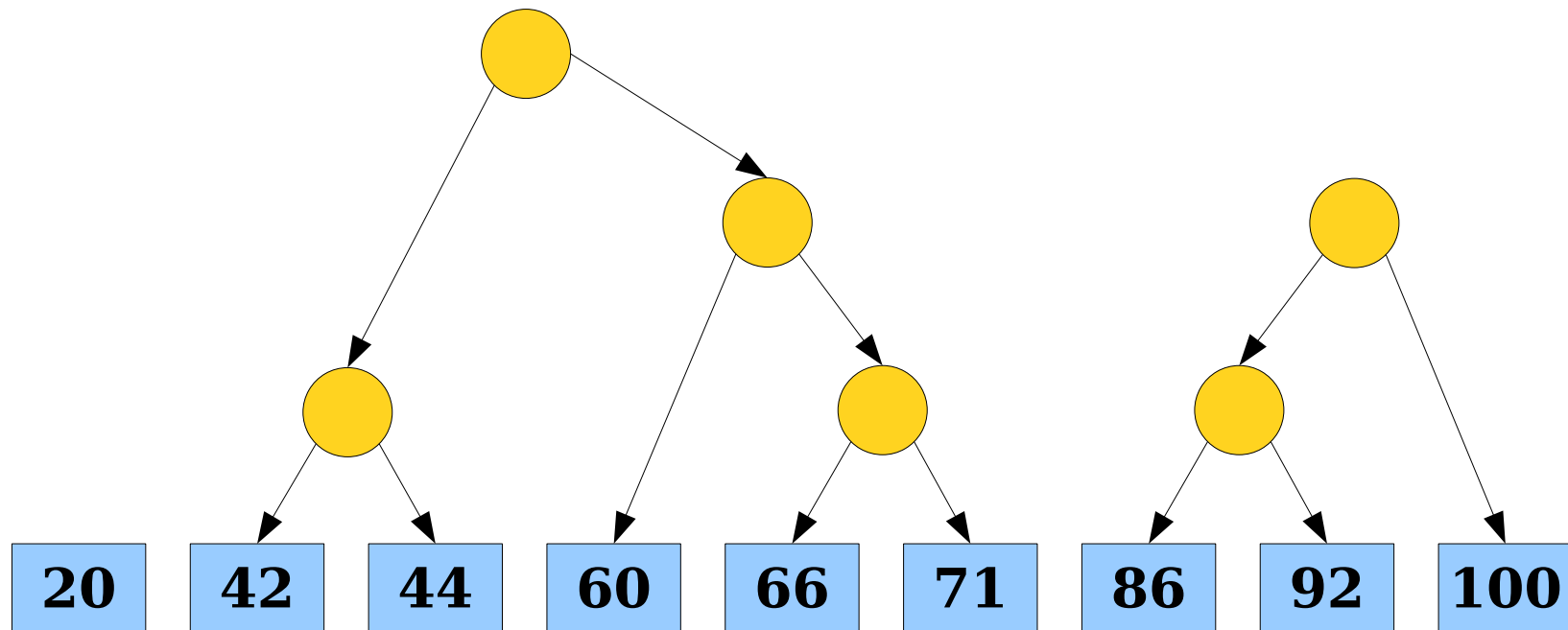
1D Hierarchical Clustering

20	56.6					92.67		
	42	44	60	66	71	86	92	100



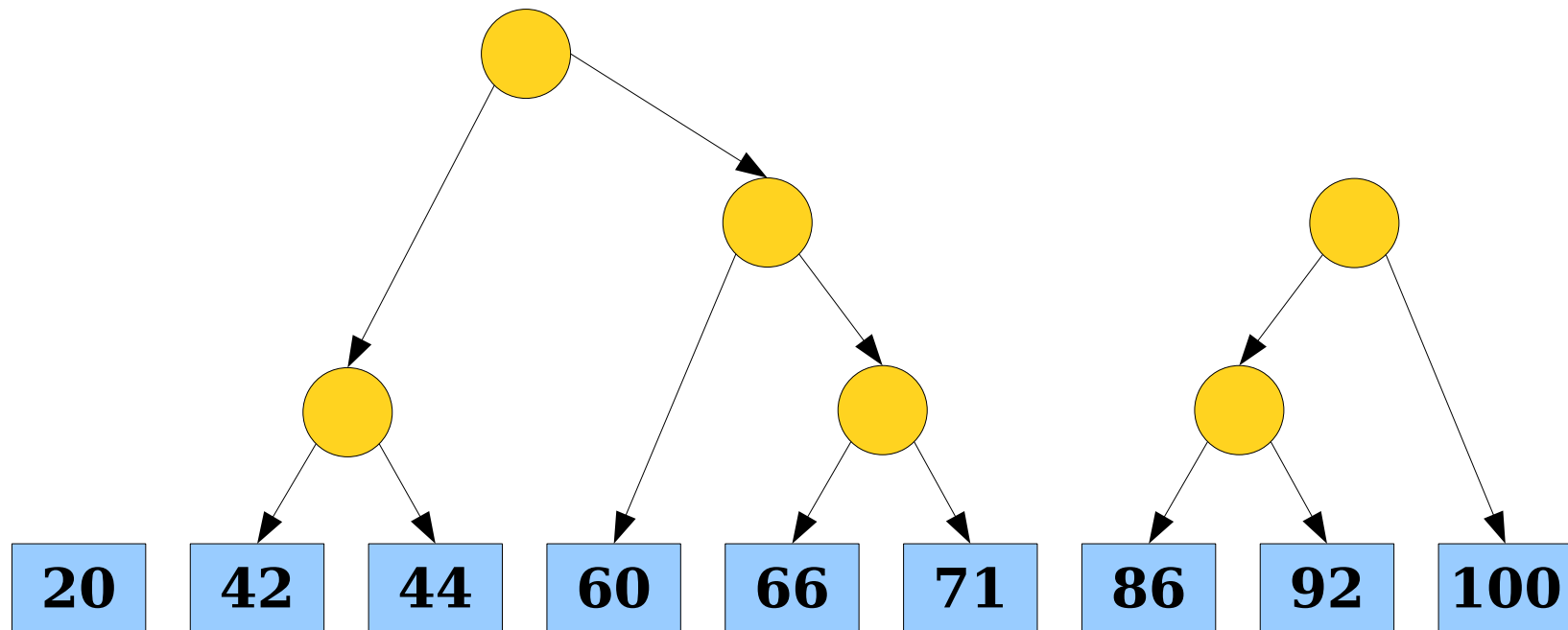
1D Hierarchical Clustering

20	56.6					92.67		
	42	44	60	66	71	86	92	100



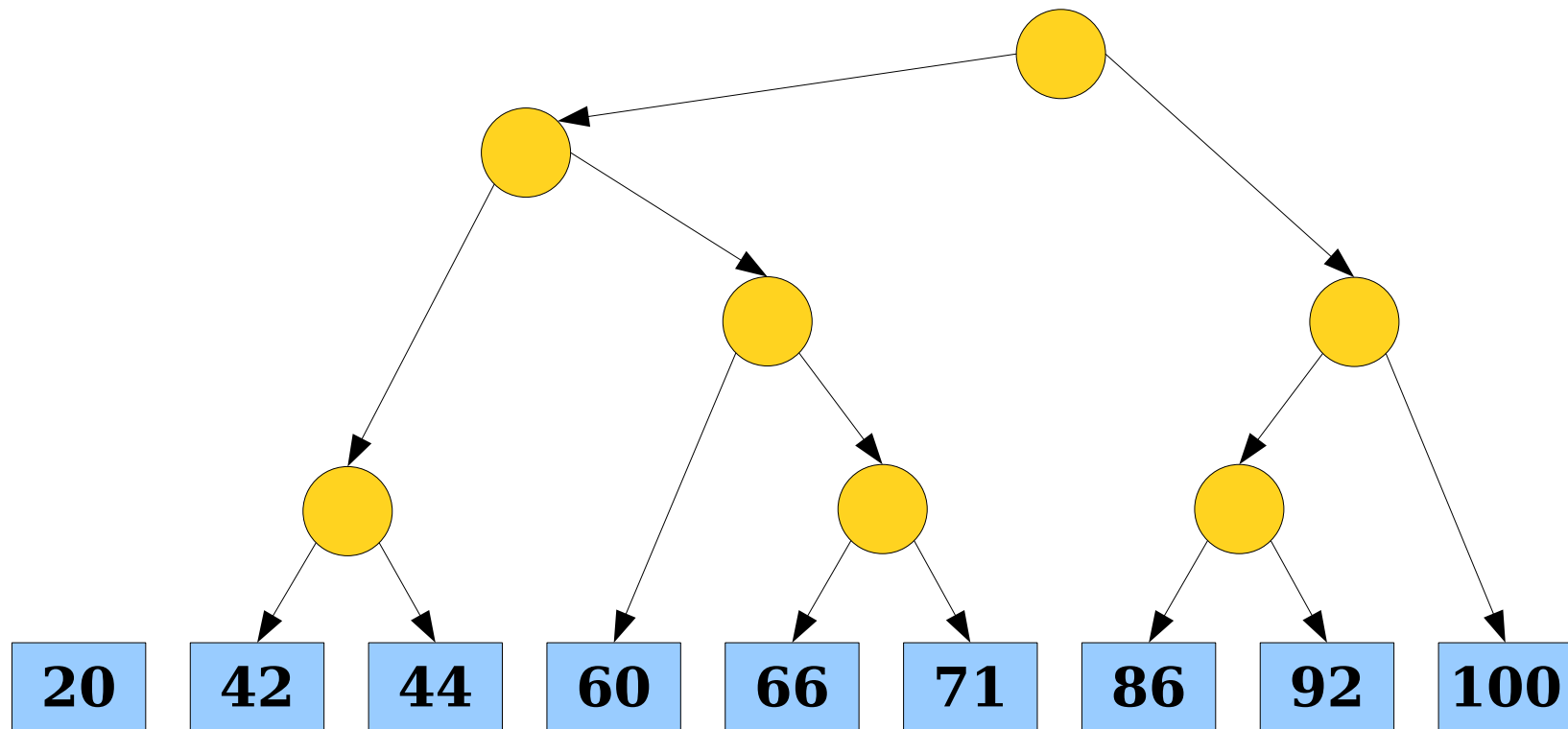
1D Hierarchical Clustering

20	70.13							
	42	44	60	66	71	86	92	100



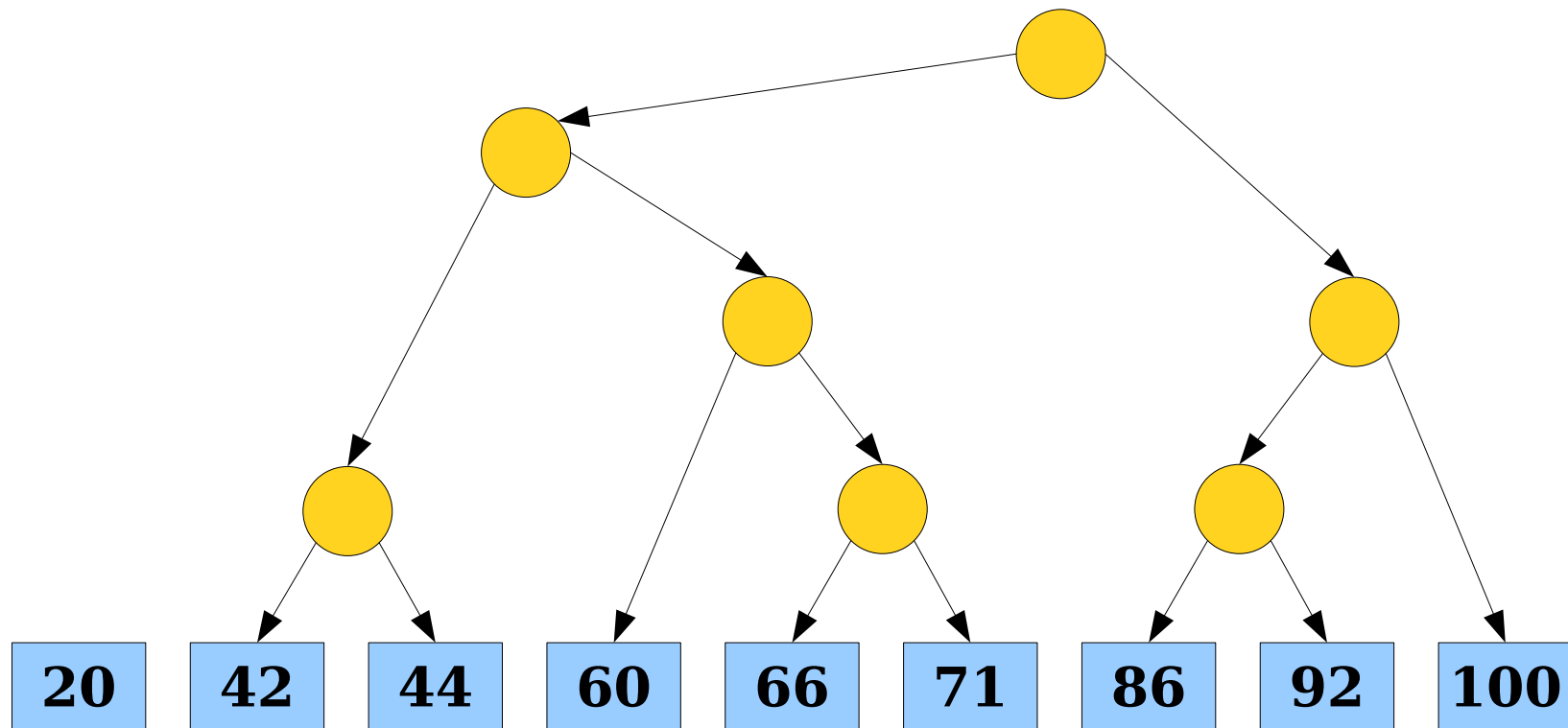
1D Hierarchical Clustering

20	70.13							
	42	44	60	66	71	86	92	100



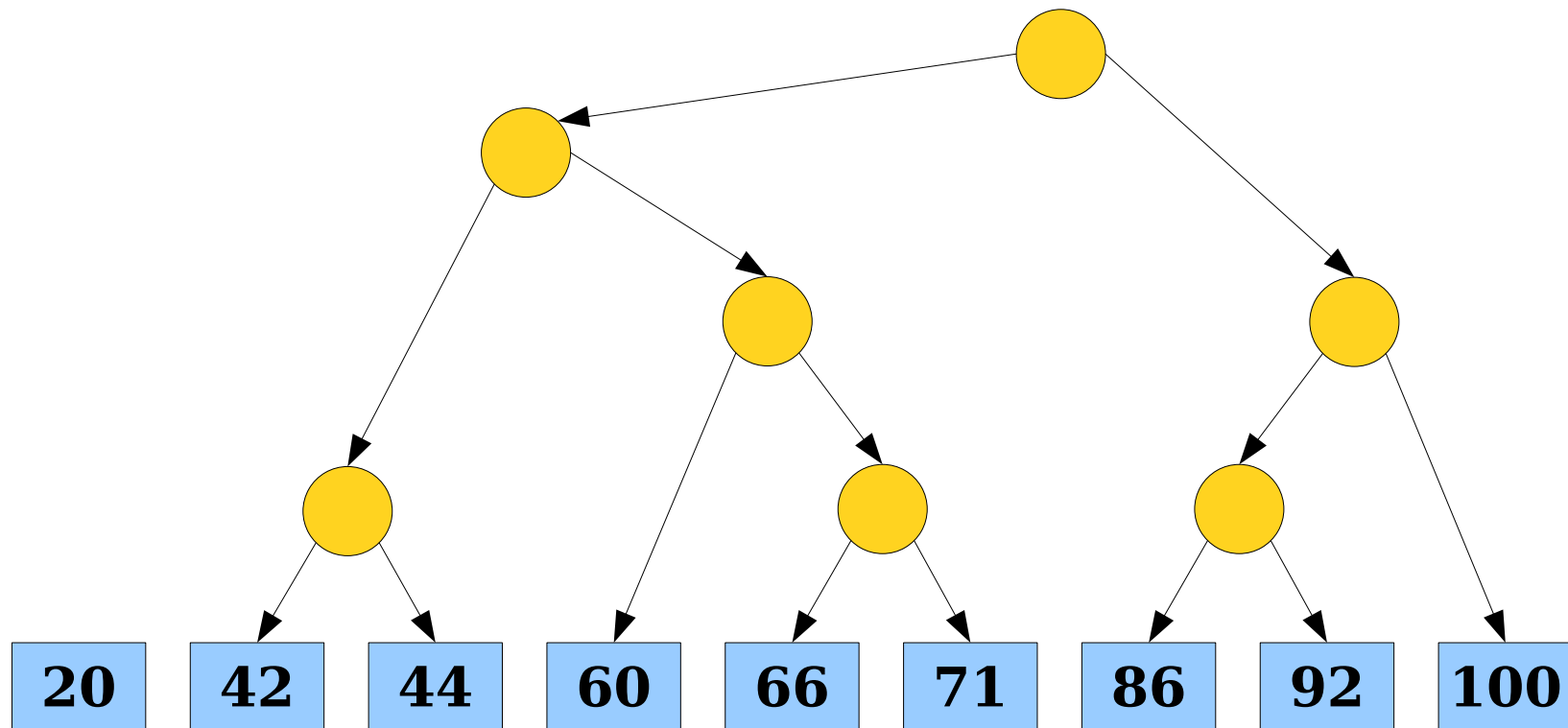
1D Hierarchical Clustering

20	70.13							
	42	44	60	66	71	86	92	100



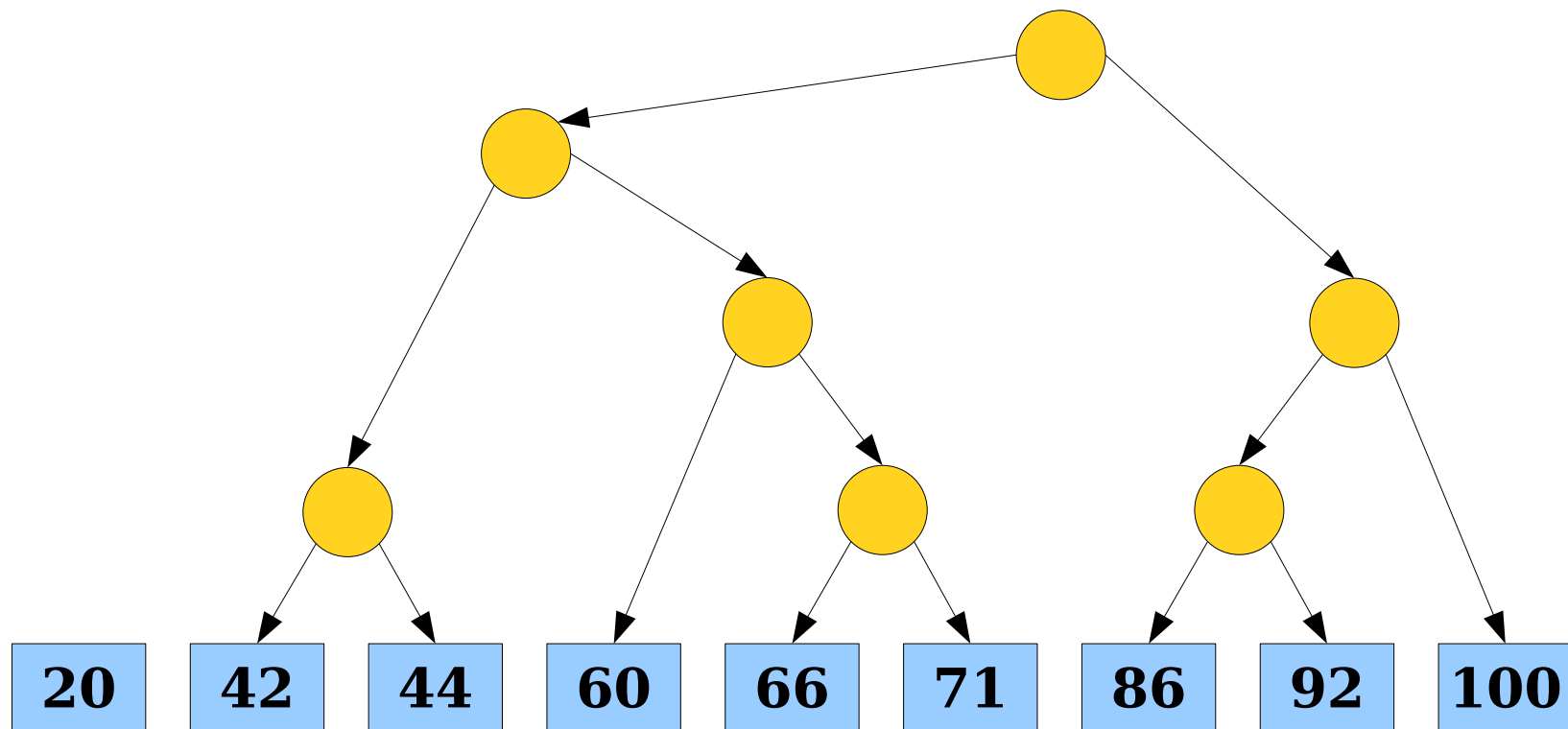
1D Hierarchical Clustering

20	70.13							
	42	44	60	66	71	86	92	100

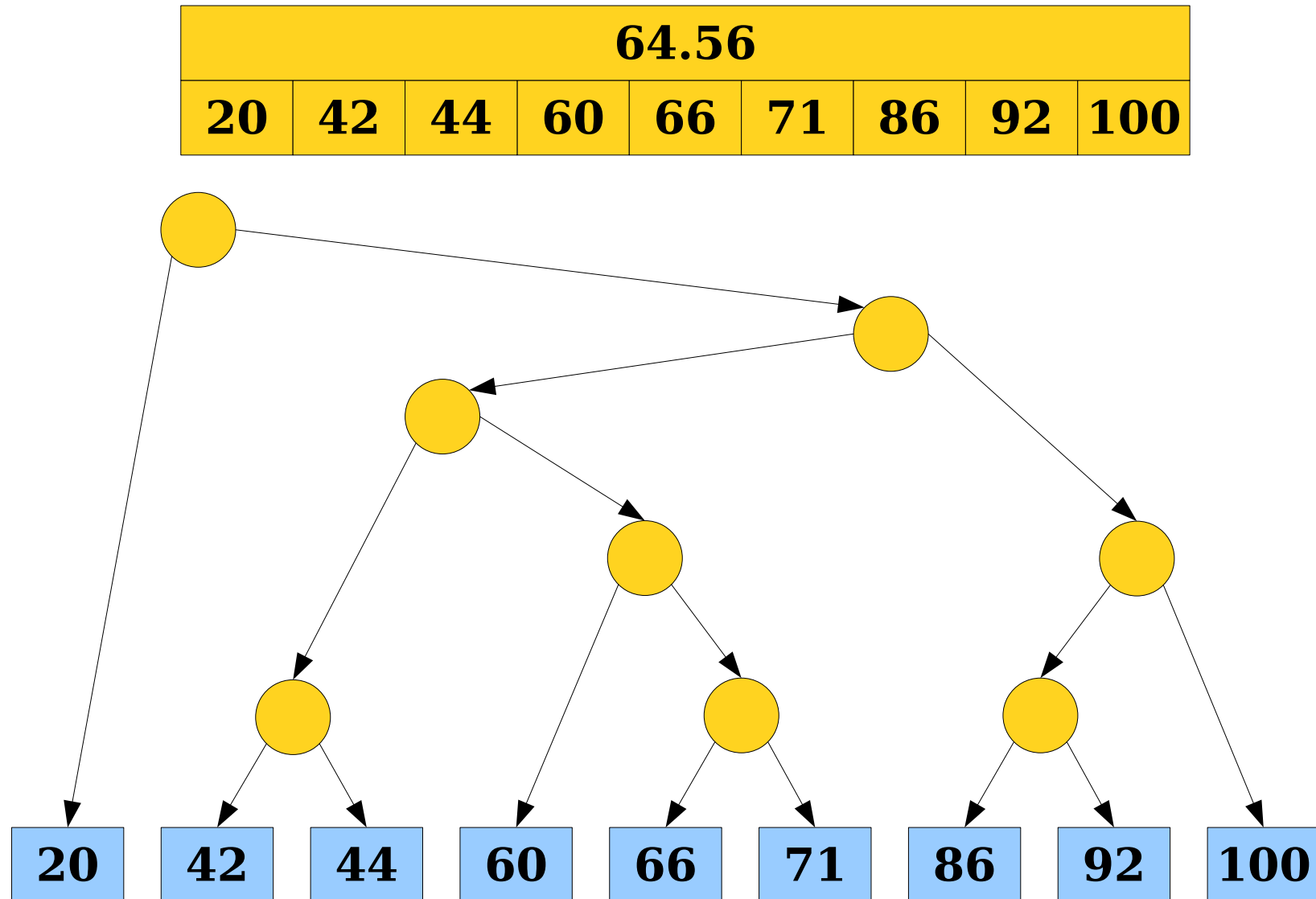


1D Hierarchical Clustering

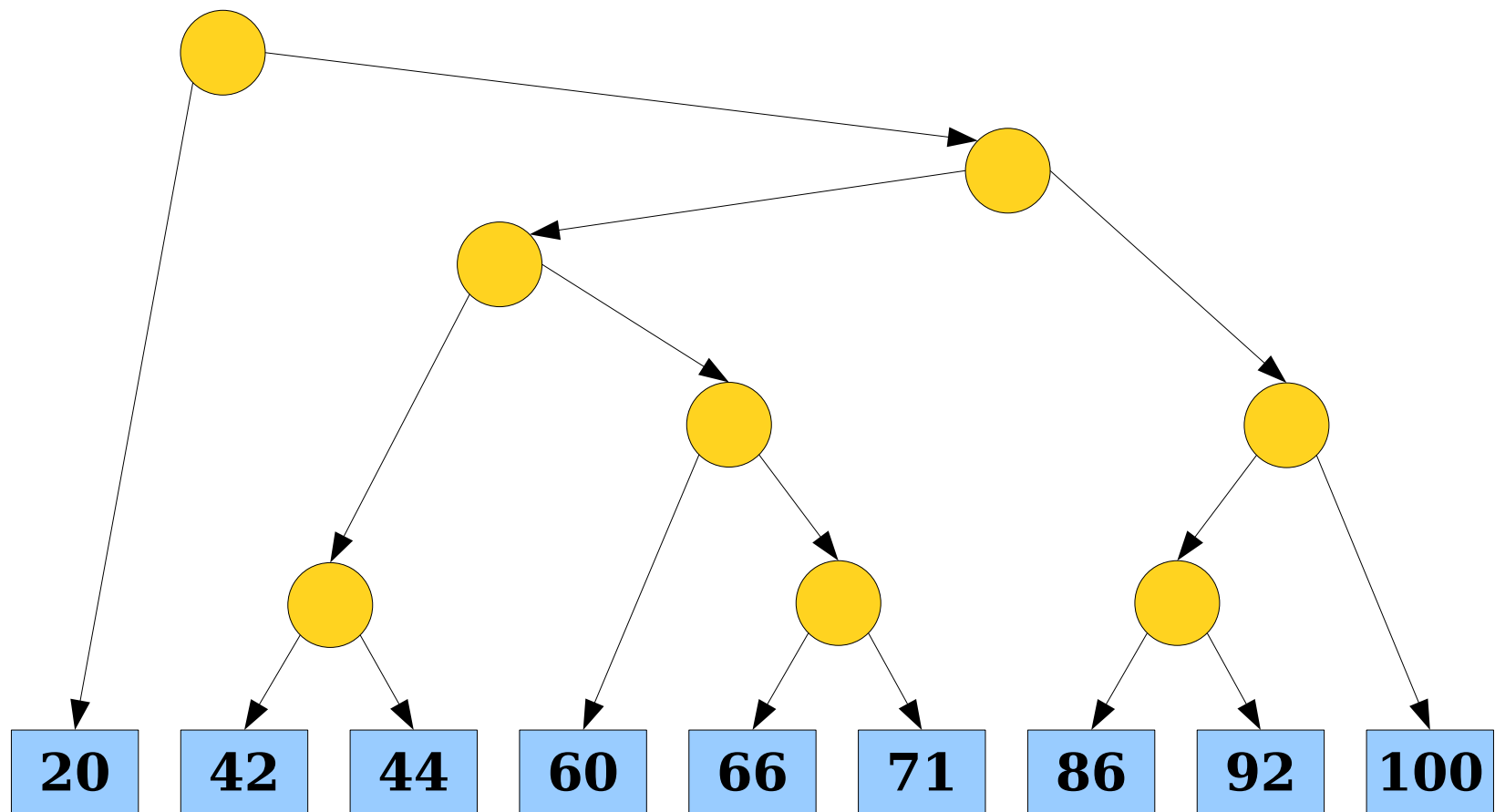
64.56								
20	42	44	60	66	71	86	92	100



1D Hierarchical Clustering

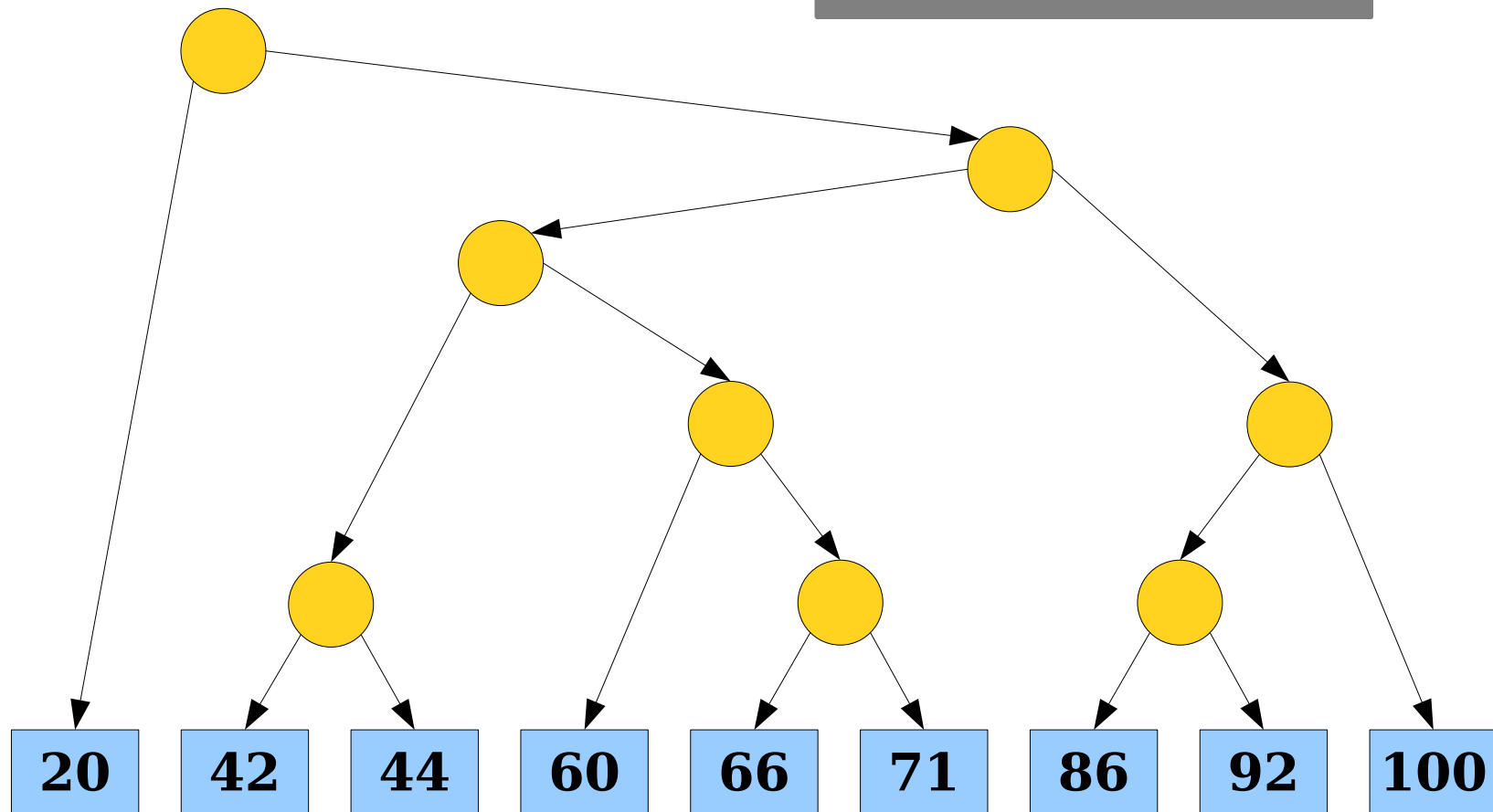


1D Hierarchical Clustering



1D Hierarchical Clustering

This tree is called
a ***dendrogram***.



Analyzing the Runtime

- How efficient is this algorithm?
 - Number of rounds: $\Theta(n)$.
 - Work to find closest pair: $O(n)$.
 - Total runtime: $\Theta(n^2)$.
- Can we do better?

Analyzing the Runtime

How efficient is this algorithm?

Number of rounds: $\Theta(n)$.

- Work to find closest pair: $O(n)$.

Total runtime: $\Theta(n^2)$.

Can we do better?

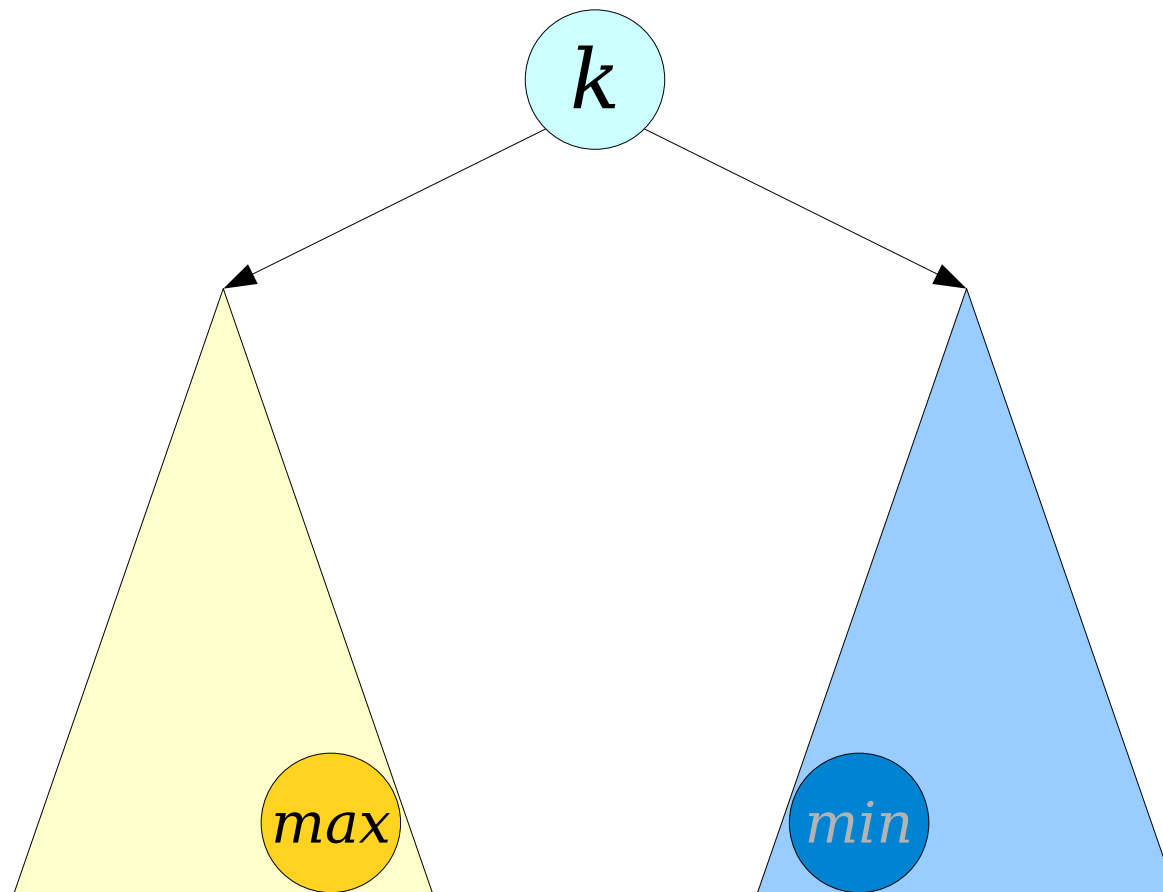
Dynamic 1D Closest Points

- The *dynamic 1D closest points problem* is the following:

Maintain a set of real numbers undergoing insertion and deletion while efficiently supporting queries of the form “what is the closest pair of points?”

- Can we build a better data structure for this?

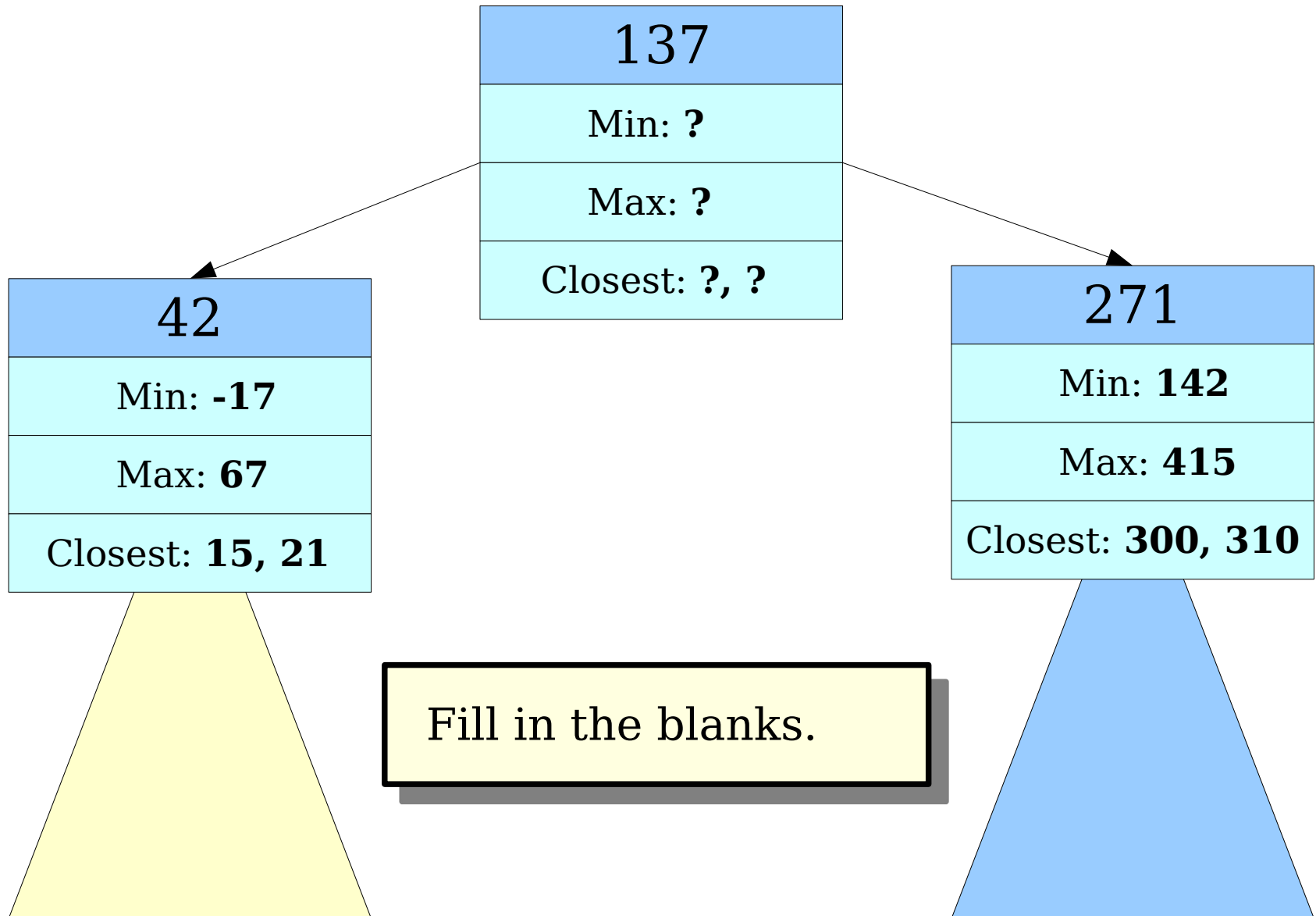
Dynamic 1D Closest Points



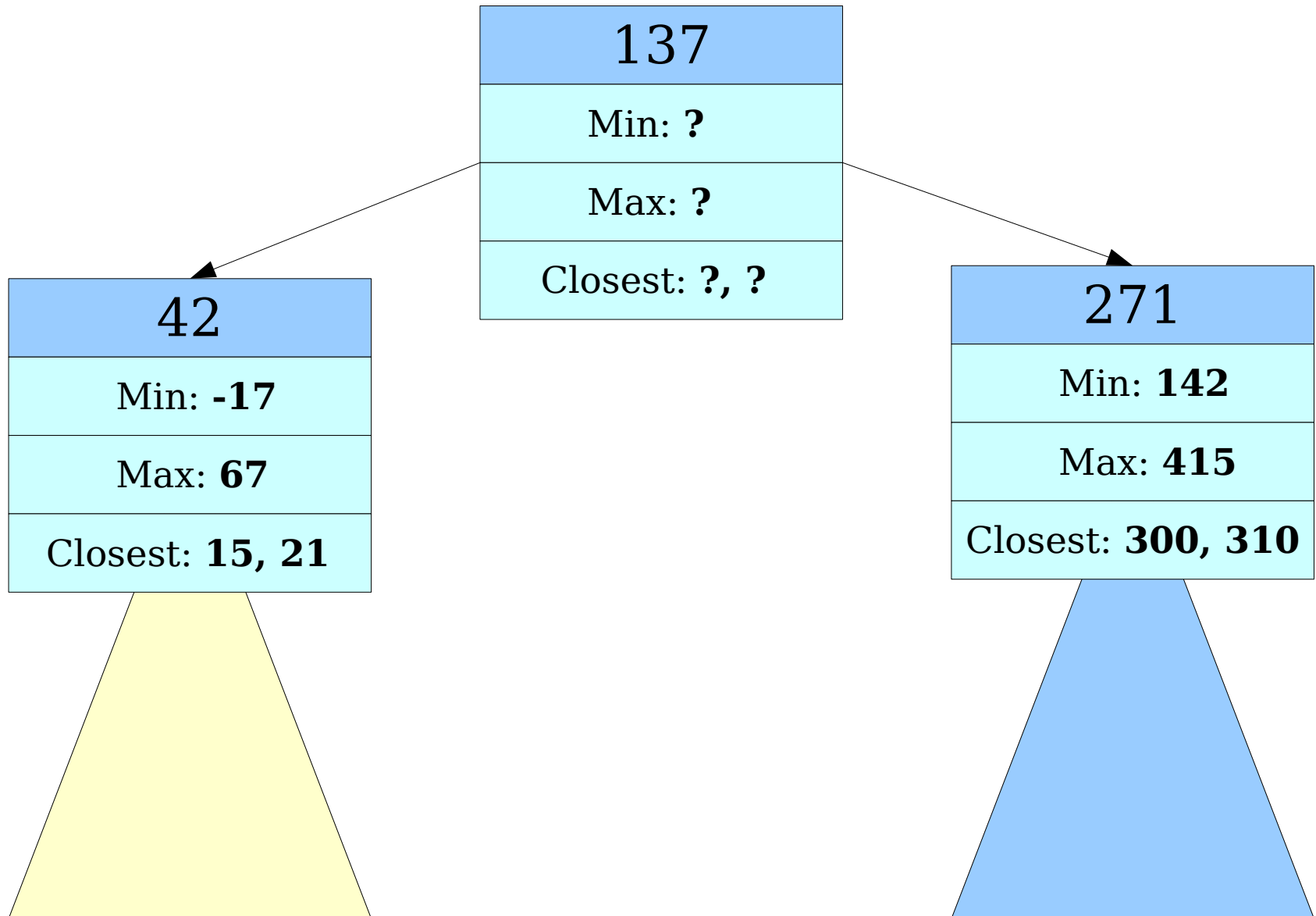
A Tree Augmentation

- Augment each node to store the following:
 - The maximum value in the tree.
 - The minimum value in the tree.
 - The closest pair of points in the tree.
- **Claim:** Each of these properties can be computed in time $O(1)$ from the left and right subtrees.
- These properties can be augmented into a red/black tree so that insertions and deletions take time $O(\log n)$ and “what is the closest pair of points?” can be answered in time $O(1)$.

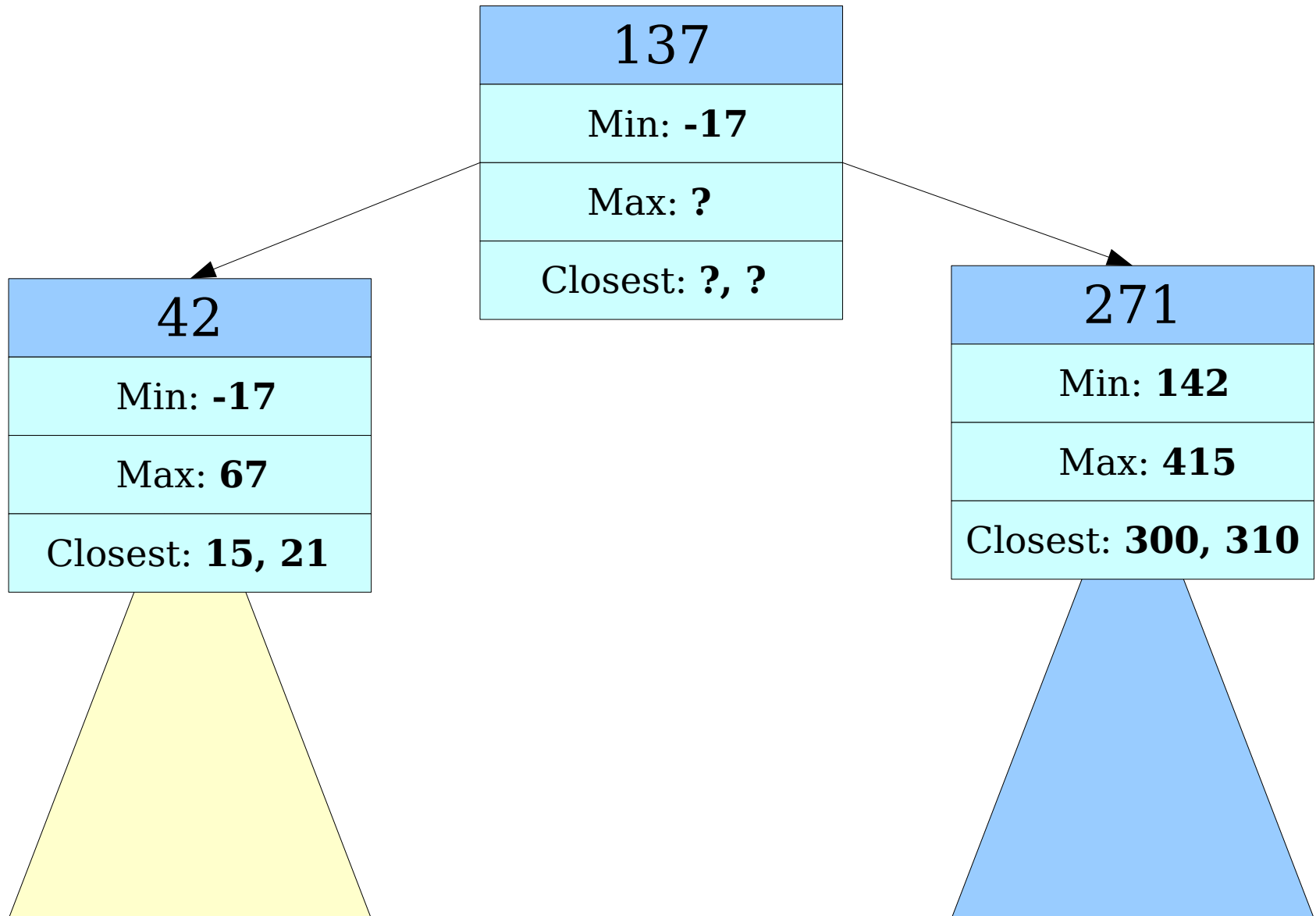
Dynamic 1D Closest Points



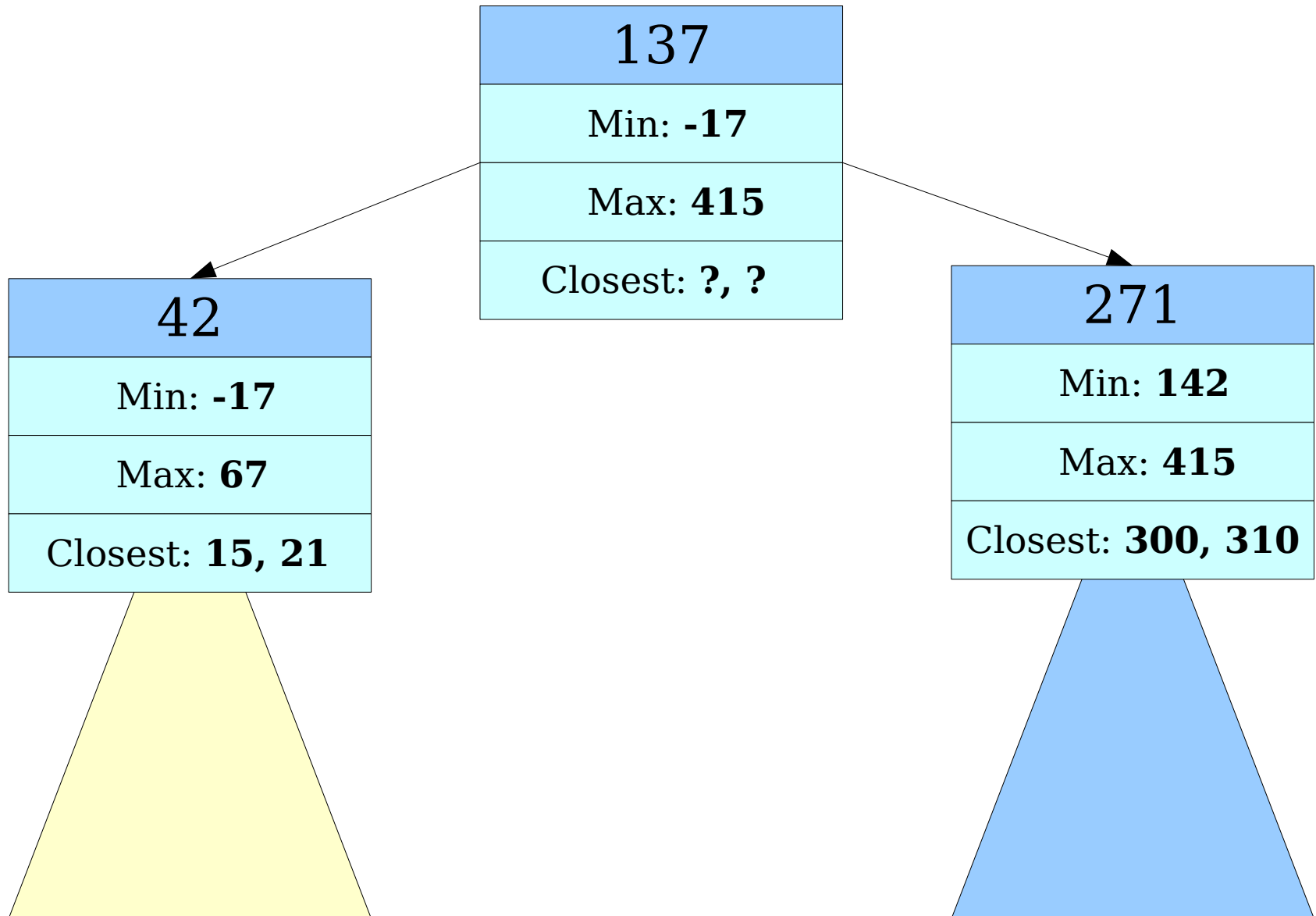
Dynamic 1D Closest Points



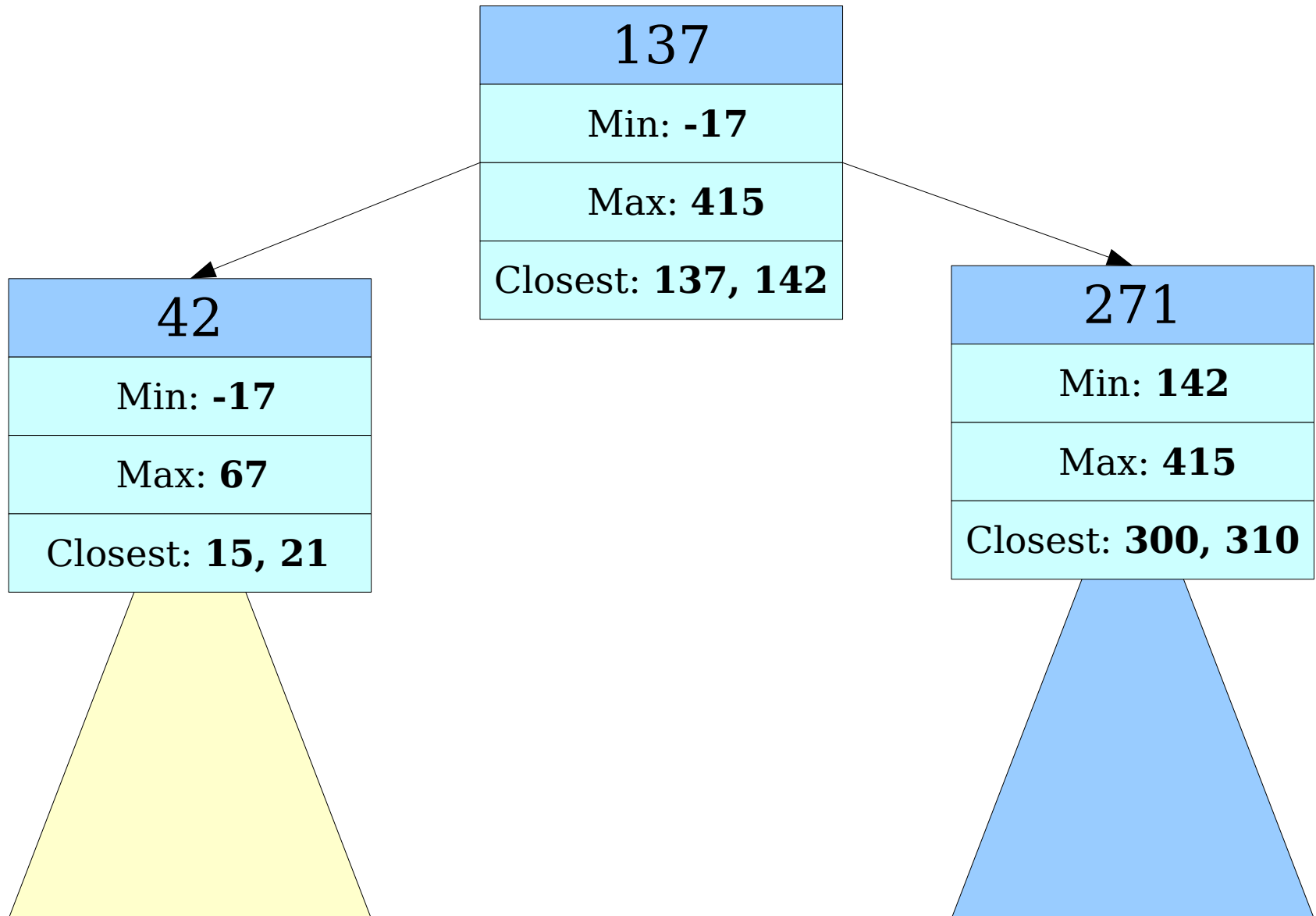
Dynamic 1D Closest Points



Dynamic 1D Closest Points



Dynamic 1D Closest Points



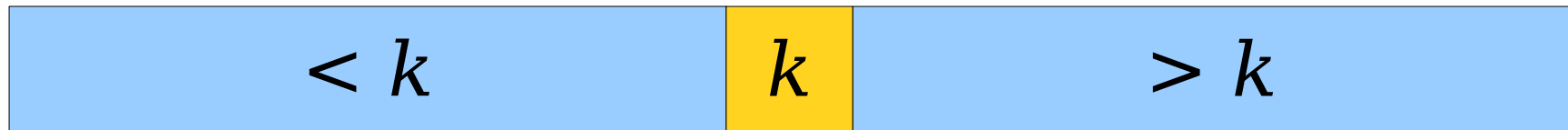
Some Other Questions

- How would you augment this tree so that you can efficiently (in time $O(1)$) compute the appropriate weighted averages?
- ***Trickier:*** Is this the fastest possible algorithm for this problem?
 - What if you're guaranteed that the keys are all integers in some nice range?

A Helpful Intuition

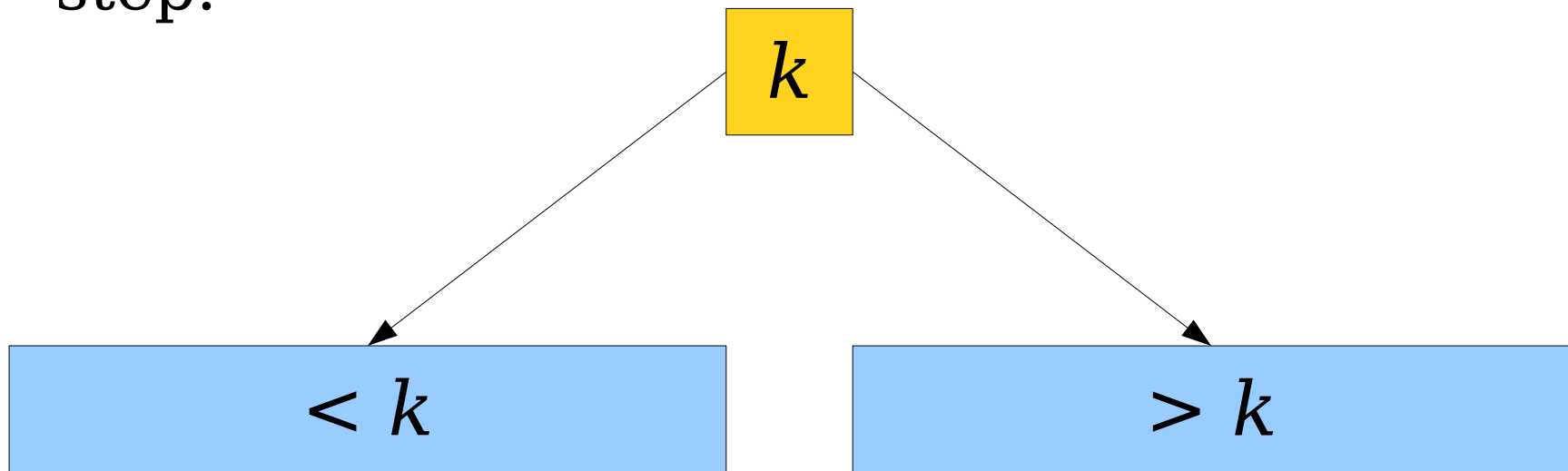
Divide-and-Conquer

- Initially, it can be tricky to come up with the right tree augmentations.
- ***Useful intuition:*** Imagine you're writing a divide-and-conquer algorithm over the elements and have $O(1)$ time per “conquer” step.



Divide-and-Conquer

- Initially, it can be tricky to come up with the right tree augmentations.
- Useful intuition:*** Imagine you're writing a divide-and-conquer algorithm over the elements and have $O(1)$ time per “conquer” step.



Next Time

- ***String Data Structures***
 - Storing and manipulating sequences.
- ***Tries and Patricia Trees***
 - Storing a collection of strings efficiently.
- ***Suffix Trees***
 - The Swiss Army Knife of strings.